EKNATHALABS · INTERVIEW PREP

Git in Production Environments

Platform, SRE & DevOps interview guide

Questions	**177 questions · spoken answers included**
Online (always latest)	**interviewhub.eknathalabs.com/git_production_interview_guide.html**
Printed	**2026-10-21**

Free to use and share for personal study. Answers are written for interview practice, so check command flags and versions against the official documentation before running anything in production.

certs.eknathalabs.com

Q1 How does Git store data internally — what is the object model?

Fundamentals

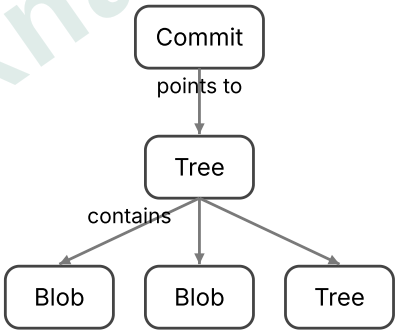
🎙 Spoken answer · ~87 s

Git is a content-addressable key-value store with a version control interface on top. Everything lives in the .git directory as objects identified by the SHA hash of their content. There are four object types. A blob stores the contents of a file, with no name or metadata. A tree represents a directory: a list of entries mapping names and modes to blobs or other trees. A commit points to one top-level tree, which is the snapshot of the project, to zero or more parent commits, and holds the author, committer, timestamp, and message. An annotated tag points to another object, typically a commit, with a name and message. Because identical content hashes to the same ID, Git deduplicates automatically, and because each commit's ID depends on its tree and parents, history is tamper-evident: changing anything changes every descendant hash. Objects start as loose files, zlib-compressed, then git gc packs them into packfiles with delta compression. Branches, tags, and HEAD are just small files containing hashes or references under .git/refs. The index, or staging area, is a binary file describing the next commit's tree. This model explains why branching is cheap, why rebase creates new commits rather than moving them, and why git recovers from mistakes via the reflog.

Likely follow-ups

- Why are branches so cheap in Git?
- What does a commit object contain?

🗺 Git object model



Git is a content-addressable filesystem. Every piece of data is stored as an object identified by its SHA-1 (now SHA-256 in new repos) hash of its content. There are four object types.

OBJECT TYPES

```
blob    — stores file content (not filename, not path)
```

```
tree    — stores directory listing: blob/tree references + names + permissions
```

```
commit  — stores tree pointer + parent commit(s) + author + message
```

```
tag     — annotated tag: pointer to commit + tagger + message
```

certs.eknathalabs.com

```
git cat-file -t <sha>          # show object type
```

```
git cat-file -p <sha>          # print object content
```

```
git cat-file -p HEAD           # inspect current commit
```

```
git cat-file -p HEAD^{tree}    # inspect current tree
```

```
find .git/objects -type f      # see all stored objects
```

A commit is a snapshot, not a diff. It stores a pointer to the full tree at that moment. The efficiency comes from sharing: if 99% of files are unchanged, those blobs are reused. Git computes diffs on the fly when you run git diff — it is not stored. This immutable, content-addressed model is what makes branching and merging cheap, and what makes rewriting history (rebase, amend) create new objects rather than modifying existing ones.

Interview depth: Understanding that a branch is just a file in .git/refs/heads/ containing a SHA, and HEAD is just a file containing either a branch name or a SHA (detached HEAD), reveals you understand Git at the plumbing level — not just the porcelain commands.

📚 **Official docs**

- Pro Git: Git internals (objects)
- Git documentation

certs.eknathalabs.com

Q2 What is the difference between git fetch, git pull, and git pull --rebase?

Fundamentals

🎙 Spoken answer · ~90 s

These commands differ in how they bring remote changes in. git fetch downloads new commits, branches, and tags from the remote and updates your remote-tracking references, such as origin/main, but doesn't touch your local branches or working tree. It's safe and lets you inspect what changed with git log or git diff before integrating. git pull is a fetch followed by an integration step into your current branch; by default that integration is a merge, so if both sides have new commits you get a merge commit, which can add noise to history. git pull --rebase fetches and then rebases your local commits on top of the updated remote branch, replaying them one by one, producing a linear history without merge commits. It's what many teams prefer for keeping feature branches tidy, and can be made default with pull.rebase true. Risks: rebase rewrites your local commits, so don't do it for commits that others already based work on; conflicts may need resolving per commit; and a merge-based pull preserves exact history. In production workflows I prefer to fetch explicitly, look at what came in, and then choose merge or rebase deliberately. Also pull.ff only is a safe setting that refuses to create unexpected merge commits. Understanding the difference prevents surprise merge commits and lost work.

Likely follow-ups

- Why might you prefer fetch over pull?
- When is pull --rebase risky?

```
git fetch origin              # download remote changes, do NOT modify working tree
```

```
git fetch origin main         # fetch specific branch only
```

```
git pull                      # fetch + merge (creates merge commit if diverged)
```

```
git pull --rebase             # fetch + rebase local commits on top of remote
```

```
git pull --rebase=interactive # fetch + interactive rebase (squash/reorder)
```

```
# Set rebase as default pull strategy:
```

```
git config --global pull.rebase true
```

```
# See divergence before pulling:
```

```
git fetch origin && git log --oneline HEAD..origin/main
```

git fetch is always safe — it updates remote-tracking branches (origin/main) but never touches your working directory or local branches. git pull = fetch + merge: if your local branch has diverged, it creates a merge commit, producing an ugly history. git pull --rebase replays your local commits on top of the fetched remote, keeping history linear. For feature branch development, --rebase is almost always preferred. Set pull.rebase=true globally to make this the default and avoid accidental merge commits.

certs.eknathalabs.com

Never rebase shared branches: git pull --rebase on a branch others are working on rewrites commit SHAs. Anyone else who has pulled that branch now has diverged history. Only rebase local commits that have not been pushed, or use rebase on your private feature branch before merging to main.

📚 **Official docs**

- git-rebase
- Git documentation

Q3 **Explain HEAD, staging area, and working tree — what are the three trees of Git?** Fundamentals

🎙 **Spoken answer · ~80 s**

Git has three main areas where your files live. The working tree is the actual directory of files you edit on disk. The staging area, also called the index, is a snapshot of what will go into the next commit: you add changes selectively with git add, even individual hunks with git add -p, so each commit can be a logical unit. The repository, or object database, holds committed history; HEAD is a pointer to the current commit, normally via a branch reference, so HEAD points to a branch, which points to a commit. Commands move data between these areas: git add copies from working tree to index; git commit turns the index into a commit and moves the branch forward; git restore --staged unstages; git restore file discards working tree changes; git checkout or switch updates HEAD and working tree; git diff compares working tree to index, git diff --staged compares index to HEAD, and git diff HEAD compares working tree to HEAD. The three reset modes map onto these: soft moves only the branch, mixed also resets the index, and hard resets the working tree too, which can discard uncommitted work.

Likely follow-ups

- What do the three modes of git reset change?
- How do you stage only part of a file?

```
Working Tree  — files on disk as you see them in your editor
```

```
Index (Stage) — snapshot prepared for next commit (.git/index)
```

```
HEAD          — pointer to last commit on current branch
```

```
git status    # compares all three trees
```

```
git diff                # working tree vs index (unstaged changes)
```

```
git diff --staged       # index vs HEAD (staged, what will be committed)
```

```
git diff HEAD           # working tree vs HEAD (all uncommitted changes)
```

certs.eknathalabs.com

```
cat .git/HEAD           # see what HEAD points to
```

```
git ls-files --stage    # show index contents
```

The three-tree model explains every Git operation: git add moves changes from working tree to index. git commit creates a new commit from the index and advances HEAD. git checkout branch moves HEAD to the branch and updates both index and working tree to match that commit. git reset --soft HEAD~1 moves HEAD back but leaves index and working tree untouched. git reset --hard HEAD~1 moves HEAD back AND resets index AND working tree — destructive. Understanding which trees each command touches is the key to avoiding data loss.

The mental model: Every Git command moves data between these three trees. Before running any destructive operation, ask: which of the three trees does this affect? git reset --hard affects all three (dangerous). git reset --soft affects only HEAD (safe). This framework prevents most common Git mistakes.

📚 **Official docs**

- Git documentation

Q4 **What is a detached HEAD state — how do you get into it and how do you get out?** Fundamentals

🎙 **Spoken answer · ~87 s**

Detached HEAD means HEAD points directly at a commit instead of at a branch name. Normally HEAD references a branch, and new commits move that branch. When detached, you can look around and make commits, but no branch tracks them, so once you switch away, those commits become unreachable and are eventually garbage collected, although the reflog remembers them for a while. How you get there: checking out a specific commit hash, git checkout v1.2.0 for a tag, checking out a remote branch like origin/main directly, or during operations such as a rebase, bisect, or submodule update. CI systems often check out a commit this way too. Getting out. If you made no changes worth keeping, just switch back with git switch main. If you made commits you want to keep, create a branch right where you are: git switch -c rescue-branch, or git branch rescue-branch, which attaches them to a name. If you already switched away and lost track, use git reflog to find the commit hash and create a branch from it. Detached HEAD is useful for inspecting old versions, testing a specific tag, or bisecting, as long as you know what you're doing. Git warns when you enter the state, and git status tells you.

Likely follow-ups

- How do you rescue commits made in a detached HEAD?
- Why is detached HEAD common in CI?

```
git checkout v1.2.3         # checking out a tag → detached HEAD
```

certs.eknathalabs.com

```
git checkout abc1234        # checking out a commit SHA → detached HEAD
```

```
cat .git/HEAD               # shows SHA instead of ref: refs/heads/branch
```

```
git status                  # "HEAD detached at abc1234"
```

```
# Recover: create a branch to save your work
```

```
git checkout -b my-experiment   # create branch at current detached HEAD
```

```
git switch -c my-experiment     # modern syntax
```

```
# Or discard: go back to a branch
```

```
git checkout main           # HEAD re-attaches to main branch
```

```
git switch main             # modern syntax
```

Detached HEAD means HEAD points directly to a commit SHA instead of a branch name. You can make commits in this state, but they are not on any branch — when you switch away, they become unreachable and will eventually be garbage collected (git gc). Any work done in detached HEAD state must be saved to a branch before switching away. Detached HEAD is not an error — it is intentional when inspecting historical commits, checking out tags for a build, or doing temporary experiments.

Recovering lost commits: If you made commits in detached HEAD and switched away, use git reflog to find the SHA of your last commit in that state, then git checkout -b rescue <sha> to create a branch at that point. git reflog keeps all recent HEADs for 30-90 days.

📚 **Official docs**

- Git documentation

certs.eknathalabs.com

Q5 What is git stash — how do you use it effectively in a team environment? Fundamentals

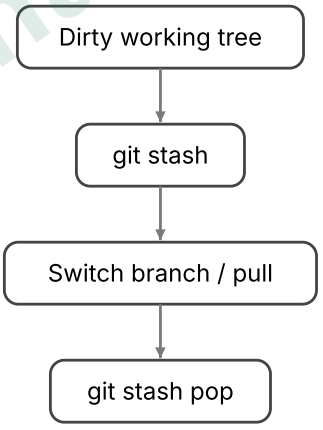
🎙 Spoken answer · ~90 s

git stash temporarily shelves your uncommitted changes, both staged and unstaged tracked changes, restoring a clean working tree so you can switch tasks, then lets you bring them back. Basic use: git stash push -m 'message' saves the changes; git stash list shows entries; git stash pop applies the latest and removes it, while git stash apply applies it and keeps the entry; git stash drop removes one. Useful flags: -u includes untracked files, -a adds ignored files, -p selects hunks interactively, and git stash push -- path stashes only specific files. You can create a branch from a stash with git stash branch, which is handy if it conflicts with later work. In a team setting, effective use means: stashes are local, not shared, so they're not a collaboration tool or a backup; always give descriptive messages, because stash@{0}-style names are easily forgotten; don't let stashes pile up, since old ones are hard to apply as code moves on; prefer a WIP commit on a branch for anything you want to keep more than a day, since commits are visible, pushable, and safer; and be careful with pop when conflicts occur, as the stash is kept in that case. Use stash for quick context switches like urgent hotfixes, or for pulling with uncommitted changes.

Likely follow-ups

- What is the difference between stash pop and apply?
- When is a WIP commit better than a stash?

🗺 Stash




```
git stash                      # stash tracked changes (index + working tree)
```

```
git stash push -u              # include untracked files
```

```
git stash push -m "WIP: auth refactor"  # named stash
```

```
git stash list                 # show all stashes
```

```
git stash show -p stash@{0}    # show diff of top stash
```

certs.eknathalabs.com

```
git stash pop                    # apply top stash + remove from stash list
```

```
git stash apply stash@{2}      # apply specific stash (keep in list)
```

```
git stash drop stash@{0}       # remove specific stash
```

```
git stash branch feature/new stash@{0}  # create branch from stash
```

```
git stash clear                  # drop ALL stashes (destructive)
```

Stash is a stack of partial work saves. Use it when you need to switch context quickly without committing incomplete work. In team environments, stash is local — stashes are not pushed to remote and are not visible to teammates. Stash pop can conflict if the stashed changes conflict with the current state of the branch — treat it like any other merge conflict. For longer-term WIP storage, create a WIP commit (git commit -m "WIP: [description]") and push it to a remote branch — this backs up work and is visible to the team.

Stash is not a backup: Stashes are stored in .git/refs/stash and are lost if you delete the repo or if something corrupts .git. For work you care about, commit to a branch and push. Never rely on stash as long-term storage for important work.

📚 **Official docs**

- git-stash
- Git documentation

certs.eknathalabs.com

Q6 What is a Git branch internally — how do branches and tags differ?

Branching

🎙 Spoken answer · ~88 s

Internally a branch is just a tiny file under .git/refs/heads whose content is the 40-character hash of the commit at its tip; creating a branch writes that one small file, which is why branching is nearly instant and cheap. When you commit while on a branch, Git creates the new commit and updates that file to point to it. HEAD normally holds the name of the current branch. A tag is also a reference, stored under .git/refs/tags, pointing to a commit, but it's meant to be fixed: it marks a specific point in history such as a release and doesn't move when you commit. Lightweight tags are just a ref to a commit; annotated tags are full objects with a tagger, date, message, and optional GPG signature, and are what you should use for releases. Branches move; tags stay. That difference matters operationally: you deploy and reference releases by tag, while branches represent ongoing lines of work. Tags aren't pushed by default; you push them explicitly with git push --tags or by name. Moving or deleting a published tag causes confusion because others may already have the old one, so protect release tags with rules. Remote-tracking branches like origin/main are references that record where the remote branch was at the last fetch.

Likely follow-ups

- Why should releases use annotated tags?
- What is a remote-tracking branch?

```
cat .git/refs/heads/main       # branch file: just a 40-char SHA
```

```
cat .git/refs/tags/v1.0.0      # lightweight tag: same as branch
```

```
git cat-file -p v1.0.0         # annotated tag: separate tag object
```

```
git branch                     # list local branches
```

```
git branch -vv                 # show tracking info + last commit
```

```
git branch -a                  # local + remote-tracking branches
```

```
git branch --merged main       # branches merged into main (safe to delete)
```

```
git branch --no-merged main    # branches NOT yet merged (caution deleting)
```

```
git tag -a v1.2.0 -m "Release 1.2.0"  # annotated tag (recommended)
```

```
git tag v1.2.0                 # lightweight tag (just a pointer)
```

A branch is simply a file containing a 40-character commit SHA. When you commit, Git updates that file to point to the new commit. Creating a branch costs almost nothing — one file write. Tags also point to commits, but annotated tags are tag objects (with tagger identity, date, message) pointing to the commit. The key

certs.eknathalabs.com

difference: branches move automatically with new commits; tags are permanently fixed to a specific commit. Use annotated tags for releases (they have metadata), lightweight tags for private/temporary markers.

Branch naming conventions: Consistent naming enables automation. Common patterns: feature/TICKET-123-description, fix/TICKET-456-bug-name, chore/update-dependencies, release/v1.2.0, hotfix/critical-payment-bug. CI/CD pipelines can use these patterns: deploy to staging on feature/*, deploy to production on release/*, auto-create GitHub releases on tags matching v*.*.*.

📚 **Official docs**

- Git documentation

Q7 What branching strategies exist — GitFlow vs trunk-based development vs GitHub Flow?

Branching

🎙 **Spoken answer · ~88 s**

These are three common strategies. GitFlow has long-lived main and develop branches, plus feature, release, and hotfix branches. Features merge into develop; a release branch is cut for stabilisation, then merged to main and tagged, and hotfixes branch from main. It suits products with scheduled, versioned releases and multiple supported versions, but it's heavy: many branches, big merges, delayed integration, and it conflicts with continuous delivery. GitHub Flow is simpler: main is always deployable; you create a short-lived feature branch, open a pull request, get review and CI, merge to main, and deploy. It fits web services that deploy continuously. Trunk-based development has developers integrate into a single trunk, main, at least daily, using very short-lived branches of a day or two, or committing directly, with incomplete work hidden behind feature flags. Releases are cut from trunk, or by tagging, with release branches only if needed. Research on DORA metrics links it to higher delivery performance, because small, frequent merges reduce conflicts and integration risk, but it needs strong automated testing, fast CI, feature flags, and discipline. Choosing: for continuously delivered services, I'd choose trunk-based or GitHub Flow; for packaged software with multiple versions in the field, a release-branch model. Whatever you choose, keep branches short-lived, protect main, and automate checks.

Likely follow-ups

- Why does trunk-based development need feature flags?
- When does GitFlow still make sense?

STRATEGY COMPARISON

```
GitFlow:       main + develop + feature/* + release/* + hotfix/*
```

```
GitHub Flow:   main + feature/* (PR → merge → deploy)
```

```
Trunk-Based:   main only + short-lived feature branches (merge daily)
```

certs.eknathalabs.com

```
Release Flow:  main + release/vX.Y branches for production fixes
```

GitFlow: well-defined for teams releasing versioned software (mobile apps, desktop software). Drawback: complex, long-lived branches cause integration pain, release branches create merge debt. GitHub Flow: simple two-tier model ideal for continuous deployment teams — branch off main, PR, merge, deploy. Every merge to main triggers a deployment. Trunk-Based Development: developers commit directly to main (or merge short-lived branches within 1-2 days). Requires feature flags to hide incomplete work. Enables true continuous integration — the most advanced practice used by Google, Facebook, Microsoft.

Platform Engineer recommendation: For a Platform Engineering team deploying infrastructure and tooling, trunk-based development or GitHub Flow is usually best — they enable fast iteration and simple CI/CD pipelines. GitFlow is overkill unless you have strict versioned releases or multiple concurrent release streams (like firmware or on-prem product versions). Match the strategy to release cadence: continuous delivery = trunk-based, versioned releases = GitFlow.

📚 **Official docs**

- Trunk Based Development
- Atlassian: Gitflow workflow

certs.eknathalabs.com

Q8 How do you protect the main branch in production — what branch protection rules matter?

Branching

🎙 Spoken answer · ~87 s

Protecting main means making sure only reviewed, tested code can reach production-bound history. Key rules on GitHub, GitLab, or similar. Require pull requests before merging, with no direct pushes, including for admins where possible. Require at least one or two approving reviews, and dismiss stale approvals when new commits are pushed, so approved code isn't silently changed. Require code owner review for sensitive paths via CODEOWNERS. Require status checks to pass, such as build, tests, linting, and security scans, and require branches to be up to date before merging, or use a merge queue to avoid semantic conflicts between concurrent merges. Require conversation resolution. Require signed commits where provenance matters. Require linear history if you want to disallow merge commits. Block force pushes and branch deletion, which protect history. Restrict who can push or merge. Apply rules to administrators, and use rulesets at organisation level so every repository gets the baseline. Also protect release tags. Consider requiring deployments to succeed before merge for some workflows. Balance: too many rules slow teams down, so keep checks fast and reliable, and provide an emergency bypass with audit logging for incidents. Monitor changes to protection settings themselves via audit logs, because disabling them is a common step in attacks. Document the policy.

Likely follow-ups

- Why dismiss stale approvals?
- What does a merge queue solve?

```
# GitHub Branch Protection Rules (Settings → Branches → Add rule):

Require pull request before merging

Require approvals: minimum 1 (production: 2)

Dismiss stale pull request approvals when new commits are pushed

Require review from Code Owners (CODEOWNERS file)

Require status checks to pass before merging:

  - CI tests (unit, integration)

  - Security scan (Trivy, Snyk)

  - Lint / code quality

Require branches to be up to date before merging
```

certs.eknathalabs.com

```
Require signed commits
```

```
Do not allow bypassing the above settings (even admins)
```

```
Restrict who can push to matching branches
```

Branch protection rules enforce your deployment gates at the VCS level — they cannot be bypassed by individuals (when "do not allow bypassing" is checked). CODEOWNERS file maps paths to required reviewers: infrastructure changes require the platform team, security-sensitive files require the security team. "Dismiss stale approvals" prevents the attack where code is approved, then bad commits are added after approval. Require status checks ensures no merge without passing CI — this is the first line of defense against broken deployments.

> **Admin bypass:** Repository admins can bypass branch protection by default. Enable "Do not allow bypassing the above settings" to protect even from admin push. For compliance (SOC2, PCI-DSS), this setting is required — auditors look for it specifically. Create a separate break-glass procedure for genuine emergencies that is logged and reviewed.

> 📚 **Official docs**
>
> - GitHub: Protected branches

certs.eknathalabs.com

Q9 How do you handle long-running feature branches — what problems do they create?

Branching

🎙 Spoken answer · ~88 s

Long-running feature branches diverge from main over time, and the longer they live, the more problems they create. Merge conflicts grow, and are larger and harder to resolve, often by someone who no longer remembers the context. Integration risk piles up: the first time the feature meets everyone else's changes is at the end, so surprises appear late, such as incompatible refactors or broken assumptions. Review becomes difficult, because a giant pull request with thousands of lines gets superficial review. Testing happens on a stale base, so passing CI on the branch doesn't mean it will pass on main. Work is invisible to the rest of the team, causing duplication and design drift. Releases get delayed and morale suffers on big-bang merges. Mitigations. Prefer short-lived branches and small pull requests merged within a day or two. Use feature flags or branch by abstraction to merge incomplete work to main safely, hidden from users. Slice features into vertical increments that can ship independently. If a long branch is unavoidable, merge or rebase main into it frequently, such as daily, to keep the gap small; consider git rerere to reuse conflict resolutions; set up CI against the integration of the branch and main; and agree on the design early with smaller design reviews.

Likely follow-ups

- How do feature flags avoid long-lived branches?
- What does git rerere do?

```
# Long-lived branch problems:
```

```
git log --oneline main..feature/big-refactor | wc -l  # 200 commits behind main
```

```
git diff main...feature/big-refactor --stat            # massive diff
```

```
# Keep feature branch updated:
```

```
git fetch origin
```

```
git rebase origin/main                 # replay feature commits on latest main
```

```
# Or merge main into feature (creates merge commits but is safer):
```

```
git merge origin/main                  # merge latest main into feature
```

```
# Feature flags to ship incomplete code:
```

```
if feature_enabled("new-payment-flow", user_id):
```

```
    return new_payment_flow()
```

certs.eknathalabs.com

```
return legacy_payment_flow()
```

Long-lived feature branches accumulate merge debt — the longer a branch lives, the more diverged it becomes from main, and the harder the eventual merge. Problems: large conflict resolution effort, integration surprises (your feature works alone but not with other merged features), delayed feedback (bugs discovered late). Solutions: sync the feature branch with main daily (rebase or merge), break large features into smaller mergeable increments, and use feature flags to merge incomplete code to main safely while hiding it from users. Feature flags decouple deployment from feature release — the ultimate solution to long-lived branches.

Branch age SLO: Set a team agreement: no feature branch lives longer than 2-3 days without being merged or rebased onto main. A branch older than 1 week is a smell — the feature should be broken down smaller or feature-flagged. Enforcing this SLO prevents the "big bang" merge nightmare.

📚 **Official docs**

- Atlassian: Gitflow workflow

Q10 What are Git worktrees — how do they help when working on multiple branches simultaneously?

Branching

🎙 **Spoken answer · ~86 s**

A Git worktree lets one repository have multiple working directories, each with a different branch checked out, sharing the same object database. Create one with git worktree add ../hotfix hotfix-branch, or git worktree add -b new-branch ../path origin/main. Each worktree has its own working files and index and HEAD, but they share history, so there's no duplicate clone and no extra disk for objects, and commits made in one are immediately visible to others. Benefits. Parallel work without stashing: while in the middle of a feature, you can handle an urgent hotfix in a separate directory without disturbing your uncommitted changes. Reviewing a pull request by checking it out in its own directory, running tests while continuing your own work. Running long builds or tests on one branch while editing another. Comparing behaviour across versions side by side. CI or tooling that needs multiple checkouts. Constraints: a branch can only be checked out in one worktree at a time, which prevents conflicting edits; worktrees must be removed with git worktree remove, or pruned with git worktree prune if the directory was deleted manually; submodules have some limitations; and build artefacts and dependencies like node_modules are per directory, so each needs installing. git worktree list shows them all.

Likely follow-ups

- Why can't a branch be checked out in two worktrees?
- How do worktrees compare to multiple clones?

```
git worktree add ../hotfix-123 hotfix/payment-crash  # checkout branch in new dir
```

certs.eknathalabs.com

```
git worktree add -b feature/new-api ../new-api origin/main  # new branch + dir
```

```
git worktree list                          # show all worktrees
```

```
git worktree remove ../hotfix-123          # remove worktree
```

```
git worktree prune                         # clean up stale worktree metadata
```

Worktrees let you check out multiple branches simultaneously in different directories, all sharing the same .git repository (no duplicate .git overhead). Use cases: context switching between a feature branch and an urgent hotfix without stashing, comparing two branches side by side, running tests on two branches concurrently. Without worktrees, you would need to stash, switch, work, switch back, unstash — error-prone and slow. With worktrees, each branch is a separate directory you can switch between instantly.

Practical pattern: Keep your main working directory on the main branch (for reference). Create worktrees for each active feature. Your editor can open multiple directories. When you finish a worktree, run git worktree remove to clean up. This workflow eliminates stash entirely for multi-branch work.

📚 **Official docs**

- git-worktree
- Git documentation

certs.eknathalabs.com

Q11 What is the difference between merge and rebase — which do you use when?

Merging & rebasing

🎙 Spoken answer · ~83 s

Both integrate changes from one branch into another, but with different history shapes. Merge creates a new merge commit with two parents, tying the histories together; it preserves the full, true history including when branches diverged, is non-destructive, and doesn't rewrite any existing commits. The downside is a more complex, non-linear history with many merge commits. Rebase takes your branch's commits and replays them on top of another base, creating new commits with new hashes; the result is a clean linear history, as if you'd started from the latest main. It rewrites history, so it's dangerous on shared branches: if others have based work on the old commits, force pushing rebased ones causes duplicates and confusion. The golden rule: never rebase commits that have been pushed and shared, unless the branch is yours alone. When I use each. Rebase my own local or personal feature branch onto main to update it and tidy commits with interactive rebase before opening a pull request. Merge, or squash merge, when integrating the finished feature into main through the pull request, depending on the team's history preference. Merge when updating a shared branch, because it never rewrites. Use fast-forward only merges for linear history.

Likely follow-ups

- What is the golden rule of rebasing?
- Why does rebase change commit hashes?

```
# Merge: preserves history, creates merge commit
```

```
git checkout main
```

```
git merge feature/login        # creates merge commit (two parents)
```

```
# Rebase: rewrites history, linear result
```

```
git checkout feature/login
```

```
git rebase main                # replay feature commits on top of main
```

```
git checkout main
```

```
git merge feature/login        # now fast-forward only (no merge commit)
```

```
# Fast-forward merge (when no divergence):
```

```
git merge --ff-only feature    # fail if not fast-forwardable
```

```
# No-ff merge (always create merge commit):
```

certs.eknathalabs.com

```
git merge --no-ff feature      # preserves feature branch boundary
```

Merge preserves the true history — you can see when branches diverged and merged. Merge commits are noise in a linear history but accurately represent what happened. Rebase rewrites history to be linear — all commits appear as if they were made sequentially on main. Rebase produces cleaner git log output but obscures the true timeline. Rule of thumb: rebase your private feature branch before merging to main (clean history), never rebase commits already pushed to shared branches (rewrites history others depend on). Many teams use squash-and-merge for PRs (one clean commit per feature) as a middle ground.

The golden rule of rebase: Never rebase commits that exist outside your local repository. If others have based work on those commits, rebasing creates parallel histories with the same changes under different SHAs — the definition of a nightmare merge situation.

📚 **Official docs**

- git-rebase

Q12 How do you resolve merge conflicts — what is your systematic approach?

Merging & rebasing

🎙 **Spoken answer · ~89 s**

Conflicts happen when Git can't automatically combine changes that touch the same lines. My systematic approach. First, understand the situation: git status lists conflicted files, and I check what operation is in progress, merge, rebase, or cherry-pick, and which branches are involved. Before resolving, know the intent of both sides: read git log for each side's commits on those files, and talk to the other author if the logic is unclear. Open each file and look at the markers: the section between the start marker and the separator is my side, the section until the end marker is theirs; a diff3 conflict style, set with merge.conflictStyle diff3 or zdiff3, also shows the common ancestor, which makes the intent much clearer. Resolve deliberately: combine both changes when both are needed, instead of picking one blindly, remove all markers, and then check the result compiles and makes sense. Use a good three-way merge tool, such as VS Code, or git mergetool, for complex cases. Stage the resolved files with git add, and continue with git merge --continue or git rebase --continue. Crucially, run the build and tests after resolving, because textually clean merges can still be semantically broken. If it goes wrong, abort and restart with --abort. Reduce future conflicts with small, frequent merges and rerere.

Likely follow-ups

- What does diff3 style add to conflict markers?
- Why must you test after a clean merge?

```
git merge feature/payments      # triggers conflict
```

```
git status                      # see conflicting files (both modified)
```

certs.eknathalabs.com

```
git diff                        # show conflict markers in files
```

```
# Conflict markers in file:
```

```
<<<<<<< HEAD (current branch)
```

```
current branch version
```

```
=======
```

```
incoming branch version
```

```
>>>>>>> feature/payments
```

```
# Use a merge tool:
```

```
git mergetool                   # opens configured diff tool (vimdiff, VS Code, etc.)
```

```
git config merge.tool vscode
```

```
# Accept one side entirely:
```

```
git checkout --ours   file.txt  # keep current branch version
```

```
git checkout --theirs file.txt  # keep incoming branch version
```

```
git add file.txt
```

```
git merge --continue
```

```
# Abort and start over:
```

```
git merge --abort
```

certs.eknathalabs.com

Systematic conflict resolution: (1) understand what each side is trying to do (read the commits, not just the diff), (2) decide on the correct semantics (not just text), (3) resolve with proper understanding of both changes, (4) run tests after resolution. Mechanical resolution (accept all ours/theirs) is dangerous — the result may compile but be logically wrong. Configure a visual merge tool (VS Code, IntelliJ, vimdiff) — three-way diff showing base/ours/theirs makes resolution faster and safer.

rerere (Reuse Recorded Resolution): git config --global rerere.enabled true — Git records how you resolved conflicts and automatically reapplies the same resolution if the same conflict occurs again (common during long rebases). Essential for large rebases with recurring conflict patterns.

📚 **Official docs**

- git-merge

Q13 What is a squash merge — when is it appropriate vs a regular merge?

Merging & rebasing

🎙 Spoken answer · ~82 s

A squash merge takes all the commits from a feature branch and combines them into a single new commit on the target branch, without preserving the individual commits or a merge commit; the branch's history is not linked as ancestry. When it is appropriate. When the branch has noisy, work-in-progress commits like 'fix typo' and 'try again', which add no value to main history, so you want one clean commit per pull request with a well-written message. It keeps main's history linear and readable, with each commit mapping to a pull request, which makes reverting a whole feature a one-commit operation and simplifies release notes and bisecting by feature. It's the default in many teams on GitHub. When a regular merge is better. When the branch contains meaningful, well-structured commits worth preserving for blame and bisect granularity; with long-lived or large branches, where squashing makes one huge commit that hides detail; when several people collaborated on the branch and you want to retain attribution of individual commits, though co-authored-by trailers can help; or when other branches are based on the feature branch, since squashing makes the original commits unrelated to main and can produce confusing conflicts afterwards.

Likely follow-ups

- Why does squashing complicate branches stacked on top?
- How do you preserve attribution when squashing?

```
git merge --squash feature/login    # stage all feature changes as one commit
```

```
git commit -m "feat: add login with OAuth2 (#PR-456)"
```

```
# GitHub/GitLab: Squash and Merge button in PR UI
```

```
# Check if PR was squashed:
```

```
git log --oneline main | head -5    # clean single commits per feature
```

```
# Squash local commits before pushing:
```

```
git rebase -i HEAD~5                # interactive rebase last 5 commits
```

```
# then change pick to squash/s for commits to collapse
```

Squash merge collapses all commits from a feature branch into a single commit on main. Advantages: clean linear history on main (one meaningful commit per feature), individual commits on feature branches can be messy WIP commits without polluting main. Disadvantages: you lose the detailed commit history of the feature (all those "fix typo" and "WIP" commits are gone), making blame/bisect less granular. Use squash-

certs.eknathalabs.com

and-merge for: small features where the PR is the unit of change, teams with messy commit discipline. Use regular merge with --no-ff for: large features where individual commits provide meaningful history, or when you want to preserve detailed blame information.

Team consistency: Pick one merge strategy and enforce it. If you use squash-and-merge, every PR becomes one commit — consistent and clean. If you use regular merge, require conventional commits (feat:, fix:, chore:) so the history is meaningful. Mixing strategies makes the history confusing.

📚 **Official docs**

- git-rebase

Q14 How does git cherry-pick work — what are its risks in production?

Merging & rebasing

🎙 **Spoken answer · ~84 s**

git cherry-pick takes the changes introduced by one or more specific commits and applies them as new commits on the current branch. Internally it computes the diff of the chosen commit relative to its parent and applies that patch, creating a new commit with a different hash but the same message and author, and a new committer. Use it to backport a fix from main to a release branch, to pull a single commit from a feature branch, or to recover a commit from elsewhere. Syntax: git cherry-pick <hash>, ranges with A..B, and -x appends 'cherry picked from commit' to the message, which is useful for traceability in release branches; -n applies without committing; and merge commits need -m to choose a parent. Risks in production. Duplicate commits: the same change exists with two hashes, so when the branches are later merged, Git may handle it fine, but sometimes produces conflicts or confusing history, and git log can't tell they're the same. Missing context: the commit may depend on earlier changes that were not picked, so it applies cleanly but behaves wrongly, or conflicts. Drift between branches: forgetting to cherry-pick a fix leaves branches inconsistent. Hard to audit what's on which branch.

Likely follow-ups

- Why use the -x flag?
- What is the risk of cherry-picking dependent commits?

```
git cherry-pick abc1234         # apply single commit to current branch
```

```
git cherry-pick abc1234..def5678 # apply range of commits
```

```
git cherry-pick -x abc1234      # add "(cherry picked from commit abc)" to message
```

```
git cherry-pick --no-commit abc1234  # stage changes without committing
```

```
git cherry-pick --abort         # abort if conflicts
```

```
# Common use: backport a fix to a release branch
```

certs.eknathalabs.com

```
git checkout release/v1.2
```

```
git cherry-pick <fix-commit-sha>
```

```
git push origin release/v1.2
```

Cherry-pick copies a commit to another branch, creating a new commit with the same changes but a different SHA. Production use cases: backporting a security fix to multiple release branches, hotfixing production without pulling in other in-progress changes, picking a fix from a feature branch before the full feature is ready. Risks: cherry-picked commits create duplicate history — the same logical change exists as two different SHAs in two branches, causing confusion in git log and git blame. If the cherry-picked commit depends on other commits for context, applying it in isolation can cause subtle bugs.

Cherry-pick debt: Heavy cherry-pick usage across multiple release branches creates significant merge debt. Backports must be carefully tracked — use the -x flag to leave a trail in the commit message. Some teams use a bot (like GitHub's backport action) to automate cherry-picks to release branches when a fix is merged to main.

📚 **Official docs**

- git-cherry-pick
- Git documentation

certs.eknathalabs.com

Q15 How do you perform an interactive rebase — what operations does it support?

Merging & rebasing

🎙 Spoken answer · ~88 s

Interactive rebase, git rebase -i <base>, opens an editor with a list of commits from the base to HEAD, each with a command, and Git replays them according to your edits. It rewrites history, so use it on commits you haven't shared, or on your own branch. Supported operations: pick, which keeps the commit as is; reword, to change the commit message; edit, to stop at that commit so you can amend its content or split it into several; squash, to combine it with the previous commit and merge the messages; fixup, which is like squash but discards the message; drop, to remove the commit; reordering, by moving lines; exec, to run a command after a commit, for example running tests on every commit to verify each one builds; and in recent versions, label, reset, and merge for rebasing merge structures, plus update-ref for stacked branches. Practical use: before opening a pull request, tidy messy work-in-progress commits into a logical, reviewable sequence; fix up commits using git commit --fixup <hash> and then git rebase -i --autosquash, which automatically arranges and squashes them; and split a large commit with edit and git reset HEAD^. If things go wrong, git rebase --abort restores the original state, and the reflog can recover from mistakes.

Likely follow-ups

- What does autosquash do with fixup commits?
- Why use force-with-lease?

```
git rebase -i HEAD~5            # interactive rebase last 5 commits
```

```
git rebase -i origin/main       # rebase commits not yet on main
```

```
# Interactive rebase operations:
```

```
pick    — use commit as-is
```

```
reword  — use commit, edit message only
```

```
edit    — pause to amend commit (add/remove files)
```

```
squash  — meld into previous commit (keep both messages)
```

```
fixup   — meld into previous commit (discard this message)
```

```
drop    — remove this commit entirely
```

```
exec    — run shell command after this commit (for testing)
```

```
break   — pause here (for manual inspection)
```

certs.eknathalabs.com

```
# Reorder commits by reordering lines in the editor
```

```
# After editing: git rebase --continue or git rebase --abort
```

Interactive rebase lets you rewrite the last N commits before pushing. Use it to: squash WIP/fixup commits before a PR, reorder commits for logical clarity, edit commit messages to follow conventions, remove a commit that was accidentally included, or split a large commit into smaller ones (edit, then unstage and recommit in parts). Run it only on commits that haven't been pushed to shared branches. Many developers run git rebase -i origin/main before every PR to clean up their commit history.

exec for safety: Add exec make test between commits during a rebase to verify tests pass at each step. If a test fails mid-rebase, Git pauses and lets you fix it before continuing. This is invaluable when rebasing complex branches where order matters.

📚 **Official docs**

- git-rebase

Q16 **How do you write good commit messages — what conventions matter in production?** History & commits

🎙 **Spoken answer · ~83 s**

Good commit messages explain what changed and, most importantly, why, so future readers, including you during an incident, can understand the intent without reading the whole diff. Conventions. A concise subject line of around fifty characters or fewer, in the imperative mood, such as 'Fix race condition in cache invalidation', followed by a blank line and a body wrapped at about seventy-two characters, explaining the motivation, the approach, trade-offs, and side effects, and referencing issues or tickets. The diff shows how; the message explains why. Keep each commit atomic, a single logical change that builds and passes tests, so it can be reverted, cherry-picked, or bisected cleanly. Many teams adopt Conventional Commits, a structured format like feat, fix, docs, refactor, with optional scope and a breaking change marker, which enables automated changelogs and semantic version bumps. Include trailers such as Co-authored-by, Signed-off-by for developer certificate of origin, and issue links. Avoid vague messages like 'update' or 'fixes', and avoid mixing unrelated changes. In production environments, good messages speed up incident investigation with git blame and log, support audits, and make release notes easier. Enforce conventions with a commit-msg hook or CI check such as commitlint, and with pull request templates.

Likely follow-ups

- Why imperative mood in the subject?
- What does Conventional Commits enable?

```
# Conventional Commits format:
```

certs.eknathalabs.com

```
type(scope): short description (50 chars max)

Longer body explaining WHAT and WHY (not HOW), 72 chars per line.
HOW is in the code. WHY is in the commit message.

Closes #123
# Types: feat, fix, docs, style, refactor, test, chore, ci, perf
# Examples:
feat(auth): add OAuth2 login with Google provider
fix(payments): prevent double-charge on network timeout
chore(deps): upgrade axios from 0.27 to 1.6.0
ci(github-actions): add matrix build for node 18 and 20
# Breaking change:
feat!: remove deprecated v1 API endpoints
BREAKING CHANGE: /api/v1 endpoints removed, use /api/v2
```

Good commit messages are engineering documentation. The subject line answers "what does this commit do?" — use imperative mood ("add feature" not "added feature"). The body answers "why was this change made?" — context that cannot be inferred from the code. Conventional Commits enables automation: semantic-release auto-generates changelogs and version bumps based on commit type (feat → minor bump, fix → patch, BREAKING CHANGE → major). CI/CD pipelines can enforce message format with commitlint.

The "why" is priceless: "Fix off-by-one error in loop" describes what. "Fix off-by-one error in loop causing silent data loss when processing batches at partition boundaries (found via Datadog alert at 2024-01-15 02:30 UTC)" tells you why anyone should care. Three months later, that context is gold for whoever bisects a regression to this commit.

📚 **Official docs**

- Conventional Commits

certs.eknathalabs.com

Q17 How do you use git log effectively to navigate project history?

History & commits

🎙 Spoken answer · ~85 s

git log is the main tool for exploring history, and its options make it far more powerful than the default. Compact views: --oneline for short hashes and subjects, --graph --decorate --all to see the branch structure with labels, and --stat or --patch for file summaries or full diffs. Filtering: by author with --author, date ranges with --since and --until, message text with --grep, file or directory by appending a path after --, and limiting count with -n. Searching the content of changes: -S 'string', the pickaxe, finds commits that added or removed that string, and -G with a regex finds changes matching a pattern, ideal for finding when a function or config value was introduced or removed; -L :funcname:file or -L start,end:file shows the history of a function or line range. Comparing branches: git log main..feature lists commits in feature not in main, and A...B with three dots shows commits unique to either side; --no-merges hides merge commits, --first-parent follows only the mainline, useful for seeing what landed on main; --follow tracks a file across renames. Custom output with --pretty=format. Combine with git show for a single commit, git diff for ranges, and git shortlog for contributor summaries. Define aliases for your favourite views.

Likely follow-ups

- What does the pickaxe option find?
- What does --first-parent show?

```
git log --oneline --graph --all --decorate  # visual branch graph
```

```
git log --since="2 weeks ago" --author="Eknatha"
```

```
git log --grep="payment" --oneline  # search commit messages
```

```
git log -S "function processPayment"  # search for code added/removed (pickaxe)
```

```
git log -G "processPayment"          # search using regex (pickaxe)
```

```
git log --follow -p -- src/auth.ts   # history of a specific file (follows renames)
```

```
git log main..feature/login          # commits on feature not yet on main
```

```
git log feature/login..main          # commits on main not yet on feature
```

```
git log --merges --oneline           # only merge commits
```

```
git shortlog -sn                     # contributor commit count summary
```

```
git log --format="%H %ai %ae %s" --since="1 month ago" > audit.log
```

certs.eknathalabs.com

git log -S (the "pickaxe") is one of the most powerful debugging tools — it finds every commit that added or removed a specific string, even across renames and moves. Use it to answer "when did this function first appear?" or "which commit deleted this configuration key?". git log --follow tracks a file through renames. Two-dot notation (A..B) shows commits reachable from B but not A — what's unique to one branch relative to another. Three-dot (A...B) shows commits reachable from either but not both — symmetric difference.

📚 **Official docs**

- Git documentation

git log aliases: Add to ~/.gitconfig: alias.lg = log --oneline --graph --all --decorate. Then git lg gives a beautiful branch graph. Share aliases across the team via a dotfiles repo to standardize the Git experience.

Q18 **What is git blame — how do you use it productively for incident investigation?**

History & commits

🎙 **Spoken answer · ~85 s**

git blame shows, for each line of a file, the commit, author, and date that last modified it. Use it as an entry point to history, not to assign fault. Basic: git blame file; with -L start,end for a range or a function; -w ignores whitespace changes; -M detects lines moved within a file and -C detects lines moved or copied from other files, so the blame points to the original author instead of a refactoring commit. Mass reformatting or renaming commits pollute blame, so a .git-blame-ignore-revs file, configured via blame.ignoreRevsFile, lists those commits to skip. Productive use during incident investigation: identify the line that looks suspicious, blame it to find the commit, then read the commit message, diff, and linked pull request or ticket to understand intent and context, which tells you whether the behaviour was deliberate. Check when it was introduced relative to when the incident started, correlating with deployment times. Then look at related commits in that pull request and whether tests were added. Don't stop at the last change: use git log -L or follow blame to the parent commit with git blame <commit>^ -- file to see earlier history. Use the web UI's blame view with history navigation for convenience.

Likely follow-ups

- What problem does .git-blame-ignore-revs solve?
- How does -C improve blame accuracy?

```
git blame src/payments/processor.go          # annotate each line with last commit
```

```
git blame -L 45,60 src/payments/processor.go  # specific line range
```

```
git blame -w src/payments/processor.go       # ignore whitespace changes
```

```
git blame -M src/payments/processor.go       # detect moved lines within file
```

```
git blame -C src/payments/processor.go       # detect lines copied from other files
```

certs.eknathlabs.com

```
# If blame shows a refactor commit (not the original change):
```

```
git log -p <refactor-commit> -- src/payments/processor.go
```

```
# Then blame at the parent of that commit:
```

```
git blame <refactor-commit>^ -- src/payments/processor.go
```

```
# GitHub: "View blame" then "View as of this commit" to walk back through history
```

git blame is most useful for incident investigation: "which commit introduced this behavior?" and "who can explain this code?". The -w flag prevents refactoring commits (whitespace/formatting changes) from obscuring the actual logic author. -M and -C detect when code was moved or copied, attributing to the original author rather than whoever moved it. When blame shows a trivial commit (formatting, rename), use git log on that commit to trace back to the original change.

Blame culture: git blame is a code archaeology tool, not a blame assignment tool. Use it to understand context and find the person who can explain the code — not to assign fault. "Who wrote this?" should lead to "let me ask them about the context" not "I found who broke it."

📚 **Official docs**

- Git documentation

certs.eknathalabs.com

Q19 What is git bisect — how do you use it to find which commit introduced a bug?

History & commits

🎙 Spoken answer · ~83 s

git bisect finds the commit that introduced a bug by performing a binary search through history. You tell Git a known bad commit, usually the current one, and a known good commit; Git checks out the midpoint; you test and mark it good or bad; and it halves the range each time. With a thousand commits between good and bad, you need only about ten tests. Manual workflow: git bisect start, git bisect bad, git bisect good <hash>, then repeat the test and mark each step with git bisect good or bad, and when Git reports the first bad commit, run git bisect reset to return to where you were. It can be fully automated: git bisect run <script>, where the script exits with zero for good, one to 124 or 126 and 127 excluded for bad, and 125 to skip an untestable commit; Git runs it at every step, which is ideal with a failing test or a reproduction script. Tips: write a reliable, fast reproduction test first; make sure the build works at each commit, using skip for broken ones; with a messy merge-heavy history, --first-parent can help; and flaky tests mislead the search, so verify the result.

Likely follow-ups

- How many steps does bisecting 1,000 commits take?
- What does exit code 125 mean in bisect run?

```
git bisect start
```

```
git bisect bad                  # current commit has the bug
```

```
git bisect good v1.2.0          # last known good version
```

```
# Git checks out the middle commit — test it:
```

```
make test                       # does the bug exist?
```

```
git bisect good                 # no bug here
```

```
git bisect bad                  # bug present
```

```
# Repeat until Git identifies the first bad commit
```

```
git bisect reset                # return to original HEAD when done
```

```
# Automated bisect with a test script:
```

```
git bisect start HEAD v1.2.0
```

```
git bisect run ./test-for-bug.sh # 0=good, non-zero=bad
```

certs.eknathalabs.com

git bisect uses binary search through commit history to find the first bad commit. With 1000 commits between good and bad, it takes only ~10 steps (log2(1000)). The automated form (git bisect run) is the most powerful: provide a script that returns 0 if the bug is absent, non-zero if present, and Git binary-searches automatically. This reduces what would be hours of manual investigation to minutes. Essential for production incident root cause analysis when you know the bug was introduced between two versions.

Production use: When a Kubernetes cluster upgrade introduces a regression, or a dependency update breaks behavior, git bisect against a tag range (git bisect start HEAD v1.8.0) with an automated test script finds the exact commit. Even complex bugs become tractable — the exponential reduction in search space is transformative.

📚 **Official docs**

- git-bisect
- Git documentation

Q20 **How do you amend commits — what is the difference between amend, fixup, and reset?** History & commits

🎙 **Spoken answer · ~84 s**

These tools all modify recent history, with different scopes. git commit --amend replaces the most recent commit: it lets you change its message or add forgotten changes by staging them and running amend, producing a new commit with a new hash. Use --no-edit to keep the message. It's safe if the commit isn't pushed or shared; otherwise it requires force pushing. A fixup is a commit created with git commit --fixup <hash> that's meant to correct an earlier commit in the branch, not just the latest; later, git rebase -i --autosquash automatically reorders and squashes it into its target. This is great during code review: you push fixup commits so reviewers see the incremental changes, then squash before merging. git reset moves the current branch pointer to another commit: --soft keeps the changes staged, --mixed, the default, keeps them in the working tree unstaged, and --hard discards them. To undo the last commit but keep the work, use git reset --soft HEAD~1 and recommit differently, or to split a commit apart. Difference: amend and fixup modify content of existing commits, reset moves the branch pointer and is lower-level; reset --hard is destructive for uncommitted work, though the reflog can recover committed states.

Likely follow-ups

- Why is fixup useful during code review?
- What is the difference between reset --soft and --mixed?

certs.eknathalabs.com

```
# Amend last commit (message or content):
```

```
git add forgotten-file.txt
```

```
git commit --amend              # opens editor for message change
```

```
git commit --amend --no-edit    # keep message, just add staged changes
```

```
# Fixup commit (for interactive rebase cleanup):
```

```
git commit --fixup abc1234      # create "fixup! <original message>" commit
```

```
git rebase -i --autosquash origin/main  # auto-squashes fixups into targets
```

```
# Reset to unstage or undo commits:
```

```
git reset HEAD~1                # undo last commit, keep changes staged (--mixed)
```

```
git reset --soft HEAD~1         # undo last commit, keep changes staged
```

```
git reset --hard HEAD~1         # undo last commit, DISCARD changes (destructive)
```

```
git reset HEAD file.txt         # unstage a specific file
```

Amend rewrites the last commit — creates a new SHA. Use it before pushing; after pushing requires force-push. Fixup commits are WIP commits meant to be squashed into a specific earlier commit — they follow a naming convention ("fixup! original commit message") that --autosquash understands. Reset --soft is the safest reset: undoes the commit but keeps all changes staged for recommitting. Reset --mixed (default) unstages changes but keeps them in working tree. Reset --hard destroys changes — use with extreme care and verify with git diff HEAD first.

After pushing: git commit --amend followed by git push rewrites published history. Force-push is required. On shared branches this is catastrophic — anyone who has pulled the original commit now has a diverged history. Only amend commits that exist only in your local repository or on a private feature branch no one else has pulled.

📚 **Official docs**

- git-rebase

certs.eknathalabs.com

Q21 How does the pull request / merge request workflow work in practice — what makes a good PR?

Workflows

🎙 Spoken answer · ~87 s

A pull request, or merge request, is a proposal to merge a branch into another, with a space for review, discussion, and automated checks. Typical workflow: create a short-lived branch from main, make commits, push, and open the PR describing the change; CI runs builds, tests, linters, and security scans; reviewers comment, request changes, or approve; the author updates the branch; once approvals and checks pass, it's merged, by merge, squash, or rebase according to policy, and the branch is deleted; deployment may follow automatically. What makes a good PR. Small and focused: one logical change, ideally a few hundred lines at most, because review quality drops sharply with size. A clear title and description: what and why, links to the ticket, how it was tested, screenshots for UI changes, risks, and rollout or rollback notes. Self-reviewed by the author before requesting review, with a green CI build. Well-structured commits that tell a story. Tests included for new behaviour. Backward-compatible and safe to deploy, behind a flag if incomplete. Reviewers should respond promptly, focus on correctness, design, security, and maintainability rather than style, which linters handle, give actionable and kind feedback, and distinguish blocking issues from suggestions. Use templates, CODEOWNERS to route reviews, and draft PRs for early feedback.

Likely follow-ups

- Why does PR size affect review quality?
- What belongs in a PR description?

```
# PR workflow steps:
```

```
1. Create feature branch from main
```

```
2. Make focused commits (one logical change per commit)
```

```
3. Push branch, open PR with descriptive title and body
```

```
4. CI runs automatically (tests, lint, security scan)
```

```
5. Reviewers approve (check required status checks pass)
```

```
6. Author addresses feedback with new commits or rebases
```

```
7. Squash or merge to main
```

```
8. CD pipeline deploys automatically
```

```
# Keep PRs small:
```

```
Ideal PR: under 400 lines changed, single concern, reviewable in 20 min
```

certs.eknathalabs.com

```
Large PR: split into a stacked PR series or use feature flags
```

A good PR has: a clear title (conventional commit format), a description explaining what changed and why (not how), screenshots/recordings for UI changes, a link to the ticket/issue, and a testing notes section. Small PRs get reviewed faster, merged sooner, and have fewer conflicts. Studies show review quality degrades sharply above 400 lines of diff. Large PRs often signal either a feature that wasn't broken down small enough, or a refactoring that should be done incrementally. Review latency (time from PR opened to first review) is a key developer experience metric.

PR description template: Create a .github/pull_request_template.md with sections for: What does this PR do, Why is it needed, How to test, Screenshots (if applicable), Related issues. A template nudges contributors toward completeness without requiring it.

📚 **Start here (official docs)**

- Git documentation

Q22 What are Git hooks — how do you use them in a team environment?

Workflows

🎙 **Spoken answer · ~83 s**

Git hooks are scripts that Git runs automatically at certain points in its workflow, stored in .git/hooks. Client-side hooks include pre-commit, which runs before a commit is created and is used for linting, formatting, secret scanning, and quick tests; prepare-commit-msg and commit-msg, which can template or validate messages, for example enforcing Conventional Commits; pre-push, for running tests before pushing; and post-checkout and post-merge, for tasks like installing dependencies. Server-side hooks run on the remote, such as pre-receive and update, which can reject pushes that violate policy, for example force pushes, bad commit messages, or large files, and post-receive, to trigger notifications or deployments. In hosted platforms, server-side hooks are replaced by branch protection rules, CI checks, and webhooks. The challenge in a team: .git/hooks is not versioned or shared, so developers have to install them. Solutions: set core.hooksPath to a directory committed in the repository, or use frameworks like pre-commit, Husky, or Lefthook, which manage hook configuration in a versioned file and install it with one command. Because client hooks can be bypassed with --no-verify and aren't guaranteed to be installed, treat them as developer convenience that gives fast feedback, and enforce the same checks authoritatively in CI and server-side.

Likely follow-ups

- Why can't client-side hooks be relied on for enforcement?
- How do you share hooks across a team?

```
# Client-side hooks (run locally):
```

```
.git/hooks/pre-commit          # run before committing (lint, test)
```

certs.eknathalabs.com

```
.git/hooks/commit-msg           # validate commit message format
.git/hooks/pre-push             # run before pushing (integration tests)
.git/hooks/post-checkout        # run after checkout (install deps?)
# Server-side hooks (run on remote):
pre-receive                     # validate pushes before accepting
post-receive                    # trigger deployments after push
update                          # per-branch validation
# Share hooks with the team using husky (Node.js):
npx husky init
echo "npx lint-staged" > .husky/pre-commit
echo "npx commitlint --edit" > .husky/commit-msg
# Or pre-commit (Python, language-agnostic):
pip install pre-commit && pre-commit install
```

Client-side hooks are not pushed to remote (they live in .git/hooks which is not tracked). Use husky or pre-commit framework to share hooks via package.json or .pre-commit-config.yaml. Pre-commit hooks: run linters (fast, targeted — only on changed files using lint-staged), validate commit message format with commitlint (enforces conventional commits). Pre-push hooks: run tests (heavier, only on push). Server-side hooks on GitHub/GitLab: use branch protection required status checks instead — they cannot be bypassed by clients.

Slow hooks hurt DX: A pre-commit hook that runs the full test suite makes every commit painful. Run only fast checks in pre-commit (lint, format, type-check — under 30 seconds). Run tests in pre-push or CI. Developers will git commit --no-verify to bypass slow hooks, defeating their purpose entirely.

📚 **Official docs**

- githooks
- Git documentation

certs.eknathalabs.com

Q23 What is a monorepo — what are the Git-specific challenges and how do you address them?

Workflows

🎙 Spoken answer · ~88 s

A monorepo holds many projects, services, and libraries in a single repository, as at Google and Meta, rather than one repository per project. Benefits: atomic cross-project changes in a single commit, easy code sharing and refactoring, unified tooling and dependency versions, and better visibility. Git-specific challenges as it scales. Size and performance: clone and fetch times grow, and commands like git status and checkout slow down with huge numbers of files and history. Solutions: partial clone with --filter=blob:none, fetching file contents on demand; shallow clones with limited depth; sparse checkout and cone mode, to materialise only the directories you need; the filesystem monitor and untracked cache for faster status; commit-graph and multi-pack index maintenance via git maintenance; and Scalar, which configures many of these. CI: building and testing everything for every change is too slow, so you need affected-target detection with build tools such as Bazel, Nx, Turborepo, or Pants, path filters, and caching, running only what's impacted. Ownership and access: CODEOWNERS for review routing, and permission granularity limited, since access to the repo means access to all code, unless you use tooling. Merge contention: many commits to main cause frequent rebases, so use merge queues. Release and versioning: independent versioning of components requires tooling like changesets. Tags and branch noise.

Likely follow-ups

- What does partial clone do?
- How do you avoid building everything on each commit?

```
# Monorepo: all services/packages in one repository
repo/
  services/api/
  services/worker/
  libraries/shared/
  infrastructure/terraform/
# Sparse checkout (only clone what you need):
git clone --no-checkout --filter=blob:none <url>
git sparse-checkout set services/api libraries/shared
# Git LFS for large files:
```

certs.eknathalabs.com

```
git lfs track "*.pkl" "*.bin" "infrastructure/certificates/*.pem"
```

```
git lfs ls-files
```

```
# Shallow clone for CI (faster, less history):
```

```
git clone --depth=1 <url>
```

```
# CODEOWNERS for monorepo path-based ownership:
```

```
services/api/         @api-team
```

```
infrastructure/       @platform-team
```

```
libraries/shared/     @core-team
```

Monorepo challenges: large repository size (slow clones, deep history), cross-service CI (every commit triggers every service's tests), and ownership conflicts (who approves changes to shared libraries?). Solutions: sparse checkout (only materialize needed paths), Git LFS for binary files, shallow clones in CI (--depth=1 is sufficient for most CI tasks), path-based CODEOWNERS for automated review assignment, and affected-service detection (only run tests for services with changed files). Tools like Nx, Turborepo, and Bazel provide monorepo-aware build and test systems.

> **Affected-change detection:** git diff --name-only origin/main HEAD | xargs -I{} dirname {} | sort -u gives changed directories. A CI pipeline can use this to only test and deploy the services whose code actually changed — reducing CI time from 30 minutes to 3 minutes on a monorepo with 20 services.

> 📚 **Official docs**
>
> - Git documentation

certs.eknathalabs.com

Q24 How do you manage Git submodules in production — what are the pitfalls?

Workflows

🎙 Spoken answer · ~88 s

A submodule embeds another Git repository inside your repository at a specific path, pinned to a particular commit. The superproject records the exact commit hash of the submodule, plus a .gitmodules file with its path and URL. Pitfalls in production. Cloning requires extra steps: git clone --recurse-submodules, or git submodule update --init --recursive afterwards; forgetting this leaves empty directories, a frequent CI failure. The pinned commit is separate from the submodule's branches, so developers commonly end up in a detached HEAD inside the submodule, make commits there, and forget to push the submodule before pushing the superproject, leaving others with a reference to a commit that doesn't exist on the remote. Updating is manual: bumping the pointer needs a commit in the superproject. Merge conflicts on submodule pointers are confusing. Authentication: CI needs credentials to fetch private submodule repositories, and relative URLs can break across forks and mirrors. Tooling and IDE support is inconsistent, and the workflow is hard for newcomers. Mitigation: set submodule.recurse true and push.recurseSubmodules check or on-demand; automate pointer updates with dependency bots; keep submodules few and stable; document the workflow; verify in CI that referenced commits exist. Alternatives: package managers and versioned artefacts for shared libraries, git subtree, which copies code into your repository, or a monorepo.

Likely follow-ups

- Why does a pushed superproject sometimes reference a missing commit?
- What alternatives avoid submodules?

```
git submodule add <url> vendor/lib  # add submodule
```

```
git submodule update --init --recursive  # initialize after clone
```

```
git clone --recurse-submodules <url>   # clone with all submodules
```

```
git submodule update --remote           # update to latest remote
```

```
git submodule foreach git pull origin main  # update all submodules
```

```
cat .gitmodules                          # submodule config
```

```
git submodule status                     # show SHA each submodule is at
```

```
# Check for uncommitted submodule changes:
```

```
git submodule foreach git status
```

Submodules pin a specific commit of another repository inside your repo. Use cases: vendoring third-party code at a known version, sharing infrastructure modules between repos. Pitfalls: forgetting --recurse-submodules on clone (submodule directories are empty), updating the submodule SHA without pushing the

certs.eknathalabs.com

change (others get empty directories), detached HEAD confusion (submodules always check out a specific commit, not a branch), and two-step push requirement (push submodule changes before pushing parent). Most teams prefer Git subtree or package management (npm, pip, go modules) over submodules for dependency management.

Submodule footgun: After git submodule update --remote, you must commit the updated SHA in the parent repo. If you forget, everyone else who pulls your commit gets the old submodule version. The submodule SHA embedded in the parent is what controls which version everyone gets — not the remote submodule's default branch.

📚 **Official docs**

- Pro Git: Submodules
- Git documentation

Q25 What is git reflog — how does it save you from disasters?

Recovery & undo

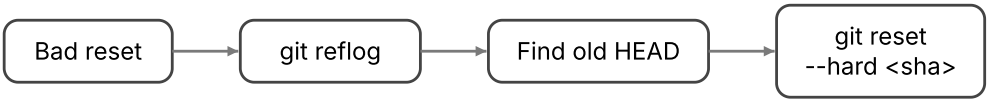
🎙 **Spoken answer · ~72 s**

The reflog is a local log of where HEAD and each branch tip have pointed over time, recording every checkout, commit, reset, rebase, merge, and amend, typically for ninety days for reachable entries and thirty days for unreachable ones. Because Git rarely deletes objects immediately, even commits that no branch references anymore usually still exist until garbage collection, and the reflog is your index to them. It saves you from disasters like: a hard reset that discarded commits; a rebase that went wrong; a deleted branch; an amend that lost content; a detached HEAD session you left. Use: git reflog shows entries like HEAD@{3} with the operation description; find the state before the mistake, then recover with git reset --hard HEAD@{3} to move the branch back, or git branch rescue <hash> or git switch -c rescue <hash> to create a branch at that commit without disturbing the current one, or cherry-pick specific commits. You can also see a branch's own history with git reflog show branchname, and use time-based references such as main@{yesterday}.

Likely follow-ups

- What doesn't the reflog protect?
- How long do reflog entries last?

🗺 **Reflog recovery**




```
git reflog                    # every HEAD movement in last 90 days
```

```
git reflog show main          # reflog for specific branch
```

ceris.eknathalabs.com

```
git reflog --date=iso           # with timestamps
```

```
# Recover from accidental reset --hard:
```

```
git reflog                      # find the SHA before the reset
```

```
git reset --hard HEAD@{3}       # go back to that point
```

```
# Recover accidentally deleted branch:
```

```
git reflog                      # find last commit SHA on the branch
```

```
git checkout -b recovered-branch abc1234  # recreate branch at that SHA
```

```
# Recover dropped stash:
```

```
git fsck --lost-found           # find dangling commits (stash objects)
```

```
git show dangling-sha           # inspect the recovered stash content
```

git reflog is the ultimate safety net — it records every position HEAD has been, including commits on deleted branches, before resets, before rebases. The reflog is local only (not pushed to remote) and expires after 90 days (configurable). Before running any destructive operation (reset --hard, rebase, branch deletion), note the current SHA — you can always return to it via reflog. "I accidentally reset --hard and lost my work" is recoverable within 90 days. "I force-pushed and others have no local copy" is trickier but often still recoverable via their reflogs.

> **Before any dangerous operation:** Run git log --oneline -5 and note the current SHA. Then run the operation. If something goes wrong, git reset --hard <noted-sha> is your escape hatch. This 5-second habit has saved countless hours of lost work.

> 📚 **Official docs**
> - git-reflog
> - Git documentation

certs.eknathalabs.com

Q26 How do you undo a bad commit that has already been pushed to main?

Recovery & undo

🎙 Spoken answer · ~86 s

On a shared branch like main, the safe way is git revert, not reset. git revert <hash> creates a new commit that applies the inverse of the bad commit, leaving history intact; others who have pulled simply get another commit and nothing breaks. It's the right approach for a production branch where history must not be rewritten. For a merge commit, specify the mainline parent with -m 1, which reverts the changes brought in by the merge relative to the first parent, typically main. Note a subtlety: after reverting a merge, if you later want to re-merge the same branch, Git thinks those changes were already merged and are removed, so you must revert the revert first, or recreate the changes. For several commits, revert each, or a range, in reverse order, possibly with -n to combine them into one commit. Then run tests, push through the normal pull request process, and deploy; in an incident, speed matters, so a revert pull request with an expedited review is the standard. Rewriting history with reset and force push is only acceptable in rare cases, such as removing a committed secret, and then needs coordination, branch protection disabled temporarily, and everyone re-syncing; even then, the secret must be rotated.

Likely follow-ups

- Why does revert beat reset on a shared branch?
- What is the catch when reverting a merge commit?

```
# Option 1: git revert (SAFE — adds new commit undoing changes)

git revert abc1234              # create new commit that reverses abc1234

git revert HEAD~3..HEAD         # revert last 3 commits

git revert --no-commit abc1234  # stage the revert without committing

git push origin main            # normal push (no force needed)

# Option 2: git reset + force push (DANGEROUS on shared branches)

git reset --hard HEAD~1         # undo last commit locally

git push --force-with-lease origin main  # force push (safer than --force)

# --force-with-lease: fails if remote has commits you haven't fetched

# Option 3: revert a merge commit

git revert -m 1 <merge-commit-sha>  # -m 1 = keep the mainline parent
```

certs.eknathalabs.com

Revert is the only safe option for shared branches — it adds a new commit that inverts the bad commit, preserving history. Reset + force-push rewrites history: anyone who has pulled the bad commit now has a diverged history and will need to reset or rebase. Use force-push only on private branches you own entirely. --force-with-lease is safer than --force: it refuses to push if someone else has pushed since your last fetch, preventing you from overwriting their work. For a bad merge commit, revert -m 1 specifies which parent (mainline = first parent) to keep.

Force-push on main: Even with --force-with-lease, force-pushing to main in a team causes pain for everyone. If you must force-push main: (1) communicate to the team, (2) have everyone run git fetch origin && git reset --hard origin/main afterward. Better: prevent it with branch protection "Do not allow force pushes."

📚 **Official docs**

- git-reset

Q27 How do you recover a deleted branch that was not merged?

Recovery & undo

🎙 **Spoken answer · ~87 s**

Deleting a branch only removes the reference file; the commits remain in the object database until garbage collection, which gives a recovery window. Steps. If you just deleted it, Git often prints the last hash: 'Deleted branch feature (was abc1234)'; recreate with git branch feature abc1234. Otherwise use git reflog to find the last commit the branch pointed to: look for entries mentioning checking out or committing on that branch, such as 'checkout: moving from feature to main', and the hash just before it is the tip. Then git branch feature <hash>. If the reflog doesn't help, for example in another clone, git fsck --lost-found, or git fsck --unreachable, lists dangling commits not reachable from any ref; inspect candidates with git show, and recreate a branch from the right one. If the branch had been pushed, recover from the remote: other clones might have origin/feature remote-tracking refs, and hosting platforms like GitHub keep deleted branch information: GitHub's pull request page has a 'Restore branch' button, and its API and events record the SHA; GitLab has similar options. Ask a teammate to push their copy. Time matters, since git gc prunes unreachable objects after the expiry period, normally two weeks for unreachable loose objects, and thirty days in the reflog.

Likely follow-ups

- How does git fsck help find lost commits?
- Where else can you recover a deleted remote branch from?

```
# Step 1: find the last commit SHA on the deleted branch
```

```
git reflog | grep "branch-name"     # if you remember the branch name
```

```
git reflog | head -30               # scan recent HEADs
```

certs.eknathalabs.com

```
git fsck --lost-found              # find unreferenced commits (dangling)
```

```
# Step 2: recreate the branch
```

```
git checkout -b recovered-branch <sha>
```

```
git switch -c recovered-branch <sha>
```

```
# If branch was remote-only and locally pulled:
```

```
git checkout -b recovered origin/deleted-branch  # if remote still has it
```

```
# If deleted from GitHub PR:
```

```
# GitHub keeps deleted PR branches — Restore Branch button in PR
```

```
# Or: GitHub API: POST /repos/{owner}/{repo}/git/refs
```

A "deleted" branch just removes the pointer file — the commits are still in the object database until garbage collected. As long as git gc has not run and the commits are in the reflog (within 90 days), recovery is straightforward. GitHub retains deleted branch commits for the lifetime of the repository and provides a "Restore branch" button on closed PRs — use this for recently deleted remote branches. The nuclear recovery option is git fsck --lost-found which writes all unreferenced objects to .git/lost-found/ — parse through the dangling commits to find your branch tip.

> **Prevention:** Configure GitHub to never auto-delete branches after merge (uncheck "Automatically delete head branches" in repo settings). Instead, delete branches manually after confirming the merge is stable in production. This gives a window to restore if the deployment reveals a problem.

> 📚 **Official docs**
> - git-reflog

certs.eknathalabs.com

Q28 How do you handle a corrupted Git repository — what are recovery steps?

Recovery & undo

🎙 Spoken answer · ~85 s

Corruption is rare but can occur from disk failures, interrupted writes, bad filesystems, or damaged transfers. First, diagnose: run git fsck --full, which checks object integrity and connectivity and reports missing or corrupt objects, such as 'missing blob' or 'bad object'. Before changing anything, make a backup copy of the entire directory. Recovery options in order of simplicity. If the remote has a good copy, the easiest is to re-clone, then copy over any unpushed local work: your working tree files, and branches that exist only locally. Alternatively, fetch missing objects: if only a few objects are corrupt, delete the corrupt object file or empty pack, then run git fetch origin, which re-downloads missing objects, and repeat fsck. If a pack file is corrupt, move it away and fetch again. For corrupt refs, check .git/refs and packed-refs, and repair or recreate branch references pointing to valid commits, using the reflog, or git update-ref. A zero-length or damaged index can be recreated with git reset or by deleting .git/index and running git reset. You can reconstruct from another clone of the same repository by copying missing objects across. If objects are truly lost and exist nowhere else, recover file content from working trees or editor history.

Likely follow-ups

- Why back up the directory before repairing?
- When is a re-clone the best fix?

```
git fsck --full                # check object database integrity
```

```
git fsck --lost-found          # find dangling objects
```

```
# Common corruption: corrupted pack file
```

```
cd .git/objects/pack
```

```
ls -la                        # find .pack and .idx files
```

```
git verify-pack -v pack-*.idx  # verify pack integrity
```

```
# Rebuild pack from loose objects:
```

```
git gc --aggressive            # repack all objects
```

```
# If pack is corrupt: remove and re-fetch from remote
```

```
rm .git/objects/pack/pack-*.pack .git/objects/pack/pack-*.idx
```

```
git fetch --all                # re-download all objects from remote
```

certs.eknathalabs.com

```
# Nuclear option: fresh clone
```

```
git clone <remote-url> repo-recovered
```

```
# Copy uncommitted work from corrupted repo:
```

```
rsync -av --exclude=".git" corrupted-repo/ repo-recovered/
```

Repository corruption is rare in normal operation but can happen due to hardware failure, full disk during a write, or process kill during git gc. git fsck --full validates every object in the database. For pack corruption (most common): delete the corrupt pack files and re-fetch from remote. For missing objects: git fetch restores anything that exists on the remote. The nuclear option (fresh clone + copy working tree) recovers most situations. Prevention: never let the disk fill up during git operations, avoid killing git gc mid-run.

CI/CD prevention: Configure CI runners to use --depth=1 shallow clones — this uses a single pack file with no history. If it corrupts, rm -rf and clone again. Never reuse CI workspaces between jobs without git clean -fdx — stale objects from previous runs can cause mysterious errors.

📚 **Official docs**

- Git documentation

Q29 How does Git handle large binary files — what is Git LFS and when do you need it?

Collaboration

🎙 **Spoken answer · ~68 s**

Git stores every version of every file in history, and binary files like images, videos, datasets, and compiled artefacts don't delta-compress well and can't be merged, so the repository grows quickly, clones and fetches become slow, and operations get heavier; deleting a big file later doesn't shrink history. Git Large File Storage, LFS, addresses this by replacing large files in the repository with small text pointer files, containing the object's hash and size, while the actual content is stored on a separate LFS server or storage and downloaded on demand. You configure which patterns to track with git lfs track '*.psd', which records them in .gitattributes; after installing the git lfs hooks, add and commit as usual. Cloning fetches pointers and then downloads the large files for the checked-out commit only, so history stays light; git lfs fetch --all downloads everything. You need LFS for: design assets, game assets, ML models and datasets, large media, and compiled binaries that must be versioned alongside code.

Likely follow-ups

- What does a pointer file contain?
- When would you avoid LFS and use object storage instead?

```
git lfs install                         # initialize LFS in repo
```

coris.eknathalabs.com

```
git lfs track "*.psd" "*.zip" "*.bin"  # track patterns in LFS
```

```
git lfs track "models/*.pkl"
```

```
cat .gitattributes                     # LFS tracking rules
```

```
git add .gitattributes large-file.psd
```

```
git lfs ls-files                       # list LFS-tracked files
```

```
git lfs status                         # LFS file status
```

```
git lfs fetch --all                    # fetch all LFS objects
```

```
# Check LFS usage:
```

```
git lfs env                            # LFS configuration
```

```
git lfs migrate info                   # analyze what should be in LFS
```

```
git lfs migrate import --include="*.bin"  # retroactively move to LFS
```

Git stores every version of every file — committing a 100MB binary file 100 times uses 10GB of repository storage. Git LFS replaces large files with text pointers in the repository and stores the actual content on a separate server. Without LFS: slow clones (everyone downloads all versions of binaries), repository size bloat, GitHub's 100MB file size limit blocks pushes. With LFS: the repository stays small (pointers are tiny), large files are fetched on demand. Use LFS for: ML model files, compiled binaries, design assets, test fixtures with large data files.

LFS in CI: CI runners must have Git LFS installed and configured. A checkout without LFS support leaves pointer files instead of actual content — builds fail with missing files. Add GIT_LFS_SKIP_SMUDGE=1 to CI environments that don't need LFS files to speed up checkouts, and explicitly fetch only needed LFS files.

certs.eknathalabs.com

📚 **Official docs**

- Git LFS
- Git documentation

Q30 What is a CODEOWNERS file — how does it scale code review in large teams?

Collaboration

🎙 Spoken answer · ~73 s

CODEOWNERS is a file in the repository, in the root, .github, or docs directory on GitHub, or the equivalent on GitLab and Bitbucket, that maps file path patterns to the users or teams responsible for them. Syntax is like .gitignore patterns followed by owners, for example 'docs/ @org/docs-team', or '*.tf @org/platform', with later matching rules taking precedence. When a pull request changes files matching a pattern, the platform automatically requests review from the owners, and with branch protection requiring code owner approval, the PR can't merge without their approval. How it scales code review. Routing: the right experts see the right changes without authors needing to know who to ask, which matters in large repositories and monorepos with many teams. Accountability: clear ownership of directories, services, security-sensitive files like CI workflows, authentication code, infrastructure, and the CODEOWNERS file itself. Governance: critical paths get mandatory review by a security or platform team, while the rest follows normal flow. Reduced reviewer fatigue: using team ownership instead of individuals spreads load and avoids single points of failure.

Likely follow-ups

- Why assign teams rather than individuals?
- How does CODEOWNERS interact with branch protection?

```
cat .github/CODEOWNERS
```

```
# Global owners (review everything):
```

```
*                       @core-team
```

```
# Path-specific owners:
```

```
infrastructure/         @platform-team
```

```
security/               @security-team
```

```
services/payments/      @payments-team @security-team
```

```
*.Dockerfile            @platform-team
```

```
**/terraform/**         @platform-team
```

```
# Individual file owners:
```

```
.github/workflows/      @devops-team
```

```
package.json            @frontend-lead
```

certs.eknathalabs.com

CODEOWNERS automatically assigns reviewers based on which files changed in a PR. Combined with branch protection "Require review from Code Owners," changes to critical paths cannot merge without approval from the responsible team. Last matching pattern wins in CODEOWNERS. This scales code review: the security team is automatically notified of changes to payment processing; the platform team reviews all Dockerfiles and Terraform changes; no manual tagging of reviewers needed. Reduces both review latency and the risk of changes slipping through without appropriate domain review.

Ownership granularity: Start with coarse ownership (teams own directories) and refine as pain points emerge. Over-granular CODEOWNERS (every file has an owner) creates review bottlenecks — every PR needs 10 different approvals. The goal is ensuring the right people see relevant changes, not creating approval bureaucracy.

📚 **Official docs**

- GitHub: About code owners

Q31 How do you handle secrets accidentally committed to a Git repository?

Collaboration

🎙 **Spoken answer · ~81 s**

The first priority is to treat the secret as compromised, since once pushed, it may have been copied, scanned, or cached, and removing it from Git doesn't undo exposure. Steps. Revoke or rotate the credential immediately, creating a new one and deploying it before or alongside the cleanup. Check audit logs for the credential's use to see if it was abused, and escalate to an incident if it was. Then clean the repository, for hygiene and to prevent re-exposure. If the commit is local and unpushed, amend or reset. If pushed, remove it from the full history with git filter-repo, the recommended tool, or BFG Repo-Cleaner, using a file or text replacement, then force push all affected branches and tags. Coordinate with the team: everyone must re-clone or carefully rebase, otherwise old history comes back on the next push. Also deal with copies: forks, pull request refs such as refs/pull on GitHub, which may need the hosting provider's support to purge, cached views, CI logs, and clones; GitHub support can remove cached views. Note that even after rewriting, the secret exists in places you don't control, which is why rotation is the real fix.

Likely follow-ups

- Why does rotation matter more than rewriting history?
- What tool rewrites history to remove a file?

```
# IMMEDIATE: revoke/rotate the exposed secret first (before anything else)
```

```
# Step 1: Remove from history using git-filter-repo (modern tool):
```

```
pip install git-filter-repo
```

certs.eknathalabs.com

```
git filter-repo --path secrets.env --invert-paths  # remove file from history
```

```
git filter-repo --replace-text patterns.txt         # replace secret values
```

```
# patterns.txt: LITERAL:actual-secret==>REDACTED
```

```
# Step 2: Force-push all branches and tags:
```

```
git push origin --force --all
```

```
git push origin --force --tags
```

```
# Step 3: Notify all contributors to re-clone (their local copies have the secret)
```

```
# Step 4: Contact GitHub Support to purge caches
```

```
# Prevention: pre-commit secrets detection
```

```
pip install detect-secrets && detect-secrets scan > .secrets.baseline
```

```
pip install git-secrets && git secrets --install
```

Assume the secret is compromised the moment it is pushed — GitHub notifications, webhooks, and CI systems may have already processed it. Rotation/revocation is the only true fix; history rewriting is secondary cleanup. git filter-repo is the modern replacement for the deprecated BFG Repo Cleaner and git filter-branch. After rewriting, force-push, and have every contributor re-clone — their local copies still contain the secret in reflog/pack files. GitHub can purge cached views of the old content but the exposed window may have been enough for automated scanners.

GitHub automatically scans for secrets: GitHub Secret Scanning detects common secret patterns (AWS keys, GCP service account keys, GitHub tokens) and notifies the provider. AWS and GCP will alert you when your credentials are detected. Treat it as a known compromise and rotate immediately — do not wait to finish the history rewrite.

📚 **Official docs**

- Git documentation

certs.eknathalabs.com

Q32 What is git rebase --onto — how do you use it to transplant a branch?

Collaboration

🎙 Spoken answer · ~75 s

git rebase --onto lets you transplant a range of commits from one base to another, with three arguments: the new base, the old base to exclude, and the branch containing the commits. The form is git rebase --onto <newbase> <oldbase> <branch>. It takes commits that are reachable from branch but not from oldbase, and replays them on top of newbase. Classic scenario: you created feature-b from feature-a, but feature-a was squash-merged into main, so feature-b's history still contains feature-a's old commits, and a plain rebase onto main would try to reapply them and cause conflicts. Fix: git rebase --onto main feature-a feature-b, which says 'take the commits on feature-b that are after feature-a, and put them on main'. Another scenario: moving a topic branch that was started from the wrong branch, such as branched from develop but meant for main; or removing a range of commits from the middle of history: git rebase --onto A B drops commits between A and B when A is an ancestor of B, hence in the form git rebase --onto A B branch.

Likely follow-ups

- When would a plain rebase fail where --onto works?
- What do the three arguments mean?

```
# Situation: feature-b was branched off feature-a, but feature-a is delayed
```

```
# You want to move feature-b commits directly onto main
```

```
git rebase --onto main feature-a feature-b
```

```
# Syntax: git rebase --onto <newbase> <upstream> <branch>
```

```
# Moves commits on feature-b that are NOT on feature-a onto main
```

```
# Another use: remove a range of commits from a branch
```

```
git rebase --onto HEAD~5 HEAD~2   # remove commits 3-5 (keep 1-2 and 6+)
```

```
# Preview what will be rebased:
```

```
git log --oneline feature-a..feature-b  # commits that will move
```

--onto is the Swiss Army knife of rebasing — it lets you specify exactly what commits to move and where to put them. The three-argument form: newbase (where to put them), upstream (defines what commits to move — everything NOT on upstream), branch (the branch to rebase). Use cases: extracting part of a branch into its own PR, moving a branch stacked on another branch to base it directly on main, and surgically removing a range of commits from the middle of a branch history.

certs.eknathalabs.com

Stacked PRs pattern: Teams using stacked PRs (each feature branch built on top of another) use --onto frequently. When the bottom PR merges, use git rebase --onto main old-base next-branch to rebase the dependent branch onto the updated main. Tools like Graphite and ghstack automate this workflow.

📚 **Official docs**

- git-rebase
- Git documentation

Q33 How does Git integrate with CI/CD — what triggers what in a typical pipeline?

CI/CD & GitOps

🎙 **Spoken answer · ~76 s**

Git is the trigger and source of truth for CI/CD. A typical flow. Developers push a feature branch and open a pull request; the hosting platform sends webhooks to the CI system, or runs built-in workflows such as GitHub Actions, which check out the merge result or the branch head, then run lint, unit tests, builds, and security scans; status checks are reported back to the pull request, and branch protection blocks merging until they pass. When merged to main, a push event triggers the main pipeline: build the artefact once, tag it with the commit SHA, run integration tests, publish to a registry, and deploy to staging, then promote to production with approvals or automatically. Tags can trigger release pipelines: pushing v1.4.0 builds the release artefacts, generates a changelog, and publishes. Scheduled triggers run nightly builds or dependency updates. Path filters limit pipelines to changed directories in monorepos. Manual dispatch triggers one-off jobs. In GitOps, CI builds and pushes an image, then updates the image reference in a configuration repository, and a controller like Argo CD detects the Git change and deploys.

Likely follow-ups

- What event triggers a release pipeline?
- Why tag artefacts with the commit SHA?

```
# GitHub Actions trigger patterns:
```

```
on:
```

```
  push:
```

```
    branches: [main, release/**]   # any push to main or release branches
```

```
  pull_request:
```

```
    branches: [main]               # any PR targeting main
```

```
  push:
```

certs.eknathalabs.com

```
    tags: [v*.*.*]                # semantic version tags
  workflow_dispatch: {}           # manual trigger
  schedule:
    - cron: "0 2 * * *"           # nightly at 2am UTC
# Path filtering (only run when relevant files change):
  push:
    paths:
      - "services/api/**"
      - "!**.md"                  # exclude markdown changes
```

CI/CD pipelines are triggered by Git events — push, pull_request, tag creation, and schedule. Path filters prevent unnecessary pipeline runs: a docs commit should not redeploy the API. Tag-based triggers enable release workflows: creating a semver tag triggers a production deployment pipeline. The canonical pattern: feature branch push → PR CI (test, scan), main branch push → staging deployment, tag push → production deployment. Separate jobs per service in a monorepo with path filters keep CI fast and cheap.

GitOps pattern: In a GitOps model, the pipeline does not deploy directly. Instead, it updates the image tag in a separate manifests repository. ArgoCD detects the manifest change and deploys. The deployment history is the Git history of the manifests repo. Rollback = git revert in the manifests repo. This separation of application code (app repo) and deployment config (manifests repo) is the production standard for Kubernetes deployments.

📚 **Official docs**

- Git documentation

certs.eknathalabs.com

Q34 What is semantic versioning and how do you automate releases with Git tags?

CI/CD & GitOps

🎙 Spoken answer · ~71 s

Semantic versioning, SemVer, uses version numbers in the form MAJOR.MINOR.PATCH. Increment MAJOR for incompatible, breaking changes to the public API; MINOR for backward-compatible new functionality; and PATCH for backward-compatible bug fixes. Pre-release suffixes like 1.2.0-rc.1 and build metadata are supported. It communicates the impact of upgrading to consumers. Automating releases with Git tags: the tag, such as v1.4.2, marks the commit that is the release. Approaches. Manual tagging triggers a pipeline: pushing an annotated tag starts the release workflow that builds, tests, and publishes artefacts and release notes. Fully automated: adopt Conventional Commits, where feat commits imply a minor bump, fix commits a patch, and a breaking change marker a major bump; tools such as semantic-release, release-please, standard-version, or GitVersion read the commit history since the last tag, calculate the next version, update changelogs and version files, create the tag and a GitHub release, and publish packages, all from CI on merges to main. Release-please opens a release pull request that you merge to cut the release, giving a human gate.

Likely follow-ups

- How do Conventional Commits drive version bumps?
- Why must you never move a published tag?

```
# SemVer: MAJOR.MINOR.PATCH (e.g., 2.4.1)
```

```
MAJOR: breaking change (incompatible API change)
```

```
MINOR: new feature (backward-compatible)
```

```
PATCH: bug fix (backward-compatible)
```

```
# Manual tagging:
```

```
git tag -a v2.4.1 -m "Release v2.4.1: fix payment timeout"
```

```
git push origin v2.4.1
```

```
# Automated: semantic-release (reads conventional commits):
```

```
npx semantic-release          # determines version bump from commits
```

```
# feat: → minor bump (1.2.0 → 1.3.0)
```

```
# fix:  → patch bump (1.2.0 → 1.2.1)
```

```
# BREAKING CHANGE → major bump (1.2.0 → 2.0.0)
```

certs.eknathalabs.com

```
# Get latest tag:
```

```
git describe --tags --abbrev=0
```

```
git describe --tags            # v2.4.1-3-gabc1234 (tag + commits since + sha)
```

Semantic versioning communicates the impact of a release. semantic-release automates the entire release process: reads conventional commits since last release, determines version bump, creates Git tag, generates CHANGELOG.md, publishes to npm/PyPI/etc., and creates a GitHub Release — all from a single CI step. git describe generates version strings for non-release builds: v2.4.1-3-gabc1234 means "3 commits after v2.4.1, current SHA is abc1234" — perfect for Docker image tags in feature branch builds.

Docker image tagging strategy: Production: v2.4.1 (exact semver tag). Staging: main-abc1234 (branch-sha). Feature: feature-login-def5678. Never use :latest in production. Use git describe --tags for builds that need a version string but aren't on a tag yet. This makes every deployed image traceable to an exact Git commit.

📚 **Official docs**

- Semantic Versioning
- Git documentation

certs.eknathalabs.com

Q35 How do you use Git in a GitOps workflow — what does the repository structure look like?

CI/CD & GitOps

🎙 Spoken answer · ~78 s

In GitOps, Git repositories hold the declarative desired state of infrastructure and applications, and an agent like Argo CD or Flux reconciles clusters to match. Repository structure. A common pattern separates application source repositories from a configuration, or environment, repository. The app repo contains code, Dockerfile, and CI; CI builds an image and then updates the image tag in the config repo. The config repo holds Kubernetes manifests, Helm charts, or Kustomize overlays. A typical layout: a base directory with shared manifests or charts, and overlays or environment directories such as dev, staging, and production, each with patches and values; or a directory per cluster and then per application, plus a bootstrap folder with Argo CD Application or Flux Kustomization definitions, including an app-of-apps or ApplicationSet. Cluster add-ons like ingress and monitoring live in their own area, and tenants' configuration is separated by team or namespace. Promotion between environments happens by pull requests that change the version in the next environment's directory, or by automated image updaters. Branching: many teams use directories per environment on a single main branch, rather than branch-per-environment, which tends to drift.

Likely follow-ups

- Why separate app and config repositories?
- Why prefer directories over branches for environments?

```
# Two-repo GitOps pattern:
app-repo/               # application source code
  src/
  Dockerfile
  .github/workflows/ci.yaml  # builds image, updates manifests repo
manifests-repo/         # deployment config (ArgoCD watches this)
  apps/
    myapp/
      base/kustomization.yaml
      overlays/
        staging/kustomization.yaml   # image: myapp:main-abc1234
```

certs.eknathalabs.com

```
        production/kustomization.yaml # image: myapp:v2.4.1
```

```
# CI pipeline update step:
```

```
cd manifests-repo
```

```
kustomize edit set image myapp=myregistry/myapp:main-${GITHUB_SHA::8}
```

```
git commit -m "ci: deploy myapp main-${GITHUB_SHA::8} to staging"
```

```
git push origin main
```

```
# ArgoCD detects the push and syncs the cluster
```

GitOps treats the Git repository as the single source of truth for cluster state. Benefits: every deployment is a Git commit (full audit trail), rollback is git revert, drift detection (ArgoCD alerts when cluster state diverges from manifests), and no direct kubectl access needed for deployments. The manifests repo should have the same branch protections as the app repo — a push to the manifests repo triggers a production deployment. PR-based changes to manifests provide review gates for infrastructure changes.

Manifests repo access control: The CI pipeline service account has push access to the manifests repo only for the staging overlay path. Production overlay changes require a human PR. This enforces a promotion gate: staging deploys automatically, production requires intentional human action. This is the security model for GitOps in regulated environments.

📚 **Official docs**

- OpenGitOps principles
- Git documentation

certs.eknathalabs.com

Q36 How do you handle Git in CI/CD runners — what configuration matters for performance and security?

CI/CD & GitOps

🎙 Spoken answer · ~87 s

Runners clone repositories constantly, so their Git configuration affects speed, cost, and security. For performance, use shallow clones with a fetch depth of one, so the job downloads only the commit it needs instead of full history; deepen only for jobs that really need history, such as changelog generation, version calculation from tags, or blame-based tooling. In large repositories, combine partial clone with blob filters and sparse checkout so the runner downloads only the paths required. On persistent runners, keep a cached mirror or reference clone, or place a regional Git mirror close to the runners to cut latency and load on the origin. Skip submodules, tags, and LFS objects unless the job needs them. Always check out the exact commit SHA for determinism. For security, give the job a short-lived, least-privilege token, such as the platform's per-job token or a read-only deploy key, and never a personal access token. Set persist-credentials to false so later steps and uploaded artefacts cannot capture the token from the .git configuration. Treat pull requests from forks as untrusted, and do not give them secrets. Prefer ephemeral runners, so no state or credentials linger between jobs. Pin the checkout action to a commit hash. Avoid uploading the .git directory as an artefact.

Likely follow-ups

- Why set persist-credentials to false?
- When do you need full history in CI?

```
# Shallow clone for CI (most CI only needs current code, not full history):
```

```
git clone --depth=1 --single-branch --branch main <url>
```

```
# Or in GitHub Actions:
```

```
- uses: actions/checkout@v4
```

```
  with:
```

```
    fetch-depth: 1              # shallow (default)
```

```
    fetch-depth: 0              # full history (for git describe, semantic-release)
```

```
# Credential management for CI:
```

```
git config credential.helper store   # use credential store
```

```
git config url."https://token@github.com/".insteadOf "https://github.com/"
```

```
# Avoid git gc in CI (slow, not needed per-run):
```

certs.eknathalabs.com

```
git config gc.auto 0
```

```
# Cache the repo between runs:
```

```
- uses: actions/cache@v4
```

```
  with:
```

```
    path: .git
```

```
    key: git-cache-${{ github.sha }}
```

CI performance: shallow clones (--depth=1) reduce checkout time from minutes to seconds for large repos with deep history. Full history (fetch-depth: 0) is needed for semantic-release (reads all commits), git describe (walks tags), and coverage reports that diff against base. Disable gc.auto in CI — automatic garbage collection wastes runner time. Cache the .git directory between runs to avoid re-downloading the full pack each time. CI credentials: use short-lived tokens (GITHUB_TOKEN in GitHub Actions, Workload Identity in GCP) rather than long-lived PATs or SSH keys.

git clean in CI: Always run git clean -fdx before building in a reused workspace — leftover artifacts from previous builds cause non-deterministic failures. Better: use ephemeral runners (each job gets a fresh container) to eliminate workspace state entirely. GitHub Actions hosted runners are ephemeral; self-hosted runners must implement their own cleanup.

📚 **Official docs**

- Git documentation

certs.eknathalabs.com

Q37 What is GPG commit signing — how and why do you enable it?

Security

🎙 Spoken answer · ~85 s

GPG commit signing attaches a cryptographic signature to a commit using the author's private key, so others can verify that the commit was made by the holder of that key and hasn't been altered. Without signing, Git's author and committer fields are just text anyone can set, so identity can be forged: git config user.name 'Someone Else' works. Enabling: generate a GPG key, or use SSH signing or Sigstore gitsign, which Git supports now; tell Git which key with user.signingkey, enable signing with commit.gpgsign true and tag.gpgsign true; add the public key to your hosting account so the platform can show a Verified badge. With SSH signing, set gpg.format ssh and use your existing SSH key, which is simpler to adopt. Why it matters. Integrity and provenance: proves the origin of commits and releases, helping defend against impersonation, such as an attacker pushing commits as a maintainer with a stolen token, or a malicious pull request pretending to be from a trusted author. Compliance: some regulations and enterprise policies require signed commits as part of the audit trail. Supply chain security: signed tags for releases let consumers verify authenticity. Enforcement: branch protection rule requiring signed commits on main; and verification in CI or admission policies.

Likely follow-ups

- What does the Verified badge confirm?
- What are alternatives to GPG for signing?

```
gpg --full-generate-key              # generate GPG key pair
```

```
gpg --list-secret-keys --keyid-format=long
```

```
git config --global user.signingkey <KEY-ID>
```

```
git config --global commit.gpgsign true   # sign all commits
```

```
git config --global tag.gpgsign true      # sign all tags
```

```
git commit -S -m "feat: add OAuth2"       # sign a single commit
```

```
git log --show-signature                  # verify signatures in log
```

```
git verify-commit <sha>                  # verify specific commit signature
```

```
# GitHub: upload public key at Settings → SSH and GPG keys
```

```
# GitHub shows "Verified" badge on signed commits
```

```
# Modern alternative: SSH commit signing (simpler, same key as auth):
```

```
git config --global gpg.format ssh
```

certs.eknathalabs.com

```
git config --global user.signingkey ~/.ssh/id_ed25519.pub
```

GPG/SSH signing cryptographically proves that commits were made by the holder of a private key — not just by anyone with the email address set in git config. Without signing, anyone can make commits attributed to your email by running git config user.email "you@company.com". Signed commits provide non-repudiation: you cannot deny making a commit that is signed with your key. Required for compliance in high-security environments (FedRAMP, some PCI-DSS scopes). GitHub's "Require signed commits" branch protection setting enforces this at the repository level.

SSH vs GPG: SSH signing (Git 2.34+) uses the same key you use to authenticate to GitHub — simpler key management, no GPG daemon needed. For most teams, SSH signing is sufficient and easier to set up. GPG provides a stronger trust model (Web of Trust, expiry, revocation certificates) relevant for open-source projects with distributed contributors.

📚 **Official docs**

- Git: Signing your work

Q38 How do you audit Git repository access and detect suspicious activity?

Security

🎙 **Spoken answer · ~88 s**

Auditing Git access and detecting suspicious activity covers both the hosting platform and the repository content. Platform audit logs: GitHub, GitLab, and Bitbucket provide audit logs, with enterprise tiers streaming them to a SIEM; they record logins, token and SSH key creation, permission changes, repository visibility changes, forks, clones, deploy key additions, branch protection changes, secret scanning alerts bypassed, webhook changes, and app installations. Detection ideas: alerts on repository made public, branch protection disabled, new personal access tokens or deploy keys, mass cloning or downloads by one user or token, access from unusual locations or IP addresses, impossible travel, logins without MFA, new OAuth apps with broad scopes, force pushes to protected branches, and unusual activity outside working hours or by dormant accounts. Access hygiene: SSO and mandatory MFA, least privilege through teams rather than individual grants, regular access reviews, removing departed users, short-lived or fine-grained tokens with expiration, and restricting IP ranges. Repository content: secret scanning, push protection, dependency review, and checking for suspicious changes such as modifications to CI workflows, new outbound network calls in build scripts, or obfuscated code, using CODEOWNERS on sensitive paths. Signed commits help attribute changes. Pull audit logs for forensic timelines during incidents. Also monitor third-party integrations and the CI systems that have repository access.

Likely follow-ups

- Which audit events indicate possible exfiltration?
- Why review OAuth app installations?

certs.eknathalabs.com

```
# GitHub audit log (organization level):
```

```
gh api /orgs/{org}/audit-log?phrase=action:git.push --paginate
```

```
# Key events to monitor:
```

```
git.push                  # who pushed where, when
```

```
protected_branch.policy_override  # branch protection bypassed
```

```
member.added              # new member with write access
```

```
repository.create/delete  # repo creation or deletion
```

```
oauth_application.token_revoked   # credential revocation event
```

```
# GitHub Advanced Security: secret scanning + code scanning alerts
```

```
gh api /repos/{owner}/{repo}/secret-scanning/alerts
```

```
# Local: recent pushes and clones
```

```
git log --all --format="%H %ai %ae %s"  # full commit audit trail
```

```
# Detect force-push on main:
```

```
# GitHub webhook on push event: compare forced=true attribute
```

Git audit trails answer: who pushed what, when, and from where. GitHub organization audit logs export to SIEM (Splunk, Chronicle) for centralized security monitoring. Key anomaly signals: force-push to main (should never happen if protected), new admin added outside normal IAM process, push outside business hours from unknown IP, secret scanning alert for a new commit, large binary files pushed (possible data exfiltration). For regulated environments, enable GitHub's audit log streaming to GCS or S3 for immutable long-term retention.

SOC2 requirement: Demonstrating access control and auditability for the code repository is a core SOC2 CC6.1 and CC6.2 control. The evidence: branch protection rules preventing direct push, required PR reviews, signed commits, and exported audit logs showing all access events. A Platform Engineer owns this evidence collection and retention.

📚 **Official docs**

- Git documentation

certs.eknathalabs.com

Q39 How do you prevent sensitive data from entering a Git repository — what tools exist? Security

🎙 Spoken answer · ~84 s

Preventing sensitive data from entering a repository requires layers, because any single control can be bypassed. Before commit: educate developers; configure .gitignore for env files, key files, and local configuration, and provide template files like .env.example; use pre-commit hooks running secret scanners such as gitleaks, detect-secrets, or TruffleHog, managed via the pre-commit framework so they are versioned and consistent. At push: platform push protection, such as GitHub secret scanning push protection, which blocks pushes containing recognised secrets and requires explicit bypass with justification; server-side pre-receive hooks on self-hosted servers. In pull requests and CI: required status checks running secret scanners across the diff and the history, plus scanning for PII patterns, large files, and keys; fail the build on findings. Continuous: scheduled full-history scans of all repositories and organisation-wide secret scanning that alerts on existing and new leaks, with validation of whether detected credentials are active. Design to avoid secrets: use secret managers, workload identity, and short-lived credentials, so there is less to leak; provide libraries and templates that fetch secrets at runtime. Also guard sensitive data beyond credentials: customer data in test fixtures, database dumps, and logs; use synthetic or anonymised data. Handle large files with LFS or exclusion rules.

Likely follow-ups

- Why combine local hooks with server-side protection?
- How do you reduce secrets that could be committed in the first place?

```
# Pre-commit hooks for local detection:
```

```
pip install detect-secrets
```

```
detect-secrets scan > .secrets.baseline    # create baseline
```

```
detect-secrets audit .secrets.baseline      # review findings
```

```
# Add to .pre-commit-config.yaml:
```

```
repos:
```

```
  - repo: https://github.com/Yelp/detect-secrets
```

```
    hooks: [{id: detect-secrets}]
```

```
# git-secrets (AWS Labs):
```

```
git secrets --install
```

```
git secrets --register-aws    # add AWS pattern detection
```

certs.eknathalabs.com

```
git secrets --add "PRIVATE KEY"
```

```
# GitHub Secret Scanning (server-side, automatic):
```

```
gh api /repos/{owner}/{repo}/secret-scanning/alerts
```

```
# CI scan:
```

```
truffleHog git <repo-url> --only-verified  # scan history for secrets
```

```
gitleaks detect --source . --verbose
```

Defense in depth for secret prevention: (1) developer workstation pre-commit hooks (detect-secrets, gitleaks) — fast feedback before commit, (2) CI pipeline scan on every PR (truffleHog, gitleaks) — catches anything that bypassed hooks, (3) GitHub Advanced Security server-side scanning — automatic, no configuration needed, alerts provider directly, (4) periodic full history scans — finds secrets in old commits. .gitignore for .env files and cloud credential files. Developer training: secrets in environment variables or secret managers, never in files committed to Git.

.gitignore is not security: .gitignore prevents accidental addition of files but can be bypassed with git add -f. A misplaced git add . in the wrong directory commits everything. Pre-commit secret scanning provides a safety net even when .gitignore fails. Neither is a replacement for proper secret management — secrets should never be on the filesystem at all, only fetched from Secret Manager at runtime.

📚 **Official docs**

- Git documentation

certs.eknathalabs.com

Q40 What is git gc — how does garbage collection work and when does it matter?

Advanced & internals

🎙 Spoken answer · ~82 s

git gc, garbage collection, optimises the repository's storage and removes unneeded objects. What it does. It packs loose objects, the individual zlib-compressed files created by each commit, into packfiles, which store objects efficiently with delta compression between similar versions, drastically reducing disk usage and speeding up operations. It consolidates multiple packs, packs references into a packed-refs file, prunes unreachable objects, meaning those not referenced by any branch, tag, reflog entry, or the stash, once they're older than the grace period, gc.pruneExpire, by default two weeks, and expires old reflog entries, ninety days for reachable and thirty for unreachable ones by default. Git runs git gc --auto automatically after certain operations when thresholds are exceeded, such as many loose objects or packs. It matters in several situations: repositories with large histories, where unoptimised object storage makes clones and fetches slower; servers hosting many repositories, where git maintenance schedules regular background tasks like commit-graph updates, prefetching, and incremental repacking; after rewriting history to remove sensitive data or large files, where you need to expire the reflog and run gc --prune=now --aggressive to actually reclaim space locally, remembering that remotes need their own gc; and when disk space is tight.

Likely follow-ups

- What does git gc prune and when?
- Why avoid gc --prune=now casually?

```
git gc                          # manual garbage collection
```

```
git gc --aggressive             # more thorough repacking (slow)
```

```
git gc --prune=now              # immediately prune unreachable objects
```

```
git count-objects -v            # show loose object count and size
```

```
git cat-file --batch-check --batch-all-objects | wc -l  # total object count
```

```
# Pack files vs loose objects:
```

```
ls -lh .git/objects/pack/       # pack files (compressed collections)
```

```
find .git/objects -type f | grep -v pack | wc -l  # loose object count
```

```
# Auto-gc threshold:
```

```
git config gc.auto             # default 6700 loose objects triggers auto-gc
```

```
git config gc.autoPackLimit    # default 50 pack files triggers repacking
```

certs.eknathalabs.com

Git gc packs loose objects into pack files, prunes unreachable objects, and compresses pack files. Loose objects are created by every git add, commit, and object operation. Git auto-runs gc when loose objects exceed gc.auto (default 6700). Pack files store delta-compressed objects — a pack of 1000 commits might be smaller than the 1000 loose objects because similar objects (consecutive commits on similar trees) compress together well. git gc --aggressive runs more expensive delta compression, useful for repositories with degraded performance from frequent rebases or large binary file history.

Performance tuning: On a server-side bare repository receiving many pushes, auto-gc can trigger mid-push and cause timeouts for users. Set gc.auto=0 and schedule gc in a cron job during off-peak hours. Server-side git gc also releases space from large files removed via git filter-repo — the actual space is not freed until gc prunes the unreachable objects.

📚 **Official docs**

- Git: Maintenance & garbage collection
- Git documentation

Q41 What is git shallow clone — when and how do you use it safely?

Advanced & internals

🎙 **Spoken answer · ~85 s**

A shallow clone downloads only a limited portion of history, for example git clone --depth 1 fetches just the latest commit and its tree, with no ancestors. It saves bandwidth, time, and disk, so it's the default choice for CI, which usually only needs to build the current commit, and for quickly grabbing a large repository to read or build. You can also use --depth N for several commits, --shallow-since a date, or --single-branch to fetch just one branch. Safe use. Know what breaks: commands needing history fail or misbehave, such as git log beyond the depth, git blame, git describe without tags, merge-base calculations, and tools that compute versions from tags, or diff against a base branch, like affected-test detection. If you need those, deepen with git fetch --deepen=N, git fetch --unshallow, or fetch the needed branch and tags explicitly. Pushing from a shallow clone is possible, though history limitations can cause problems with some servers and operations. Alternatives that keep full commit history but defer file contents: a partial clone with --filter=blob:none, which fetches blobs on demand and supports blame and log, generally better for development; or --filter=tree:0 for even less. Use shallow for automation, partial for humans working on big repositories.

Likely follow-ups

- What breaks in a shallow clone?
- How does a partial clone differ from a shallow one?

```
git clone --depth=1 <url>       # only last commit (fastest)
```

```
git clone --depth=50 <url>      # last 50 commits
```

certs.eknathalabs.com

```
git clone --shallow-since="1 month ago" <url>
```

```
git fetch --depth=1             # deepen or re-shallow an existing clone
```

```
git fetch --unshallow           # convert shallow back to full clone
```

```
git log --oneline | wc -l       # count available commits in shallow clone
```

```
# Check if repo is shallow:
```

```
cat .git/shallow                # exists if shallow clone
```

```
# CI: fetch just enough for comparison:
```

```
git fetch --depth=2 origin main  # enough for git diff HEAD~1
```

Shallow clones download only the last N commits, dramatically reducing clone time for repositories with deep history. Use in CI/CD (--depth=1 is sufficient for most builds), local exploration of large repos, or when disk space is limited. Limitations: git log history is truncated, git blame may not show original author, git describe may not find tags, and git bisect cannot traverse beyond the shallow boundary. For operations that need full history: git fetch --unshallow before running semantic-release, git describe, or full git bisect. The grafts mechanism in .git/shallow tells Git where history is artificially truncated.

CI checkout strategy: Use fetch-depth: 1 for build jobs (fastest). Use fetch-depth: 0 for semantic-release, changelog generation, and coverage diff jobs (needs full history). Separate these into different jobs rather than always fetching full history for every CI run. This optimization alone can reduce CI checkout time by 80% on repos with years of history.

📚 **Official docs**

- Git: Maintenance & garbage collection
- Git documentation

certs.eknathalabs.com

Q42 What is git sparse-checkout — how do you use it for monorepos?

Advanced & internals

🎙 Spoken answer · ~87 s

Sparse checkout lets you populate the working directory with only a subset of the repository's files, while the full repository metadata still exists. Combined with a partial clone, you also avoid downloading the unneeded file contents. It's valuable in monorepos where a developer or CI job works on a few services among hundreds of directories. Usage: git clone --filter=blob:none --sparse <url>, then git sparse-checkout set services/payments libs/common, which materialises just those directories, plus files at the root; later add more with git sparse-checkout add. Cone mode, the default in modern Git, takes directory names rather than patterns, and is much faster than the pattern-based non-cone mode; it always includes the files in the root and in parent directories of the selected ones. To restore everything, git sparse-checkout disable. Benefits: faster checkout, git status, and editor indexing; smaller disk usage; and less noise. In CI, check out only the paths that the job needs, perhaps determined from changed files. Cautions: build systems that expect dependencies in other directories will fail unless those directories are included, so define sets of directories for each project, ideally through tooling or scripts maintained in the repository; some commands such as git grep only see checked-out files unless told otherwise; and IDEs may be confused.

Likely follow-ups

- What is cone mode?
- How would CI decide which paths to include?

```
git clone --no-checkout --filter=blob:none <url> myrepo
```

```
cd myrepo
```

```
git sparse-checkout init --cone        # enable cone mode (recommended)
```

```
git sparse-checkout set services/api libraries/shared  # only these paths
```

```
git checkout main                      # now materializes only specified paths
```

```
git sparse-checkout add services/worker  # add another path
```

```
git sparse-checkout list                # show configured paths
```

```
git sparse-checkout disable             # back to full checkout
```

```
# Check what files are materialized:
```

```
git ls-files | head -20
```

certs.eknathalabs.com

Sparse-checkout downloads all objects but only materializes (writes to disk) the files in specified paths. Combined with --filter=blob:none (partial clone), even the object database is lazily populated — Git fetches blob content only when needed. This enables working with monorepos containing hundreds of services without checking out the full tree. A monorepo with 10GB of files across 50 services can be worked with as if it were a small single-service repo — only your service's files are on disk. CI runners for specific services use sparse-checkout to only test the changed service.

--filter=blob:none: The partial clone filter tells Git not to download file content (blobs) until they are needed. The repository structure (commits, trees) is downloaded, but file content is lazy-fetched. Combine with sparse-checkout for maximum efficiency in large monorepos: git clone --filter=blob:none --no-checkout. This is how engineers at Google and Meta work with their multi-GB monorepos locally.

📚 **Official docs**

- Git documentation

Q43 **How does Git handle line endings across operating systems — what is core.autocrlf?** Advanced & internals

🎙 **Spoken answer · ~88 s**

Windows uses CRLF for line endings, while Linux and macOS use LF. If developers on different systems commit files with different endings, you get noisy diffs where every line appears changed, spurious conflicts, and scripts failing, for example a shell script with CRLF fails with a bad interpreter error. Git offers settings to normalise this. core.autocrlf true, typical on Windows, converts CRLF to LF when committing and LF to CRLF on checkout, so the repository stores LF but the working tree has CRLF. core.autocrlf input, typical on macOS and Linux, converts CRLF to LF on commit but doesn't alter checkout. False does no conversion. Because autocrlf is a per-user setting, a repository-level policy is more reliable: a .gitattributes file committed to the repository, such as '* text=auto' to let Git detect text files and normalise to LF in the repository, with '*.sh text eol=lf' to force LF for scripts that must run on Linux, '*.bat text eol=crlf' for Windows batch files, and '*.png binary' to mark binary files so they're never converted. This ensures all contributors behave consistently regardless of personal settings. For an existing repository with mixed endings, add .gitattributes and run git add --renormalize . to normalise in one commit, and add that commit to the blame ignore list.

Likely follow-ups

- Why is .gitattributes better than autocrlf?
- How do you normalise an existing repository?

certs.eknathalabs.com

```
# Settings:
```

```
git config core.autocrlf true    # Windows: CRLF in working tree, LF in repo
```

```
git config core.autocrlf input   # macOS/Linux: LF everywhere, convert CRLF on add
```

```
git config core.autocrlf false   # no conversion (if you manage via .gitattributes)
```

```
# Recommended: use .gitattributes to be explicit (not per-dev settings):
```

```
cat .gitattributes
```

```
* text=auto eol=lf              # auto-detect text files, normalize to LF
```

```
*.sh text eol=lf               # shell scripts must be LF (breaks on CRLF)
```

```
*.bat text eol=crlf            # batch files must be CRLF
```

```
*.png binary                   # binary files: no conversion
```

```
# Fix existing CRLF in repo:
```

```
git add --renormalize .        # re-apply .gitattributes to all files
```

.gitattributes is the right solution — it defines line ending behavior at the repository level, independent of developer OS settings. autocrlf relies on each developer having the right setting, which breaks in mixed teams. text=auto eol=lf normalizes all text files to LF in the repository and on checkout — the modern standard for cross-platform projects. Shell scripts with CRLF line endings will fail on Linux ("command not found" errors from the invisible character). Binary files must be marked as such to prevent corruption from line ending conversion.

CI failures from line endings: A common source of mysterious CI failures: script works on Windows developer machine, fails on Linux CI runner with "/bin/bash: bad interpreter: No such file or directory" because the shebang line has appended. Enforce *.sh text eol=lf in .gitattributes to prevent this entirely.

📚 **Official docs**

- Git documentation

certs.eknathalabs.com

Q44 What is git bundle — how do you use it for air-gapped environments? Advanced & internals

🎙 Spoken answer · ~79 s

git bundle packages Git objects and references into a single file that can be transferred by any medium and used like a remote repository. It's the way to move repositories where there's no network connection between the machines, such as air-gapped or highly restricted environments, or where only file transfer is allowed, like removable media. Creating: git bundle create repo.bundle --all packages all branches and tags, giving a full backup or transfer; for incremental updates, git bundle create update.bundle main ^<last-synced-commit> or using a range like lastsync..main includes only new commits since the last transfer. Verify the bundle with git bundle verify, which checks it's valid and that the repository has the prerequisite commits. On the destination: git clone repo.bundle my-repo creates a repository from it; to update an existing repository, git fetch update.bundle or git pull update.bundle main, and you can add the bundle as a remote path. Use cases: seeding an air-gapped network with an open-source dependency or internal code, syncing between a secure and a connected environment through a controlled transfer process with scanning, offline backups of repositories, and handing a repository to someone with poor connectivity.

Likely follow-ups

- How do you create an incremental bundle?
- What does bundle verify check?

```
# Create a bundle of the full repository:
```

```
git bundle create repo.bundle --all   # everything: all branches, tags
```

```
git bundle create incremental.bundle HEAD ^origin/main  # only new commits
```

```
git bundle create release.bundle v1.2.0 v1.3.0  # between two tags
```

```
# Verify a bundle:
```

```
git bundle verify repo.bundle
```

```
# Clone from a bundle:
```

```
git clone repo.bundle myrepo
```

```
# Fetch from a bundle into existing repo:
```

```
git fetch repo.bundle main:refs/remotes/origin/main
```

```
# List what a bundle contains:
```

```
git bundle list-heads repo.bundle
```

certs.eknathalabs.com

git bundle creates a single file that contains a complete Git repository or a subset of it — it can be transferred via USB drive, email, or any file transfer mechanism. Use cases: transferring code to air-gapped networks (common in defense, government, financial), offline backup of a repository, sharing a specific range of commits without granting repository access. The bundle is treated like a remote: you can clone from it, fetch from it, and push to it. It is a complete git transport over a file rather than a network.

Infrastructure use case: Terraform modules or Ansible roles for air-gapped environments. CI/CD in a locked-down network: the pipeline bundle is created on an internet-connected machine, transferred physically, and applied on an isolated server. This pattern is common in regulated industries (finance, government, utilities) where production networks have no internet access.

📚 **Official docs**

- Git documentation

Q45 How do you set up and use Git aliases for productivity — what are the most useful ones? Advanced & internals

🎙 **Spoken answer · ~86 s**

Aliases are shortcuts for Git commands, defined in your configuration with git config --global alias.name 'command' or in the [alias] section of .gitconfig. They save typing, enforce good habits, and package complex commands. Useful examples: short forms like st for status, co or sw for switch, br for branch, and ci for commit. A readable log graph: lg = log --graph --oneline --decorate --all. unstage = restore --staged, to remove files from the index. last = log -1 HEAD --stat, to show the most recent commit. amend = commit --amend --no-edit. fixup = commit --fixup, paired with rebase -i --autosquash. pushf = push --force-with-lease, a safer force push. A tidy command for deleting merged branches using a shell alias prefixed with an exclamation mark. who = shortlog -sn for contributor counts. Aliases beginning with ! can run shell commands, allowing powerful multi-step helpers, like syncing a fork or pruning. Beyond aliases, useful settings: pull.rebase, fetch.prune true to remove stale remote-tracking branches, rerere.enabled, merge.conflictStyle zdiff3, push.autoSetupRemote, and init.defaultBranch. In teams, share recommended aliases via documentation or a dotfiles repository rather than a repository-level config, and don't rely on them in scripts, since they differ per machine. Keep them memorable and don't hide dangerous operations behind innocuous names.

Likely follow-ups

- Why prefer force-with-lease in an alias?
- How do shell aliases differ from simple ones?

```
git config --global alias.st "status -sb"
```

```
git config --global alias.lg "log --oneline --graph --all --decorate"
```

certs.eknathalabs.com

```
git config --global alias.last "log -1 HEAD --stat"
```

```
git config --global alias.unstage "reset HEAD --"
```

```
git config --global alias.undo "reset --soft HEAD~1"
```

```
git config --global alias.wip "commit -am WIP"
```

```
git config --global alias.unwip "reset HEAD~1"
```

```
git config --global alias.aliases "config --get-regexp alias"
```

```
git config --global alias.whoami "config user.email"
```

```
# Shell command aliases (prefix with !):
```

```
git config --global alias.recent "for-each-ref --sort=-committerdate refs/heads/ --
format="%(refname:short) %(committerdate:relative)""
```

```
git config --global alias.contributors "shortlog --summary --numbered --no-merges"
```

```
git config --global alias.nuke "!git reset --hard && git clean -fdx"
```

Aliases reduce cognitive overhead for frequently used command sequences. Store aliases in ~/.gitconfig and share them via a dotfiles repository so the team has a consistent Git experience. The wip/unwip pattern is a fast way to save and restore work-in-progress: git wip saves everything in a single commit, git unwip restores it. The nuke alias resets to a known clean state — useful in CI cleanup or development experiments. Aliases that start with ! execute as shell commands, enabling multi-step sequences.

> **Dotfiles repo:** Keep ~/.gitconfig, ~/.zshrc, and other config files in a dotfiles repository. New developers clone it and run an install script to get the full team configuration. This ensures everyone has the same aliases, hooks, and settings — eliminating "works on my machine" due to Git configuration differences.

> 📚 **Official docs**
>
> - Git documentation

certs.eknathalabs.com

Q46 How do you manage hotfixes in production — what is the workflow when main has moved ahead?

Workflows

🎙 Spoken answer · ~84 s

A hotfix is an urgent fix for a production problem that can't wait for the normal release train, and the complication is main may contain unreleased work. Workflow. Identify exactly what is running in production, by tag or commit, since that is the base. Create a hotfix branch from the production tag or release branch, not from main if main has moved ahead with unreleased features: git switch -c hotfix/payment-timeout v2.3.1. Make the smallest possible fix, with a test that reproduces the bug; keep the change focused to limit risk. Open a pull request against the release or production branch, get an expedited review, and let CI run. Build the artefact, tag it v2.3.2, deploy through the pipeline, using the same process as normal releases and verifying with monitoring, with a rollback ready. Then, crucially, forward-port the fix to main so it isn't reintroduced by the next release: cherry-pick the fix commit, or merge the hotfix branch back into main, resolving conflicts, and any other active release branches. Record it in the changelog. In trunk-based development with continuous deployment, the hotfix is simply a fast change to main, or a revert of the bad commit, deployed immediately, so the separation disappears.

Likely follow-ups

- Why branch from the production tag rather than main?
- What must you do after deploying the hotfix?

```
# Situation: production is running v2.3.0, main is at v2.5.0-dev
```

```
# Critical bug found in production that needs immediate fix
```

```
# Step 1: Create hotfix branch from the production tag
```

```
git checkout v2.3.0
```

```
git checkout -b hotfix/payment-crash-v2.3.1
```

```
# Step 2: Fix the bug, commit
```

```
git commit -m "fix(payments): prevent crash on null currency code"
```

```
# Step 3: Create release tag
```

```
git tag -a v2.3.1 -m "Hotfix release v2.3.1"
```

```
git push origin hotfix/payment-crash-v2.3.1 v2.3.1
```

```
# Step 4: Cherry-pick or merge the fix back to main
```

```
git checkout main
```

certs.eknathalabs.com

```
git cherry-pick <fix-commit-sha>    # port fix to main
```

```
git push origin main
```

```
# Step 5: Deploy v2.3.1 tag to production via CI/CD
```

Hotfix workflow: branch from the production tag (not main, which may have incompatible changes), apply the minimal fix, tag the hotfix release, push the tag (which triggers the CI/CD production deployment pipeline), and then cherry-pick the fix back to main so it is not lost. The critical mistake is forgetting step 4 — the hotfix gets deployed to production but the fix is never ported back to the development branch, re-introducing the bug in the next release. Set a team process: every hotfix must have a tracking ticket requiring a cherry-pick confirmation before the ticket is closed.

Test the hotfix: The temptation to skip testing under production pressure is real. At minimum: run the specific test case that exercises the bug, and deploy to staging first even if it takes 10 minutes. A bad hotfix deployed under pressure that causes a second outage is far worse than a 15-minute additional delay. Keep a fast-path staging environment for exactly this scenario.

📚 **Official docs**

- Git documentation

Q47 How do you use git format-patch and git am for email-based workflows? Collaboration

🎙 **Spoken answer · ~87 s**

git format-patch and git am support workflows where changes are exchanged as patch files, famously used by the Linux kernel through mailing lists. git format-patch turns commits into mbox-style patch files, one per commit, each containing the author, date, commit message, and diff. For example, git format-patch -3 produces patches for the last three commits, or git format-patch origin/main creates patches for all commits not in main; -o chooses the output directory, and --cover-letter generates an introductory message for a series. You can send them with git send-email, which mails them to the maintainers or list. The recipient applies them with git am, which reads the mailbox and creates commits preserving the original author, date, and message: git am *.patch or git am < mailbox. If a patch doesn't apply cleanly, git am stops; resolve conflicts, then git am --continue, or --skip, or --abort. The -3 option uses a three-way merge to apply more robustly. Contrast with git apply, which applies a diff to the working tree without creating commits, and git diff output without metadata. Where this is used: projects that review through mailing lists, such as the kernel and Git itself; sharing changes between disconnected systems or repositories without a shared remote; backporting patch series; and archiving.

Likely follow-ups

- What is the difference between git apply and git am?
- What does git am do on a failing patch?

certs.eknathalabs.com

```
git format-patch origin/main           # create .patch files for new commits
```

```
git format-patch -1 HEAD               # patch for just last commit
```

```
git format-patch HEAD~3..HEAD          # last 3 commits as patches
```

```
git format-patch --stdout > changes.patch  # single file
```

```
# Apply patches:
```

```
git am 0001-feat-add-payment-logging.patch
```

```
git am *.patch                         # apply all patches in order
```

```
git am --3way *.patch                  # use 3-way merge for conflicts
```

```
git am --abort                         # if it goes wrong
```

```
# Review a patch before applying:
```

```
git apply --check 0001-*.patch         # dry run (no changes)
```

```
git apply --stat 0001-*.patch          # show what would change
```

format-patch creates patches in the mbox email format — the original Linux kernel contribution workflow. Each patch is a self-contained commit with author, date, message, and diff. git am applies patches as commits, preserving authorship. Still used in: Linux kernel development, projects that accept contributions via email list, transferring commits between repositories without shared remote access, and distributing fixes to customers who cannot access your source repository. In air-gapped deployments, patches transfer specific code changes between environments where direct git push is not possible.

Modern equivalent: For most teams, PRs replaced the format-patch workflow. But knowing format-patch is valuable for: contributing to open-source projects that use mailing lists (Linux kernel, Git itself, many BSD projects), generating change documentation for security-sensitive deployments where reviewers need a portable patch file, and troubleshooting by reproducing exact commits in isolated environments.

📚 **Official docs**

- Git documentation

certs.eknathalabs.com

Q48 What is git bisect run — how do you automate regression hunting in CI?

Advanced & internals

🎙 Spoken answer · ~79 s

git bisect run automates the binary search for a regression by running a script at every step, instead of you manually testing each commit. You start with git bisect start, mark the known bad commit, usually HEAD, and a known good one, then git bisect run ./test.sh. The script's exit code tells Git the verdict: zero means good, any code between 1 and 127 except 125 means bad, and 125 means skip this commit because it can't be tested, for example it doesn't build. Git checks out each midpoint, runs the script, and continues until it prints the first bad commit; finish with git bisect reset. Writing a good script: it must be deterministic and fast, build the project if needed, and run the specific failing test or reproduction, not the whole suite. Make it return 125 when the build fails so unrelated broken commits don't mislead. Automating in CI: when a nightly or integration test fails, a job can bisect between the last passing and first failing commit, automatically identify the culprit commit and author, and open an issue or notify them. Many large organisations run automated culprit finding.

Likely follow-ups

- What exit codes does a bisect script use?
- Why can flaky tests mislead automated bisecting?

```
# Automated bisect with a test script:
```

```
cat test-regression.sh
```

```
#!/bin/bash
```

```
make build 2>/dev/null || exit 125   # exit 125 = skip this commit
```

```
./run-specific-test.sh payment-flow   # 0 = good, non-zero = bad
```

```
git bisect start HEAD v1.8.0         # bad=HEAD, good=v1.8.0
```

```
git bisect run ./test-regression.sh  # automated binary search
```

```
git bisect reset                     # clean up when done
```

```
# Exit codes for bisect run:
```

```
0    — commit is good (no regression)
```

```
1-127 (not 125) — commit is bad (has regression)
```

```
125  — skip this commit (cannot test, e.g., build fails)
```

certs.eknathalabs.com

```
# CI integration: run bisect as part of incident response
```

```
git bisect start --first-parent HEAD <last-known-good-tag>
```

exit 125 (special to bisect) skips commits where the test cannot run — build failures, missing dependencies, or intermediate states that are untestable. --first-parent on a merge-heavy repo restricts bisect to the mainline, ignoring merge commits where the regression was introduced via a side branch. For CI incident response, a fully automated bisect script can run overnight and identify the exact regression commit in a repository with 500 commits since the last known good release — all without human intervention.

Production incident automation: Integrate bisect into your incident runbook: when a regression is detected (monitoring alert), the on-call automation kicks off a git bisect run job in CI targeting the deployment range. By the time a human looks at the incident, the root commit is already identified. This is the highest-leverage use of git bisect.

📚 **Official docs**

- git-bisect
- Git documentation

Q49 How do you deal with a merge gone wrong after pushing — the "undo a published merge" scenario?

Recovery & undo

🎙 **Spoken answer · ~86 s**

If a bad merge is already pushed to a shared branch, don't rewrite history. Use git revert -m 1 <merge-commit>, which creates a new commit that undoes the changes the merge brought in, relative to the first parent, the mainline. Push it through a fast-tracked pull request, and the branch is back to a good state without disturbing anyone who has pulled. Important subtlety: Git records that the feature branch's commits are already merged, so after reverting the merge, merging the same branch again later will not re-apply those changes, because Git treats them as already integrated and your revert as a deliberate later change. To re-introduce the feature, first revert the revert commit, which restores the changes, and then merge any new commits from the branch; or create a new branch with the commits cherry-picked. If the problem is that the merge was done wrongly, for instance with a bad conflict resolution, the alternative is a fixing commit on top that corrects the files, which is simpler than reverting. Rewriting history with reset and force push is a last resort, justified only if sensitive data was merged, and it requires protection to be lifted, communication to all contributors, and everyone to reset their local branches.

Likely follow-ups

- Why can't you simply re-merge a reverted merge?
- When is rewriting history justified?

```
# Scenario: merged feature to main, pushed, but it broke production
```

cors.eknathalabs.com

```
# Option 1: git revert the merge commit (SAFE, preferred)
```

```
git log --merges --oneline -5    # find the merge commit SHA
```

```
git revert -m 1 <merge-commit-sha>  # -m 1 = mainline parent
```

```
git push origin main
```

```
# This undoes all changes from the merge as a new commit
```

```
# When the fix is ready, re-merge the feature branch:
```

```
git revert <revert-of-merge-sha>   # revert the revert (re-enables the feature)
```

```
git merge feature/broken-feature   # merge with actual fixes
```

```
# Option 2: Reset + force-push (DANGEROUS, coordinate with team)
```

```
git reset --hard <sha-before-merge>
```

```
git push --force-with-lease origin main
```

Reverting a merge commit requires -m to specify which parent to keep (mainline = 1). The result is a new commit that undoes all changes introduced by the merge. Critical follow-up: when you fix the feature branch and want to re-merge, you MUST first revert the revert. If you simply merge the feature branch again, Git sees that all those commits were already "merged" (the revert is a descendant) and only applies the new fix commits — the original feature code stays missing. This is the most common mistake in the "revert a bad merge" pattern.

Revert the revert: After reverting a merge, if you just merge the branch again later, Git thinks the original commits are already merged (by the revert) and skips them. You must: git revert <revert-commit> (revert the revert) to re-enable the original changes, THEN apply your fix. Forgetting this step results in the feature being silently absent from production despite the branch showing as merged.

📚 **Official docs**

- git-revert
- Git documentation

certs.eknathalabs.com

Q50 Walk through your methodology for onboarding a new repository to production Git standards.

Workflows

🎙 Spoken answer · ~88 s

Onboarding a repository to production standards is a checklist, and I'd automate it with a template or a configuration-as-code tool. Governance: define the owner team, set CODEOWNERS, and register the repository in the service catalogue. Branch protection or rulesets on main: required pull requests, minimum approvals, code owner review, dismiss stale approvals, required status checks, up-to-date or merge queue, block force pushes and deletions, and signed commits where needed. Access: least privilege through teams, SSO and MFA, no individual grants, and restricted admin rights. Hygiene files: README, LICENSE, CONTRIBUTING, a PR template, issue templates, .gitignore, .gitattributes for line endings and binary files, and .editorconfig. Quality gates: CI pipeline running build, tests, linting, and security scanning including secret scanning with push protection, dependency scanning, and SAST; pre-commit hooks managed via a framework. Commit conventions with commitlint and a defined merge strategy such as squash. Dependency automation using Dependabot or Renovate. Release process: semantic versioning, annotated tags, and automated changelog and releases. Deployment: pipeline and environment protection rules, with secrets stored per environment and OIDC rather than static keys. Observability of the repository: audit log streaming and alerts. Documentation of the workflow and runbooks. Finally, verify with a checklist or a compliance script that reports drift from the standard, and review periodically.

Likely follow-ups

- Which settings would you automate through a template?
- How do you detect repositories that drift from the standard?

This is the Principal-level systems thinking question — designing the Git governance model, not just using commands.

PRODUCTION GIT STANDARDS CHECKLIST

```
Phase 1 — Repository setup:
  Branch protection on main: require PR, 1+ approvers, status checks, signed commits
  CODEOWNERS file: map directories to owning teams
  .gitignore: comprehensive (OS files, IDE files, build artifacts, .env)
  .gitattributes: line endings, binary files, LFS configuration
  README.md: setup instructions, branching model, PR process
Phase 2 — Developer tooling:
  .pre-commit-config.yaml: secrets detection, lint, format, commit-msg
```

certs.eknathalabs.com

```
  PULL_REQUEST_TEMPLATE.md: standard PR description format

  Conventional commits enforced via commitlint

Phase 3 — CI/CD integration:

  Required status checks: tests, security scan, lint

  Semantic versioning: semantic-release or manual tag policy

  Branch → environment mapping: feature/* → preview, main → staging, tag → prod

Phase 4 — Security:

  GitHub Advanced Security: secret scanning + code scanning enabled

  Audit log streaming to SIEM or GCS

  Deploy keys or Workload Identity (no PATs with long expiry)

  Signed commits required (SSH or GPG)

Phase 5 — Documentation:

  Git workflow documented in team wiki (branching model, release process, hotfix process)

  Runbook: what to do if X (force push needed, secret exposed, branch deleted)
```

A well-governed Git repository is invisible to developers — the guardrails prevent mistakes without getting in the way. The goal: any engineer can contribute safely without knowing every edge case, because the tooling (hooks, branch protection, CI checks) catches problems before they reach production. The Principal Platform Engineer designs this system, documents it, and iterates based on incidents and team feedback.

Measure, then improve: Key Git health metrics: PR cycle time (open to merge), review response time, CI failure rate, force-push incidents, secret scanning alerts per month, branch age P95. Track these on a dashboard. An increase in PR cycle time signals friction in the review process. Force-push incidents signal a gap in branch protection configuration. Data drives improvements rather than opinions.

📚 **Official docs**

- Git documentation

certs.eknathalabs.com

Q51 A force-push overwrote a colleague’s commits on a shared branch. How do you recover them?

Recovery & undo

🎙 Spoken answer · ~86 s

A force push overwrites the remote branch pointer, but the old commits still exist in several places, so recovery is usually possible. Act quickly and calmly. First, stop further pushes to the branch, and tell the team. Sources to recover from. The colleague's own clone: they likely still have the commits locally, and their reflog or their branch has the previous state; they can simply push it back or push to a new branch. Anyone's remote-tracking branch from before the overwrite: git log origin/branch in a clone that hasn't fetched since. The reflog of your own clone if you had pulled previously: git reflog show origin/branch lists previous positions. The hosting platform: GitHub's push events in the audit log and the API show the before SHA for force pushes, and its UI may allow restoring; GitLab shows push events and has a protected-branch history; support can also help. CI logs often print the commit SHA that was built, and pull request pages retain the old commits in their timelines. Once you have the lost SHA, verify it with git show, then restore: git branch recovered <sha> and push that, or reset the shared branch back to it, then integrate the overwriting commits properly by merging or cherry-picking.

Likely follow-ups

- How does force-with-lease protect others' work?
- Where can the lost SHA usually be found?

Force-push does not delete the old commits immediately — they become unreferenced but stay in the object database until garbage collection. Recovery is almost always possible if you act before gc runs.

RECOVER THE LOST COMMITS

```
git reflog show origin/feature      # if you have a local tracking ref
```

```
# On the machine that had the old commits, the reflog is the goldmine:
```

```
git reflog                          # find the SHA before the force-push
```

```
git log -g --oneline <branch>       # reflog as log
```

```
# Restore the branch to the pre-force-push SHA:
```

```
git branch recovered-feature <good-sha>
```

```
git push origin recovered-feature
```

```
# If no local copy exists, ask the colleague — their reflog has it:
```

```
git fsck --lost-found               # find dangling commits server-side or local
```

certs.eknathalabs.com

The reflog is per-clone and local — it is not pushed. So the colleague whose work was overwritten almost certainly still has the commits in their own reflog or local branch. The fastest fix is usually to have them re-push, or to cherry-pick their dangling commits back on.

Prevention beats recovery: enable branch protection with "Require force pushes to be done via the API" disabled, or block force-push entirely on shared branches. Use `git push --force-with-lease` instead of `--force` — it refuses to overwrite if the remote moved since your last fetch, which would have prevented this incident.

📚 **Official docs**

- GitHub: Protected branches

Q52 You committed a secret (API key) and pushed it. What is the correct remediation?

Recovery & undo

🎙 **Spoken answer · ~88 s**

The correct remediation puts revoking first and history cleanup second. Step one: assume it's compromised immediately, because automated scanners harvest public pushes within minutes and even private repositories can be cloned by others. Revoke or rotate the key right away, issuing a new one, and update the systems that use it. Step two: check the provider's logs for any use of the old key since it was exposed, and if there's any suspicious activity, escalate to a full incident, looking for new resources, data access, and persistence. Step three: remove it from the repository, which is hygiene rather than protection. Remove it from the current files, replace it with a reference to a secret manager or environment injection. To purge history, use git filter-repo or BFG to rewrite all commits containing it, force push every affected branch and tag, and ask the team to re-clone or reset. Clean up other copies: forks, pull request refs, which on GitHub may need a support request to purge, CI logs and artefacts, caches. Step four: prevent recurrence with pre-commit secret scanning, platform push protection, .gitignore for configuration files, and by preferring workload identity or short-lived credentials so there's no long-lived key to leak. Step five: document and learn from it in a blameless review.

Likely follow-ups

- Why is rotation the real remediation?
- What other places can the secret linger after history is rewritten?

Order matters: rotate first, scrub second. The moment a secret hits a remote, assume it is compromised — scrubbing history does not un-leak it.

STEP 1 — ROTATE IMMEDIATELY

```
# Revoke/rotate the leaked credential at the provider. This is non-negotiable
```

```
# and comes BEFORE any git surgery. The key is already public.
```

certs.eknathalabs.com

STEP 2 — PURGE FROM HISTORY

```
# git-filter-repo is the modern, recommended tool (filter-branch is deprecated):
```

```
git filter-repo --path config/secrets.yml --invert-paths   # remove a file
```

```
git filter-repo --replace-text expressions.txt             # redact a string
```

```
# expressions.txt:  AKIA1234567890==>REDACTED
```

```
# Then force-push the rewritten history:
```

```
git push origin --force --all
```

```
git push origin --force --tags
```

Everyone with a clone must re-clone or hard-reset, because their history still contains the secret. Open PRs based on old history must be recreated. Coordinate this — a half-purged history where some forks still carry the secret is the common failure.

The secret may already be cached: GitHub keeps unreferenced commits accessible via their SHA for a window, and forks/caches/CI logs may retain it. Rotation is the only reliable mitigation. Add secret scanning (gitleaks, GitHub secret scanning) and a pre-commit hook so it never happens again.

📚 **Official docs**

- git-filter-repo
- gitleaks

certs.eknathalabs.com

Q53 How do you recover a deleted branch when you do not remember its last commit SHA?

Recovery & undo

🎙 Spoken answer · ~87 s

If I don't remember the SHA of a deleted branch's tip, I search for it using Git's records. First try the reflog: git reflog lists where HEAD has been, and I look for the last entries when I was on that branch, such as commits with recognisable messages or a checkout moving away from the branch name; I can also filter with git reflog | grep branchname. Once I spot the right entry, I verify with git show <sha> or git log <sha>, then recreate the branch with git branch recovered <sha>. If the branch was never checked out in this clone, or the reflog has expired, I look for unreachable commits with git fsck --lost-found, or git fsck --unreachable | grep commit, which lists dangling commits; then I inspect each candidate with git show and look at dates, authors, and messages to identify the tip, preferring the dangling commit that has no children, since that's the end of the chain. Other places: if the branch was ever pushed, check remote-tracking refs in clones, and the hosting platform: GitHub's pull request page keeps the branch commits and offers a restore branch button for merged or closed pull requests, and the repository's activity view shows deleted branches with the SHA.

Likely follow-ups

- What does a dangling commit with no children represent?
- Where does the hosting platform keep deleted branch info?

A branch is just a ref — a pointer to a commit. Deleting it removes the pointer, not the commits. The reflog records every position HEAD and branches have held.

```
git reflog                          # scan for the last commit on the deleted branch
```

```
git reflog --date=iso               # add timestamps to locate it
```

```
# Once you find the SHA (e.g. the 'checkout: moving from feature-x' entry):
```

```
git branch feature-x <sha>          # recreate the branch at that commit
```

```
# If reflog was pruned, search dangling commits:
```

```
git fsck --no-reflogs --lost-found
```

```
git fsck --unreachable | grep commit
```

The reflog entry you want usually reads like `checkout: moving from feature-x to main` — the SHA on that line is the tip of the branch right before you left it. `git fsck --lost-found` is the fallback when the reflog itself has been pruned (default expiry is 90 days for reachable, 30 for unreachable).

Interview signal: knowing that "delete" in Git almost never means "gone" — and that the reflog + fsck + the 90/30-day gc grace windows are your safety net — shows you understand Git is append-only at the

certs.eknathalabs.com

object level.

📚 **Official docs**

- git-reflog

Q54 A bad commit is buried 20 commits back on main and is already deployed. How do you undo it safely?

Recovery & undo

🎙 **Spoken answer · ~85 s**

When a bad commit is buried twenty commits back and already deployed, I use git revert, which creates a new commit that reverses that one commit's changes, without touching the commits after it. Find the culprit with git log, git bisect, or blame, then git revert <hash>. The revert may conflict if later commits touched the same lines; resolve conflicts carefully, thinking about what the later code now depends on. Test: run the test suite and confirm the specific bad behaviour is fixed and nothing else broke. Open a pull request, get an expedited review, merge, and deploy through the normal pipeline, monitoring metrics and logs. If the situation is urgent and the revert is tricky, consider alternatives in production terms: roll back the deployment to the previous artefact or tag immediately, to restore service, then fix forward calmly; or disable the feature using a flag. Do not use reset and force push on main: it would erase the other nineteen good commits' ordering, or force everyone to resynchronise, and break the audit trail. If the bad commit also included a database migration or data change, reverting the code may not undo the data effects, so plan a forward migration or data repair.

Likely follow-ups

- Why not reset and force push on main?
- What if the bad commit included a migration?

On a shared, deployed branch you must not rewrite history — use git revert, which creates a new commit that inverses the bad one. Reset/rebase would force-push and break everyone.

```
git revert <bad-sha>                # creates an inverse commit, safe on shared branches
```

```
git revert <sha1> <sha2> <sha3>     # revert several, newest-first ordering matters
```

```
git revert -m 1 <merge-sha>         # revert a merge commit, -m picks the mainline parent
```

```
git revert --no-commit <sha>        # stage the inverse without committing (batch)
```

`git revert` is the production-safe undo: it moves forward, never rewrites, so it is reproducible across every clone and deploy. For a merge commit you must specify the parent to keep with `-m 1` (usually mainline). Reverting a merge has a subtle gotcha: re-merging that branch later will skip the reverted changes — you typically revert-the-revert or rebase to bring them back.

certs.eknathalabs.com

reset vs revert: `git reset --hard` rewrites history and is only safe on private, unpushed branches. On main/shared branches always `revert`. Confusing the two is how teams lose work.

📚 **Official docs**

- git-revert
- Git documentation

Q55 Walk me through git bisect to find which commit introduced a regression.

History & commits

🎙 **Spoken answer · ~76 s**

Bisect finds the commit that introduced a regression by binary search. Setup: I need a good commit where the behaviour was correct, such as the last release tag, and a bad commit, usually HEAD, plus a reliable way to test. Steps: git bisect start; git bisect bad, marking the current commit as bad; git bisect good v2.1.0, marking the older good one. Git checks out the commit halfway between, and tells me roughly how many steps remain. I build and test at this commit, then tell Git git bisect good or git bisect bad; if the commit can't be tested, I use git bisect skip. Git narrows the range and repeats; after log2 of N steps, around ten for a thousand commits, it prints the first bad commit. Then I examine it with git show, look at the diff, message, author, and linked pull request to understand the cause. Finally, git bisect reset returns me to my original branch and state. To automate, git bisect run with a script that exits zero for good and non-zero for bad, returning 125 for untestable commits.

Likely follow-ups

- What if some commits don't build during bisect?
- How many steps does bisect need for 1,000 commits?

git bisect does a binary search across history to find the first bad commit — O(log n) instead of checking each commit. With 1000 commits, that is ~10 tests instead of 1000.

```
git bisect start
```

```
git bisect bad                    # current commit is broken
```

```
git bisect good v2.3.0            # this older tag/commit was fine
```

```
# Git checks out a midpoint; you test, then mark:
```

```
git bisect good   # or  git bisect bad
```

```
# repeat until Git prints 'X is the first bad commit'
```

```
git bisect reset                  # return to original HEAD
```

certs.eknathalabs.com

AUTOMATE IT

```
git bisect run ./test.sh           # script: exit 0 = good, non-zero = bad, 125 = skip
```

```
git bisect run pytest tests/test_regression.py
```

The killer feature is `git bisect run` with a test script — Git drives the entire search automatically, marking each midpoint by the script's exit code. A script returning 125 tells Git to skip an untestable commit (e.g. one that does not build).

Production use: bisect plus a reliable reproduction script turns "somewhere in the last release something broke" into a precise commit + author + diff in minutes. Mention pairing it with a minimal repro test — that is what makes it fast and trustworthy.

📚 **Official docs**

- Google SRE Book: Effective Troubleshooting
- git-bisect
- Git documentation

Q56 What does git rebase --onto do, and when would you use it in production?

History & commits

🎙 **Spoken answer · ~88 s**

git rebase --onto moves a range of commits to a different base. Its form is git rebase --onto <newbase> <oldbase> <branch>: the commits that are in the branch but not in oldbase are replayed on top of newbase. In production the most common use is cleaning up stacked or mis-based branches. Scenario one: feature-b was started from feature-a, and feature-a was then squash-merged into main. Feature-b still contains feature-a's original commits, which no longer match anything in main, so a normal rebase would replay them and conflict. Run git rebase --onto main feature-a feature-b, meaning take what's on feature-b after feature-a and put it on main, so only feature-b's own commits are moved. Scenario two: I branched from the wrong branch, such as develop instead of main, and need to move my commits to the correct base without dragging along develop's changes. Scenario three: surgically removing a range of commits from a branch's history. Scenario four: moving a hotfix commit series from one release line to another. Precautions: it rewrites history, so use it on your own unshared branches or coordinate; draw the commit graph and confirm the three arguments; make a backup branch first, or rely on the reflog; verify with git log --graph; run tests; and push with --force-with-lease.

Likely follow-ups

- Why does a normal rebase fail after a squash merge of the parent branch?
- What three arguments does --onto take?

`git rebase --onto` transplants a range of commits onto a new base — it answers "move these specific commits somewhere else," which plain rebase cannot express cleanly.

certs.eknathalabs.com

```
git rebase --onto <newbase> <upstream> <branch>
```

```
# Example: you branched feature off the wrong base (dev), want it on main:
```

```
git rebase --onto main dev feature
```

```
# Drop a range of commits: replay commits AFTER <bad> onto <bad>'s parent:
```

```
git rebase --onto <bad>~1 <bad> feature
```

```
# Move a sub-branch off a feature that itself got rebased:
```

```
git rebase --onto main old-feature-tip sub-feature
```

Read it as: take the commits in `(upstream, branch]` and replay them starting from `newbase`. The classic production use is fixing a branch that was cut from the wrong parent, or surgically removing a middle commit by replaying everything after it onto its parent.

Only on private branches. Like all rebase, this rewrites commit SHAs. Never rebase commits others have already pulled — coordinate or use a fresh branch.

📚 **Official docs**

- git-rebase
- Git documentation

certs.eknathalabs.com

Q57 How do you split one large commit into several smaller logical commits?

History & commits

🎙 Spoken answer · ~86 s

Splitting a large commit into logical pieces makes review, revert, and bisect easier. If the commit is the latest, reset it softly: git reset HEAD~1 (mixed) un-commits it but keeps all the changes in the working tree, unstaged. Then stage and commit in pieces: git add -p to select hunks interactively, answering y, n, or s to split a hunk further, or e to edit it manually; commit with a clear message; repeat for the next logical group; and verify each intermediate commit builds and passes tests, perhaps using git stash --keep-index to test the staged portion alone. If the commit is further back in history, use interactive rebase: git rebase -i <commit>^, mark the target commit as edit, and when the rebase stops there, run git reset HEAD^ to undo the commit while keeping the changes, then create the several smaller commits as above, and finally git rebase --continue to replay the remainder. For splitting by file, git add path then commit works. Tips: separate refactoring, formatting, and behavioural changes into distinct commits; put tests with the code they test; and order the commits so the history reads as a story. After rewriting, if the branch was already pushed, force push with --force-with-lease, and tell reviewers.

Likely follow-ups

- How does git add -p help split a commit?
- How do you split a commit that isn't the latest?

Use interactive rebase to mark the commit for editing, reset it to unstage its changes, then re-commit in logical chunks.

```
git rebase -i <commit>~1          # opens the todo list
```

```
# change 'pick' to 'edit' (or 'e') for the target commit, save
```

```
# Git stops at that commit. Undo it but keep the changes staged-then-unstaged:
```

```
git reset HEAD~                   # commit's changes now in working tree, unstaged
```

```
# Now stage and commit in pieces:
```

```
git add -p file1.py               # interactively stage hunks
```

```
git commit -m 'refactor: extract validation helper'
```

```
git add file2.py && git commit -m 'feat: add retry logic'
```

```
git rebase --continue             # finish the rebase
```

certs.eknathalabs.com

`git add -p` (patch mode) is the key tool — it lets you stage individual hunks within a file, so one messy commit becomes several clean, reviewable ones. This is how you turn "WIP: everything" into a tidy, bisectable history before opening a PR.

Why it matters: small, single-purpose commits make code review faster, bisect precise, and reverts surgical. Interviewers ask this to see if you treat history as a communication tool, not just a save button.

📚 **Official docs**

- git-rebase
- Git documentation

Q58 Explain git cherry-pick and a real scenario where it is the right tool.

History & commits

🎙 **Spoken answer · ~87 s**

Cherry-pick applies the changes from a chosen commit onto your current branch as a new commit, with the same message and author but a new hash. It computes the commit's diff against its parent and applies it. The right tool when you need that specific change without bringing along the rest of its branch. A real scenario: a critical security fix lands on main, but production runs release/2.3, which is two minor versions behind, and you can't take everything from main. You switch to release/2.3, run git cherry-pick -x <fix-commit>, resolve any conflicts caused by code drift, test, and ship a patch release. The -x flag records the original commit hash in the message for traceability. Another scenario: someone committed a fix to the wrong branch, so you cherry-pick it to the right one and remove it from the wrong one. Or you want one useful commit from an abandoned feature branch. Risks: it creates duplicate commits with different hashes, so history doesn't show they're the same, and later merging the branches can produce confusing conflicts; the commit might depend on earlier changes that weren't picked, applying cleanly but failing at runtime, so always test; and overuse leads to branches diverging in ways that are hard to audit.

Likely follow-ups

- Why is the -x flag recommended on release branches?
- What can go wrong with dependent commits?

cherry-pick applies the diff introduced by a specific commit onto your current branch, creating a new commit with a new SHA. It copies a change without merging the whole branch.

```
git cherry-pick <sha>              # apply one commit here
```

```
git cherry-pick <sha1>..<sha2>     # a range (exclusive of sha1)
```

```
git cherry-pick -x <sha>           # record 'cherry picked from commit ...' in message
```

```
git cherry-pick -n <sha>           # apply without committing (stage only)
```

certs.eknathalabs.com

```
git cherry-pick --abort            # bail out on conflict
```

REAL SCENARIO — HOTFIX TO MULTIPLE RELEASE BRANCHES

```
# A security fix lands on main. You must also ship it on release/2.3 and release/2.4:
```

```
git checkout release/2.3 && git cherry-pick -x <fix-sha>
```

```
git checkout release/2.4 && git cherry-pick -x <fix-sha>
```

The canonical production use is back-porting a hotfix from main to one or more maintenance release branches. `-x` records the original SHA in the message, giving an audit trail of where the change came from — valuable when the same fix exists in several branches.

Cherry-pick duplicates commits (new SHA), which can cause confusing "already applied" conflicts when those branches later merge. Prefer merging where possible; cherry-pick when you need only one change, not the whole branch.

📚 **Official docs**

- git-cherry-pick
- Git documentation

certs.eknathalabs.com

Q59 You ran git reset --hard and lost uncommitted work. Is anything recoverable?

Recovery & undo

🎙 Spoken answer · ~80 s

It depends on whether the lost work was ever staged or committed. Git hard reset discards changes in the working tree and index, so untracked and never-staged modifications are generally gone, since Git never stored them. However, anything that was committed is recoverable through the reflog: git reflog shows previous HEAD positions, and git reset --hard HEAD@{1} or a specific hash restores the previous state. If you had staged files with git add before the reset, Git wrote those file contents to the object database as blobs, even though there's no commit referencing them; they are dangling objects. You can find them with git fsck --lost-found, which writes the dangling blobs into .git/lost-found/other, where you can inspect their content and copy back the files you need, though without filenames, so you'll have to identify them by content. This works until garbage collection prunes them. For truly unstaged edits, look outside Git: your editor or IDE's local history, such as VS Code's timeline or JetBrains Local History, the operating system's backups and trash, snapshots from tools like Time Machine, and file system snapshots. Editors with autosave may have temporary or swap files too.

Likely follow-ups

- What are dangling blobs and how do you find them?
- Which uncommitted changes are unrecoverable by Git alone?

Committed work is recoverable via reflog. Truly uncommitted, never-staged changes are usually gone — but staged changes may survive in dangling blobs.

```
# If the work was ever committed (even via a WIP commit):
```

```
git reflog && git reset --hard <sha>
```

```
# If it was staged (git add) but not committed, the blobs may be dangling:
```

```
git fsck --lost-found
```

```
ls .git/lost-found/other/           # recovered blob contents land here
```

```
git cat-file -p <blob-sha>          # inspect a recovered blob
```

The honest answer: `reset --hard` on changes that were only in the working tree (never `git add`ed, never committed) destroys them — Git never had a copy. If you had staged them, `git fsck --lost-found` can surface the blobs because add writes objects to the database. This is the strongest argument for committing early and often, even as throwaway WIP.

Habit that saves you: commit WIP frequently (you can always squash later), and consider `git stash`

certs.eknathalabs.com

before destructive operations — stashes are recoverable via reflog too.

📚 **Official docs**

- git-reset
- Git documentation

Q60 What is the difference between git reset --soft, --mixed, and --hard?

History & commits

🎙 **Spoken answer · ~83 s**

git reset moves the current branch pointer to a specified commit, and the mode controls what happens to the index and working tree. --soft moves only the branch pointer: the index and working tree stay as they were, so the changes from the undone commits appear as staged and ready to commit again. Useful to combine several commits into one or to redo a commit message, such as git reset --soft HEAD~3 then commit once. --mixed, the default, moves the branch pointer and resets the index to match, but leaves the working tree alone, so the changes remain in your files but unstaged. Useful for un-committing and re-staging selectively, such as splitting a commit. --hard moves the pointer and resets both the index and the working tree to the target commit, so all uncommitted changes to tracked files are lost. Useful to throw away local work and match a known state, such as git reset --hard origin/main after fetching, but dangerous because uncommitted work can't be recovered by Git. Related: --keep resets but refuses if it would overwrite local modifications, a safer hard reset; git restore and git checkout -- file affect only files, not the branch pointer.

Likely follow-ups

- Which reset mode is destructive for uncommitted work?
- What does reset --keep do?

All three move the branch pointer (HEAD) to a target commit; they differ in what they do to the staging area (index) and the working tree.

```
git reset --soft <sha>    # move HEAD only; index + working tree untouched
```

```
git reset --mixed <sha>   # move HEAD + reset index; working tree untouched (DEFAULT)
```

```
git reset --hard <sha>    # move HEAD + reset index + reset working tree (DESTRUCTIVE)
```

MENTAL MODEL

```
--soft  : changes from undone commits become STAGED (ready to recommit)
```

```
--mixed : changes become UNSTAGED but kept in files
```

```
--hard  : changes are DISCARDED entirely
```

certs.eknathalabs.com

Common uses: `--soft HEAD~1` to undo the last commit but keep everything staged (to recommit with a better message or combine). `--mixed` (default) to unstage everything while keeping edits. `--hard` only when you genuinely want the working tree to match the target commit and accept losing local changes.

--hard is the dangerous one and the only one that touches your working files. Always check `git status` first and consider `git stash` as a reversible alternative.

📚 **Official docs**

- git-reset
- Git documentation

Q61 **How do you abort or recover from a merge or rebase that went wrong mid-operation?** Recovery & undo

🎙 **Spoken answer · ~75 s**

When a merge, rebase, cherry-pick, or similar operation goes wrong midway, Git records the in-progress state, and you can leave it cleanly. First, check git status, which tells you which operation is in progress and how to continue or abort. To abort and go back to exactly the state before starting: git merge --abort, git rebase --abort, git cherry-pick --abort, git revert --abort, or git am --abort. These discard any partial resolution work, so if you've invested effort resolving conflicts and then realise something is wrong, consider whether you can instead fix forward. For a rebase, you can also use --skip to drop the currently conflicting commit, which is appropriate if its changes are already upstream. If you've already completed the rebase or merge and find it's wrong, recover with the reflog: git reflog shows the pre-operation position, and git reset --hard ORIG_HEAD often works immediately, since Git sets ORIG_HEAD before risky operations; otherwise use the specific reflog entry such as HEAD@{5}. If you pushed a bad result, don't rewrite shared history: revert, or fix with a corrective commit.

Likely follow-ups

- What is ORIG_HEAD?
- When would you skip a commit during rebase?

Git keeps a record of where you started before the operation. Both merge and rebase have explicit abort commands, and ORIG_HEAD / reflog are the fallback.

```
# Mid-merge with conflicts you want to bail on:
```

```
git merge --abort                  # restores pre-merge state
```

```
# Mid-rebase gone wrong:
```

```
git rebase --abort                 # restores pre-rebase state
```

certs.eknathlabs.com

```
# Already finished a bad rebase/merge and want the old tip back:
```

```
git reset --hard ORIG_HEAD          # ORIG_HEAD = where you were before
```

```
git reflog && git reset --hard <pre-op-sha>   # if ORIG_HEAD was overwritten
```

`--abort` is the clean in-progress escape hatch. If the operation already completed and you only realize afterward it was wrong, `ORIG_HEAD` points at the commit you were on before the last merge/rebase/reset — a one-command rescue. The reflog is the universal backstop.

Interview depth: mentioning `ORIG_HEAD` (auto-set by merge, rebase, reset, pull) distinguishes you — most people only know `--abort` and panic once the operation has finished.

📚 **Official docs**

- git-rebase

Q62 **When would you use git revert vs git reset for undoing changes, in a team context?** History & commits

🎙 **Spoken answer · ~86 s**

The choice depends on whether the commits have been shared. git revert creates a new commit that applies the inverse of an earlier commit, leaving history intact. It's safe for shared branches because it only adds history: anyone who has pulled simply receives another commit. It also documents the undo in the log, which is good for audit and incident trails. Use revert for anything already pushed to a shared or protected branch such as main, production releases, or work others may have based changes on. git reset moves a branch pointer backward, effectively erasing commits from the branch's history; combined with force pushing, it rewrites shared history, which forces teammates to reconcile diverged branches, can lose their work, and breaks the audit trail. It's appropriate for local, unpushed commits: cleaning up before sharing, undoing a mistaken commit, or discarding experiments. Rule of thumb: local and private, reset or amend; public and shared, revert. Exceptions where reset plus force push on shared history is justified: removing a leaked secret or large accidental file, with coordination, communication, and temporary lifting of protection, and with rotation of the secret anyway. Nuances: reverting a merge commit needs -m to specify the parent, and re-merging later requires reverting the revert.

Likely follow-ups

- Why is revert safer on shared branches?
- When is a force push after reset justified?

The deciding question: has the commit been shared (pushed to a branch others use)? Shared → revert. Private/unpushed → reset is fine.

```
git revert <sha>     # NEW inverse commit, history preserved, SHAs stable — SHARED branches
```

coris.eknathalabs.com

```
git reset <sha>      # MOVES branch pointer, rewrites history — PRIVATE branches only
```

On `main` or any branch teammates have pulled, `reset` rewrites history and forces everyone to recover their clones — it breaks the team. `revert` moves forward by adding an inverse commit, so it is reproducible everywhere and leaves an auditable trail of "we undid this, here is why." On your own un-pushed feature branch, `reset` is the cleaner choice because nobody else depends on those SHAs yet.

Rule of thumb: "reset rewrites the past, revert writes a new future." If anyone else might have the commits, never rewrite — revert.

📚 **Official docs**

- git-reset
- git-revert
- Git documentation

Q63 A teammate's rebase created duplicate commits after a merge. How do you diagnose and clean it up?

Recovery & undo

🎙 **Spoken answer · ~81 s**

Duplicate commits typically appear when someone rebases a branch that others have already merged or based work on, and then history gets reconciled with the old versions. For example, a teammate rebases a feature branch after you pulled it, force pushes, and you then merge your old copy with their rewritten one: both the old commits and their rebased equivalents, with different hashes but identical changes, now appear in history, often with conflicts or repeated changes. Diagnose: git log --oneline --graph shows pairs of commits with the same message and author; git cherry -v upstream branch lists commits and marks those whose patch-ids already exist upstream with a minus sign, identifying equivalent duplicates; git range-diff compares two versions of a series of commits. Cleanup options depend on whether the duplicates are pushed to a shared branch. If only on a feature branch and not yet merged to main: do an interactive rebase on top of the correct base and drop the duplicate commits, or use git rebase --onto to move only the unique commits, then force push with --force-with-lease after coordinating. Actually, plain git rebase usually detects and skips commits already upstream by patch-id.

Likely follow-ups

- What does git cherry show?
- How does rebase detect commits already upstream?

Duplicate commits (same change, different SHAs) appear when a branch is rebased after some of its commits already reached the target via merge or cherry-pick. Git sees them as distinct objects.

```
git log --oneline --graph main feature   # visualize the divergence
```

certs.eknathalabs.com

```
git cherry -v main feature              # shows commits in feature not in main (- =
already there)
```

```
git range-diff main...old-feature main...feature   # compare two versions of a branch
```

CLEAN UP

```
# Interactive rebase to drop the duplicated commits:
```

```
git rebase -i main
```

```
# delete the 'pick' lines for commits already present upstream
```

```
# Or, if main already has everything, just fast-forward and abandon the dupes:
```

```
git checkout feature && git reset --hard main   # if feature should equal main
```

`git cherry` and the newer `git range-diff` are the diagnostic tools — they tell you which commits are genuinely new versus already-applied-with-a-different-SHA. Once identified, an interactive rebase drops the duplicates.

Root cause: this usually means the team mixes merge and rebase on the same branch without a clear policy. Pick one integration strategy per branch and document it to prevent recurrence.

📚 **Official docs**

- git-rebase

certs.eknathalabs.com

Q64 Explain the reflog: what it records, where it lives, and its expiry behavior.

History & commits

🎙 Spoken answer · ~79 s

The reflog is Git's local journal of reference movements. It records every time HEAD or a branch tip changes: commits, checkouts, merges, rebases, resets, amends, cherry-picks, and pulls, each with the old and new hash, a timestamp, and a description. It lives in .git/logs: .git/logs/HEAD for HEAD movements, and .git/logs/refs/heads/<branch> and refs/remotes for individual branches, shown with git reflog or git reflog show <ref>. Entries are addressed as HEAD@{n} for the nth previous position, or with time expressions such as main@{2.days.ago}. It's local to each clone: it's not pushed, shared, or cloned, so it's useless for a fresh clone, and it's not a backup. It records only committed or checked-out states, not uncommitted work. Expiry: entries are pruned by git gc based on configuration, by default gc.reflogExpire of ninety days for entries still reachable from the current tip, and gc.reflogExpireUnreachable of thirty days for entries whose commits are unreachable from the tip. Therefore commits discarded by a reset or deleted branch are normally recoverable for at least thirty days, until pruned. You can tune these settings, or explicitly expire with git reflog expire, but doing so reduces your safety net.

Likely follow-ups

- Is the reflog shared when you push?
- How long do unreachable entries survive by default?

The reflog is a local, per-clone log of every position your refs (HEAD and branches) have pointed to. It is your time machine and the foundation of nearly all Git recovery.

```
git reflog                          # HEAD reflog (default)
```

```
git reflog show main               # a specific branch's reflog
```

```
git reflog --date=iso              # with timestamps
```

```
git log -g                          # reflog formatted as full log
```

```
cat .git/logs/HEAD                  # the raw reflog file
```

It records resets, checkouts, commits, merges, rebases, pulls — every ref movement, with the before/after SHA and a reason. Crucially it is **local and never pushed**, so it survives operations that rewrite shared history, and each developer has their own. Expiry defaults: reachable entries are pruned after 90 days (`gc.reflogExpire`), unreachable after 30 (`gc.reflogExpireUnreachable`).

Why interviewers love it: "How would you recover X?" almost always has the same root answer — the reflog. Understanding it is local, append-only, and time-bounded shows you know why Git rarely loses committed work.

certs.eknathalabs.com

📚 **Official docs**

- git-reflog

Q65 Compare trunk-based development with GitFlow. When would you choose each in production?

Workflows

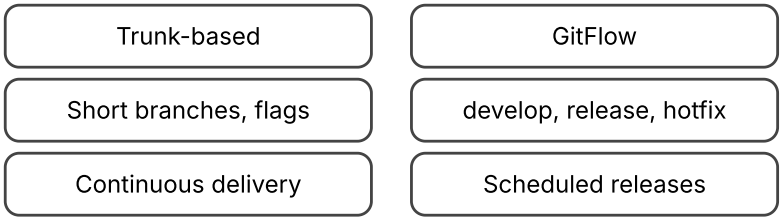
🎙 **Spoken answer · ~83 s**

The two strategies reflect different release cultures. Trunk-based development has everyone integrate into main at least daily through very short-lived branches or direct commits, with unfinished work hidden behind feature flags, and releases cut from the trunk or its tags. It minimises merge pain, keeps integration continuous, and correlates with high delivery performance in DORA research. It requires a strong automated test suite, fast CI, feature flags, and discipline, plus a culture of small changes. GitFlow uses long-lived main and develop branches with feature, release, and hotfix branches; features merge to develop, releases are stabilised on a release branch, then merged to main and tagged. It offers clear structure for scheduled releases and maintaining multiple versions, but brings overhead, delayed integration, bigger merges, and confusion about where fixes belong. I'd choose trunk-based for web services and SaaS products with continuous or frequent deployment, microservices, and teams with good CI and testing; it fits a platform that deploys many times a day. I'd choose GitFlow, or a simplified release-branch model, for software distributed in versions that customers install, like mobile apps, on-premises products, and libraries needing parallel support of several versions, or for teams with heavy manual QA gates.

Likely follow-ups

- Why does trunk-based development depend on feature flags?
- Which fits software delivered as installable versions?

🗺 **Trunk vs GitFlow**



They optimize for different release cadences. Trunk-based suits continuous deployment; GitFlow suits versioned, scheduled releases.

TRUNK-BASED DEVELOPMENT

```
main is always releasable; short-lived branches (hours/days) merge fast
```

```
Feature flags hide incomplete work instead of long-lived branches
```

certs.eknathalabs.com

```
Pairs with CI/CD: every merge can deploy; small batches, low merge pain
```

GITFLOW

```
Long-lived develop + main; feature/* release/* hotfix/* branches
```

```
Explicit release branches for stabilization before a versioned ship
```

```
Heavier process; more merge overhead; suits desktop/mobile/firmware releases
```

For a SaaS shipping many times a day, **trunk-based** wins — long-lived branches cause painful merges and hide integration risk, while feature flags decouple deploy from release. **GitFlow** earns its complexity when you ship discrete versions on a schedule (mobile app store releases, on-prem software) and need a dedicated stabilization window.

> **Staff-level nuance:** the question behind the question is "do you understand that branching strategy follows deployment model, not fashion?" Trunk-based without CI maturity and feature flags is dangerous; GitFlow on a high-velocity SaaS is needless friction.

> 📚 **Official docs**
>
> - Trunk Based Development
> - Atlassian: Gitflow workflow

certs.eknathalabs.com

Q66 How would you structure Git for a monorepo with 200 engineers? What breaks at scale?

Collaboration

🎙 Spoken answer · ~88 s

With 200 engineers in a monorepo, the first thing to break is performance and the second is process. Git operations slow with a huge number of files and history: clones take long, status and checkout lag. Mitigate with partial clone, sparse checkout, shallow clones in CI, the filesystem monitor and untracked cache, scheduled git maintenance with commit-graph, and tools like Scalar or a virtual filesystem. The main branch becomes a contention point: dozens of merges an hour cause constant rebases, and broken main from semantic conflicts. Use a merge queue that tests each pull request against the latest main plus queued changes before merging. CI cost and time explode if every change builds everything: adopt a build system with dependency graph awareness and remote caching, such as Bazel, Nx, Pants, or Turborepo, to build and test only affected targets, and path-based triggers. Ownership: use CODEOWNERS and directory-level responsibilities so reviews route correctly and sensitive areas have required reviewers. Access control is coarse; consider whether everyone can see all code, and add tooling if needed. Release and versioning: decide between a single version for everything and independent versions with tooling. Code health: shared libraries need compatibility policies and large-scale refactoring tools. Dependency management with a single version policy avoids diamond problems.

Likely follow-ups

- What does a merge queue solve at this scale?
- How do you avoid building everything on every change?

At scale, vanilla Git struggles with repo size, checkout time, and CI fan-out. The strategy is partial adoption of scaling tooling plus ownership boundaries.

SCALING TECHNIQUES

```
git clone --filter=blob:none <url>   # partial clone: fetch blobs on demand
```

```
git sparse-checkout set path/to/team  # check out only your subtree
```

```
git maintenance start                 # background gc, commit-graph, prefetch
```

```
# Commit-graph + multi-pack-index speed up log/blame on huge histories
```

OWNERSHIP & CI

```
CODEOWNERS                            # auto-assign reviewers per path
```

```
Affected-only CI: build/test only what changed (Bazel, Nx, Turborepo)
```

```
Merge queue to serialize merges and avoid semantic conflicts at HEAD
```

certs.eknathalabs.com

What breaks first: clone/checkout time (fix with partial clone + sparse-checkout), CI cost from running everything on every change (fix with affected-target detection via Bazel/Nx), and merge races (fix with a merge queue). CODEOWNERS enforces review boundaries so 200 engineers do not all review everything.

Real-world: Google, Meta, and Microsoft run massive monorepos but on custom VFS layers (VFS for Git, EdenFS). For most orgs, partial clone + sparse-checkout + a build system with affected-target detection + a merge queue is the pragmatic stack before you need custom infrastructure.

📚 **Official docs**

- Git documentation

Q67 What is a merge queue (merge train) and what problem does it solve?

Workflows

🎙 **Spoken answer · ~87 s**

A merge queue, or merge train, is a feature that serialises merges to a protected branch so that each pull request is tested against the exact state that main will be in when it lands. Without a queue, two pull requests can each pass CI against the same main, but when both merge, the combination breaks, a semantic conflict, leaving main red. Requiring branches to be up to date before merging avoids this but forces everyone to repeatedly rebase and re-run CI when others merge first, which is slow and frustrating at scale. How a queue works: when a PR is approved and passes checks, the author adds it to the queue instead of merging. The system creates a temporary merge candidate branch combining main with that PR, and with the PRs ahead of it in the queue, runs CI on that combination, and if it passes, merges it; if it fails, it's removed and subsequent entries are re-tested without it. Many queues test in parallel speculatively to keep throughput up. Benefits: main stays green, no manual rebasing, fewer wasted CI runs, and developers can fire and forget. Costs: extra CI capacity and complexity, and a failing or flaky test blocks the queue, so keep tests reliable and fast.

Likely follow-ups

- What is a semantic conflict?
- How do queues keep throughput high?

A merge queue serializes merges into a protected branch, testing each PR against the actual post-merge state rather than its stale base. It eliminates "green PR breaks main" caused by semantic conflicts.

THE PROBLEM IT SOLVES

```
PR-A and PR-B both pass CI against main independently
```

```
Both merge — but combined they break main (semantic conflict, no textual conflict)
```

```
Example: A renames a function, B adds a new caller of the old name
```

HOW THE QUEUE WORKS

certs.eknathalabs.com

```
Queue builds a speculative branch: main + A + B in order
```

```
CI runs against that combined state; only then are merges finalized
```

```
If the combined build fails, the offending PR is ejected, not merged
```

Without a queue, each PR is tested in isolation against a base that may be stale by merge time. The queue tests the real integrated result, so main stays green even at high merge volume. GitHub, GitLab, and tools like Mergify/Bors provide this. Speculative/parallel execution keeps throughput high by optimistically testing several queue positions at once.

When to introduce it: when merge volume is high enough that "passed CI but broke main" happens regularly. It is a hallmark of a mature platform org and a strong thing to mention for a Platform/SRE role.

📚 **Official docs**

- GitHub: Rulesets

Q68 How do you handle a long-running feature branch that has drifted far from main? Collaboration

🎙 **Spoken answer · ~86 s**

A drifted long-running branch is costly to merge, so I'd manage it deliberately. First assess: how far has main moved, how many files conflict, and how much of the branch's work is still valid? Then choose a strategy. Sync incrementally: merge main into the branch, or rebase onto it, in steps rather than all at once; for example, merge main as of a week ago, then a few days later, resolving smaller conflicts each time and running tests at each step. Merging main into the branch is usually safer than rebasing a large, shared branch because it preserves history and doesn't require force pushing; rebasing is cleaner if it's mine alone and the commits are few. Enable rerere so repeated conflict resolutions are remembered. Use diff3 conflict style for context, talk to authors of conflicting changes, and run the full test suite after each sync. Consider reducing the size of the problem: break the branch into smaller pieces that can land independently, cherry-picking or splitting out refactorings, and use feature flags to merge incomplete work into main early. If it has diverged so much that merging is riskier than rewriting, port the changes onto a fresh branch from current main by re-applying the logic piece by piece.

Likely follow-ups

- Why merge main into a shared long-running branch rather than rebase?
- How does rerere help repeated syncs?

Drift causes painful "big bang" merges and integration risk. The fix is to integrate continuously and, ideally, avoid long-lived branches altogether.

KEEP IT CURRENT

certs.eknathalabs.com

```
git fetch origin
```

```
git rebase origin/main              # replay your work on latest main (private branch)
```

```
# or merge main in regularly if the branch is shared:
```

```
git merge origin/main
```

```
git rerere                          # reuse recorded conflict resolutions across rebases
```

BETTER: AVOID THE SITUATION

```
Break the feature into small, independently-mergeable PRs behind a flag
```

```
Merge incomplete-but-safe slices to main continuously (feature-flagged off)
```

Tactically, rebase or merge main into the branch **frequently** (daily) so conflicts stay small, and enable `git rerere` so you do not re-resolve the same conflicts on each rebase. Strategically, the real answer is to stop creating long-lived branches: decompose the work into small PRs that land behind a feature flag, integrating continuously.

The deeper signal: a branch that drifted "far from main" is usually a process smell. Interviewers want to hear you address the root cause (batch size) not just the symptom (the painful merge).

📚 **Official docs**

- Git documentation

certs.eknathalabs.com

Q69 Describe a safe hotfix workflow for a critical production bug.

Workflows

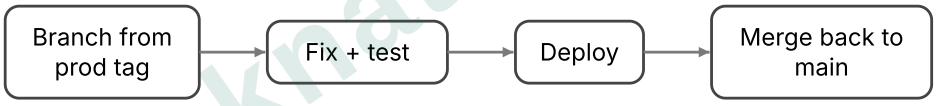
🎙 Spoken answer · ~83 s

A safe hotfix workflow balances speed with control. Triage and decide: confirm severity, and consider whether a quick mitigation like a rollback or feature flag restores service faster than a code fix. If a fix is needed, branch from what's running in production, the release tag or production branch, not necessarily from main. Reproduce the issue with a failing test first, if possible, then implement the smallest fix, avoiding refactoring or unrelated changes. Open a pull request against the production branch, with a clear description of the incident and the fix; require at least one review, even if expedited, ideally by someone with context, and keep CI mandatory. Build the artefact once, tag it with a patch version, deploy to staging for a smoke test if time allows, then to production using the standard pipeline, ideally with a canary or staged rollout and a rollback plan ready. Monitor error rates, latency, and business metrics after deployment, and communicate status to stakeholders in the incident channel. Then merge or cherry-pick the fix back into main and any active release branches, so it isn't lost in the next release; verify with a check that the fix commit is reachable from main.

Likely follow-ups

- Why backport the fix to main?
- When is a rollback better than a hotfix?

🗺 Hotfix workflow



A hotfix must ship fast without bypassing safety, and must not be lost from mainline. The pattern: branch from the production tag, fix, deploy, then back-merge.

```
git checkout -b hotfix/CVE-fix v2.3.1   # branch from the EXACT deployed tag, not main
```

```
# make the minimal fix, commit
```

```
git tag v2.3.2 && git push origin v2.3.2  # tag the patched release
```

```
# deploy v2.3.2 to production
```

```
# Back-merge so main/develop also gets the fix:
```

```
git checkout main && git merge hotfix/CVE-fix
```

```
git push origin main
```

certs.eknathalabs.com

Branching from the deployed tag (not from current main) is the critical detail — main may contain unreleased changes you do not want in an emergency hotfix. After shipping, you must back-merge into main/develop or the fix silently disappears in the next release (a classic "we fixed it twice" bug).

Keep safety on: even under pressure, the hotfix should go through CI, a fast review, and ideally a canary. Speed comes from a small diff and a pre-agreed expedited process — not from skipping checks.

📚 **Official docs**

- Git documentation

Q70 **What goes in a good PR, and how do you keep reviews fast and effective at scale?** Collaboration

🎙 **Spoken answer · ~84 s**

A good PR is small, focused, and easy to review, and it carries the context a reviewer needs. Content: one logical change, ideally under a few hundred lines; a descriptive title; a description that explains why the change is needed, what approach was taken and alternatives considered, how it was tested, linked ticket, screenshots or logs for visible changes, and any deployment or rollback considerations. Passing CI before requesting review, with tests for new behaviour and no unrelated formatting noise. Self-review first, leaving comments on tricky parts. Keeping reviews fast and effective at scale. Set expectations: a review turnaround target, such as within one business day, and track time to first review. Route automatically with CODEOWNERS and review assignment rotation to spread the load. Automate the mechanical parts: linters, formatters, type checks, and security scans so humans focus on design, correctness, maintainability, and risk. Keep PRs small through stacked PRs, feature flags, and incremental delivery. Use draft PRs for early feedback on direction. Reviewers should be constructive, distinguish blocking issues from nits and suggestions, approve with comments when appropriate, and avoid blocking on style. Use PR templates and checklists. Encourage synchronous discussion for complex disagreements. Use merge queues to reduce rebasing overhead.

Likely follow-ups

- How do you keep PRs small?
- What should humans review versus linters?

PR throughput is a platform concern: slow reviews are often a team's biggest hidden bottleneck. The levers are PR size, context, and tooling.

ANATOMY OF A REVIEWABLE PR

```
Small: <400 lines changed where possible (review quality drops sharply after)
```

```
Single purpose: one logical change, not five unrelated fixes
```

certs.eknathalabs.com

```
Clear description: what, why, how to test; link the issue
```

```
Self-reviewed: author leaves inline notes on non-obvious choices
```

PLATFORM LEVERS

```
CODEOWNERS for auto-routing to the right reviewers
```

```
Required checks (tests, lint, security) green before human review
```

```
Draft PRs for early feedback; merge queue for serialized integration
```

```
Track PR cycle time (open→merge) and review latency as team health metrics
```

The single highest-leverage factor is **small PRs** — review quality and speed both collapse on large diffs. Automate everything a machine can check (format, lint, tests, coverage, security) so humans review design and correctness, not style. Measure cycle time so review friction is visible and improvable.

> **Platform framing:** treat "PR cycle time" as a DORA-adjacent metric you own and optimize. That reframes code review from a chore into a system you engineer — exactly the staff/platform perspective.

> 📚 **Start here (official docs)**
>
> - Git documentation

certs.eknathalabs.com

Q71 How do you manage release versioning and tagging in an automated pipeline?

Workflows

🎙 Spoken answer · ~68 s

Release versioning and tagging in an automated pipeline should be deterministic, traceable, and need little manual work. Adopt semantic versioning, MAJOR.MINOR.PATCH, so consumers can reason about compatibility. Derive version numbers from commit history using Conventional Commits: feat triggers a minor bump, fix a patch bump, and a breaking change marker a major bump. Tools such as semantic-release, release-please, GitVersion, or changesets for monorepos calculate the next version, update version files and the changelog, create an annotated tag, and publish a release with notes. Release-please opens a release pull request that, when merged, cuts the release, adding a human approval gate. The pipeline flow: merges to main run CI; the release job runs only on main after tests pass, determines the next version, tags the exact commit, builds the artefact once from that tag, tags the container image with the version and commit SHA, signs it, publishes to the registry, and creates the release notes. Deployments reference immutable versions or digests, never latest.

Likely follow-ups

- How does release-please add a human gate?
- Why embed the commit SHA in the build?

Automated versioning removes human error and ties versions to commit semantics. The dominant pattern is Semantic Versioning driven by Conventional Commits.

```
git tag -a v2.4.0 -m 'release: ...'  # annotated tags carry metadata; use for releases
```

```
git push origin v2.4.0
```

```
git describe --tags                  # human-readable version from nearest tag
```

```
# semantic-release / release-please derive the version automatically:
```

```
feat: ...    → minor bump
```

```
fix: ...     → patch bump
```

```
feat!: / BREAKING CHANGE: → major bump
```

Conventional Commits let tooling (semantic-release, release-please, changesets) compute the next version, generate a changelog, create the tag, and publish — all from commit messages. Always use **annotated** tags for releases (they store author/date/message and are objects), not lightweight tags. `git describe` gives a deterministic version string for builds.

Why this matters for platform: deterministic, automated versioning is a prerequisite for reliable rollbacks and provenance. The version on a running pod should map unambiguously to a tag and a

certs.eknathalabs.com

commit SHA.

📚 **Official docs**

- Git documentation

Q72 Explain Git submodules vs subtrees vs a package manager for sharing code.

Collaboration

🎙 **Spoken answer · ~80 s**

These three approaches share code between projects differently. Submodules embed another repository at a pinned commit, retaining its separate history and identity. Pros: exact source-level pinning, the subproject remains an independent repository you can push to, and you can see precisely what version is used. Cons: clumsy workflow, detached HEAD confusion, forgetting to initialise or push submodule commits, CI complexity, and hard for newcomers. Subtrees, git subtree, copy the other project's content into a subdirectory of your repository, with history merged in; you can pull updates and push changes back. Pros: no special clone steps, everything is in one repository, simple for consumers. Cons: bigger repository, history can get messy, and syncing upstream is manual, with commands that are less familiar. A package manager approach publishes the shared code as a versioned package to a registry such as npm, PyPI, Maven, or an internal artefact repository, and consumers declare a dependency version. Pros: clear versioning and release lifecycle, semantic versioning, caching, dependency resolution, and standard tooling and security scanning; consumers don't need source access. Cons: the overhead of publishing and releasing, and slower iteration when changing the library and consumer together.

Likely follow-ups

- Why do packages usually beat submodules for shared libraries?
- When is a subtree a good fit?

All three share code across repos but differ in coupling, history, and operational pain. The right choice depends on how tightly versioned and how often updated the shared code is.

SUBMODULES

```
git submodule add <url> libs/shared
```

```
# Parent repo pins an exact SHA of the child; explicit, but easy to forget to update
```

```
git submodule update --init --recursive   # required after clone (common footgun)
```

SUBTREES

```
git subtree add --prefix=libs/shared <url> main --squash
```

```
# Child code is merged INTO parent history; no special clone steps, but bigger repo
```

certs.eknathalabs.com

Submodules keep repos separate and pin exact versions, but add clone/checkout friction and a steep learning curve (detached HEADs, forgotten updates). **Subtrees** vendor the code into the parent so clones "just work," at the cost of a larger repo and awkward upstream contribution. For most teams, neither beats a **real package manager** (npm, PyPI, Go modules, an internal registry) with semantic versioning — it gives clean version constraints, caching, and dependency resolution.

Default recommendation: reach for a package manager + internal artifact registry first. Use submodules only when you genuinely need source-level coupling (e.g. vendoring a fork you patch). Subtrees are niche.

📚 **Official docs**

- Pro Git: Submodules
- PostgreSQL: Indexes
- Git documentation

certs.eknathalabs.com

Q73 How do you enforce commit message conventions and pre-commit quality gates across a team?

Workflows

🎙 **Spoken answer · ~85 s**

Enforcing commit conventions and quality gates requires layers, because local checks alone can be skipped. Local, for fast feedback: use a hook manager such as pre-commit, Husky, or Lefthook, with configuration committed to the repository so every developer gets the same hooks after one install step. Pre-commit hooks run formatters, linters, secret scanners, and quick tests on staged files; commit-msg hooks run commitlint to validate Conventional Commits; pre-push hooks run heavier checks. Hooks give quick feedback but can be bypassed with --no-verify or not installed, so they're a convenience, not enforcement. Authoritative checks go in CI: run the same linters, formatters in check mode, commitlint over all commits in the pull request, tests, and security scans as required status checks. Platform controls: branch protection requiring those checks, pull request reviews, and linear history or squash merge; when squashing, validate the PR title as the commit message with a check action, since that becomes the commit on main. Server-side hooks on self-hosted Git can reject pushes with invalid messages. Supporting measures: pull request and commit templates, documentation of conventions, editor integration, and setup scripts. Make hooks fast, as slow ones encourage bypassing; use incremental checks on changed files. Give clear error messages with fix instructions.

Likely follow-ups

- Why can't pre-commit hooks be the only enforcement?
- How do you validate commits when squash merging?

Consistency comes from automation, not documentation. Local hooks give fast feedback; server-side checks are the real enforcement.

LOCAL (FAST FEEDBACK)

```
# .pre-commit-config.yaml (pre-commit framework):
```

```
  - lint, format (ruff/black/prettier), trailing-whitespace
```

```
  - detect-secrets / gitleaks
```

```
  - commitlint via commit-msg hook (enforce Conventional Commits)
```

```
pre-commit install                  # wires up the git hooks locally
```

SERVER-SIDE (REAL ENFORCEMENT)

```
Branch protection: required status checks must pass to merge
```

```
CI re-runs lint/format/tests/secret-scan (local hooks can be bypassed)
```

```
Required signed commits; required linear history if desired
```

Local hooks (via the `pre-commit` framework) catch issues before they leave the laptop and keep feedback fast. But local hooks can be skipped (`--no-verify`), so the authoritative gate is **server-side**: branch protection with required CI checks that re-run the same validations. Belt and suspenders — local for speed, server for guarantee.

> **Platform mindset:** ship the hook config in-repo so it is versioned and consistent, and make CI the source of truth. Never rely on humans remembering to run checks.

> 📚 **Official docs**
>
> - Conventional Commits
> - githooks

certs.eknathalabs.com

Q74 A repo is too big and slow to clone/work with. What are your options?

Collaboration

🎙 Spoken answer · ~81 s

First diagnose why it's slow: is it history size, huge files, a large number of files, or heavy binaries? Use git count-objects -vH, git rev-list --objects --all with size analysis, or tools like git-sizer to find the culprits. Options. For clone and fetch time: shallow clones for automation, partial clones with --filter=blob:none to fetch content on demand, and single-branch clones. For working tree size: sparse checkout in cone mode to materialise only needed directories. For daily command speed: enable the filesystem monitor and untracked cache, core.fsmonitor, run git maintenance start for background commit-graph, prefetch, and repacking tasks, and consider Scalar, which configures these. For large binary files: move them out with Git LFS or into an artefact store. For bloated history: rewrite to remove large files or old junk using git filter-repo, which requires a coordinated migration since all hashes change, or archive old history and start a fresh repository with a grafted link to the old one. Server side: run garbage collection and repack, use bitmaps, and provide regional mirrors or caching proxies near CI runners. Structural options: split a monolith into multiple repositories if domains are independent, or adopt tooling suited to monorepos.

Likely follow-ups

- How do you find what is making the repository large?
- What are the trade-offs of rewriting history to shrink it?

Diagnose whether the problem is history size, working-tree size, or large binaries — each has a different fix.

REDUCE WHAT YOU FETCH

```
git clone --depth 1 <url>           # shallow clone: only latest commit (CI use)
```

```
git clone --filter=blob:none <url>   # partial clone: history without blobs
```

```
git sparse-checkout set <paths>      # check out only part of the tree
```

FIX LARGE BINARIES

```
git lfs track '*.psd'                # Git LFS: store big files outside main history
```

```
git filter-repo --strip-blobs-bigger-than 10M   # purge huge blobs from history
```

If clone is slow because of **deep history**, use shallow (`--depth`) or partial (`--filter=blob:none`) clones. If the **working tree** is huge, sparse-checkout limits it. If the cause is **large binaries bloating history**, that is the real disease: migrate them to Git LFS and purge the originals with filter-repo, then everyone re-clones. Background `git maintenance` keeps things fast over time.

Prevention: binaries in Git history are permanent dead weight — every clone pays for them forever. Add a pre-commit size check and an LFS policy so large files never enter normal history.

certs.eknathalabs.com

📚 **Official docs**

- Git LFS
- git-filter-repo

Q75 How would you design a branching and deploy mapping for multiple environments (dev/staging/prod)?

Workflows

🎙 **Spoken answer · ~82 s**

Mapping branches to environments should avoid drift and keep promotion simple. A recommended approach for continuous delivery: one long-lived branch, main, representing the integrated state, and environments differing by configuration, not by branch. Every merge to main builds an immutable artefact tagged with the commit SHA; the pipeline deploys it automatically to dev, then promotes the same artefact to staging after tests, and to production after approval gates or automated checks, using environment-specific configuration injected at deploy time from separate files, Helm values, or Kustomize overlays. For GitOps, keep environment folders in a configuration repository, and promotion is a pull request or automated commit updating the version in the next environment's directory. Avoid branch-per-environment, such as dev, staging, and prod branches, because promotion means merging between them, which causes drift, cherry-pick confusion, and lets untested combinations reach production. Where a branch-based approach is mandated, keep merges strictly one-way and automated. Release-oriented models: for versioned software, use release branches such as release/2.3, deploy tags from them to production, and deploy main to dev and staging. Preview environments for pull requests, created and destroyed automatically, give early feedback. Protect production: required approvals, deployment windows, and environment protection rules.

Likely follow-ups

- Why avoid a branch per environment?
- How does promotion work in a GitOps layout?

Map Git refs to environments deterministically so every deploy is traceable to a commit. The cleanest model pairs trunk-based development with promotion by tag/SHA, not by long-lived environment branches.

RECOMMENDED: PROMOTE ARTIFACTS, NOT BRANCHES

```
main merge          → build immutable artifact, deploy to staging automatically
```

```
git tag v2.4.0      → promote the SAME artifact to production
```

```
rollback            → redeploy previous tag's artifact (no rebuild)
```

ANTI-PATTERN TO AVOID

```
Long-lived dev/staging/prod branches that drift and require cherry-picks
```

```
Rebuilding per environment (the prod artifact differs from what you tested)
```

certs.eknathalabs.com

The key principle: build the artifact **once** and promote that exact artifact through environments — staging tests the literal bytes that will run in prod. Tying promotion to tags/SHAs gives precise traceability and trivial rollback (redeploy the prior tag). Long-lived environment branches are an anti-pattern: they drift, demand error-prone cherry-picks, and let staging diverge from prod.

GitOps angle: with Argo CD/Flux, a separate config repo holds the desired state per environment, and promotion is a commit that bumps the image tag. Git becomes the audit log of every environment change — strong for a platform role.

📚 **Start here (official docs)**

- Git documentation

Q76 What is git worktree and how does it help in production/dev workflows? Collaboration

🎙 **Spoken answer · ~80 s**

A Git worktree is an additional working directory attached to the same repository, each with its own checked-out branch, index, and working files, but sharing the same object database and history. You create one with git worktree add ../review-pr pr-branch. How it helps in production and development workflows. Hotfixes without disruption: while in the middle of a feature with uncommitted changes, you can create a worktree from the production tag, fix the urgent bug, test, and push, with no stashing or context loss. Code review: check out a colleague's branch in a separate directory, run and test it while keeping your own work open. Long-running tasks: run a slow test suite or build on one branch while continuing to work on another. Comparing versions side by side, for debugging regressions, or running two versions of an application simultaneously. Parallel release maintenance: keep worktrees for each supported release branch to apply backports. Automation and tooling: CI agents or AI coding agents can each work in isolated worktrees without conflicting. Compared with multiple clones, worktrees save disk space and network, since they share objects, and commits and branches are instantly visible across them.

Likely follow-ups

- Why can't the same branch be in two worktrees?
- How do worktrees help with parallel hotfix work?

git worktree lets one repository have multiple working directories checked out simultaneously, sharing a single object store — no second clone needed.

```
git worktree add ../hotfix main      # new working dir on main, alongside your feature work
```

```
git worktree add ../review pr-123    # check out a PR to review without stashing
```

```
git worktree list                    # see all worktrees
```

certs.eknathalabs.com

```
git worktree remove ../hotfix       # clean up when done
```

The classic use: you are deep in a feature branch when a P1 hotfix lands. Instead of stashing, switching, fixing, switching back, and unstashing, you spin up a worktree on a separate directory, fix there, and your feature work stays untouched. Because worktrees share the object database, they are far cheaper than a second clone (no re-fetch, shared history).

Other uses: running a long build on one branch while coding on another, reviewing a PR without disrupting your context, or building two versions side by side. A clean, underused feature worth mentioning.

📚 **Official docs**

- git-worktree
- Git documentation

Q77 How do you implement and enforce a code review / approval policy with branch protection?

Workflows

🎙 **Spoken answer · ~87 s**

Enforcing review and approval policy relies on platform branch protection or rulesets, backed by culture and tooling. Configure protection on main and release branches: require pull requests, with no direct pushes, including for administrators where possible; require a minimum number of approving reviews, typically one or two, higher for sensitive repositories; dismiss stale approvals when new commits are pushed; require approval from someone other than the last pusher, so authors can't approve their own changes; require review from code owners for protected paths defined in CODEOWNERS, such as infrastructure, authentication, and CI workflow files; require all conversations to be resolved; require status checks, including build, test, linters, and security scans, and require branches to be up to date or use a merge queue; require signed commits if provenance matters; restrict who can push, merge, or bypass; block force pushes and deletions; and require linear history if preferred. Use organisation-level rulesets so all repositories receive the baseline, and apply them with automation or Terraform to avoid drift. Deployment protection: environment rules requiring approval from named reviewers before production deployments. Audit: monitor changes to protection rules and bypass events in audit logs. Cultural side: define review guidelines, response time expectations, and training. Provide a documented emergency bypass with post-hoc review.

Likely follow-ups

- Why block authors from approving their own pull requests?
- How do you apply the baseline to every repository?

Branch protection turns review policy into an enforced gate rather than a convention. Configure it on the default and release branches.

certs.eknathalabs.com

TYPICAL PROTECTION RULES

```
Require N approving reviews before merge (often 1–2)
```

```
Require review from CODEOWNERS for touched paths
```

```
Dismiss stale approvals when new commits are pushed
```

```
Require status checks (tests, lint, security) to pass
```

```
Require branches up to date / require linear history
```

```
Require signed commits; restrict who can push; block force-push & deletion
```

CODEOWNERS plus "require review from code owners" routes each change to the people who actually own that code. "Dismiss stale approvals" prevents the trick of getting approved, then pushing different code. Requiring up-to-date branches (or a merge queue) prevents semantic conflicts. For regulated environments, require signed commits and restrict who can administer the rules.

Watch the admin bypass: decide deliberately whether admins can bypass protection. For high-compliance repos, "include administrators" should be on — otherwise the controls are theater. Audit who has the power to change the rules.

📚 **Official docs**

- GitHub: Protected branches

certs.eknathalabs.com

Q78 Explain GPG/SSH commit signing: why, how, and how to enforce it.

Collaboration

🎙 Spoken answer · ~80 s

Commit signing attaches a cryptographic signature to commits and tags, proving they were created by the holder of a particular key and haven't been altered. Without it, author names and emails are unauthenticated text, so anyone can forge commits claiming to be someone else. Why: integrity and provenance for the software supply chain, guarding against impersonation after a stolen token, compliance and audit requirements, and trust in release tags. How: Git supports GPG, SSH keys, and X.509 certificates, and Sigstore's gitsign offers keyless signing. For GPG: generate a key, tell Git with user.signingkey, enable commit.gpgsign true and tag.gpgsign true, and upload the public key to the hosting platform. For SSH signing, simpler since developers already have keys: set gpg.format ssh, point user.signingkey to the public key, and add it to the platform as a signing key; allowed signers files enable local verification. Hardware security keys protect private keys. Verification: git log --show-signature, git verify-commit, and the platform's Verified badge. Enforcement: branch protection rules requiring signed commits on protected branches, so unsigned pushes are rejected; CI or admission checks verifying signatures on release tags; web-based merges are signed by the platform's own key.

Likely follow-ups

- What is the difference between GPG and SSH signing?
- What does a Verified badge not guarantee?

Signing cryptographically proves who authored a commit, defeating identity spoofing (anyone can set `user.email` to yours). It is increasingly required for supply-chain security.

```
git config user.signingkey <key-id>
```

```
git config commit.gpgsign true       # sign all commits automatically
```

```
git commit -S -m 'msg'               # sign a single commit explicitly
```

```
git tag -s v2.4.0                     # sign a tag
```

```
# SSH signing (simpler than GPG, since Git 2.34):
```

```
git config gpg.format ssh
```

```
git config user.signingkey ~/.ssh/id_ed25519.pub
```

```
git log --show-signature             # verify signatures
```

Without signing, commit author fields are pure metadata — trivially forged. Signing binds commits to a verified key. Enforcement happens server-side: GitHub/GitLab can require signed commits via branch protection and show a "Verified" badge. SSH signing (Git 2.34+) is now the easy path — reuse your existing

certs.eknathalabs.com

SSH key, no GPG keyring management.

> **Supply-chain context:** signed commits + signed tags + provenance (SLSA) are how you prove what shipped came from who you think. For a platform/security-adjacent role, framing signing as part of supply-chain integrity (not just "nice to have") lands well.

> 📚 **Official docs**
>
> - Git: Signing your work

Q79 What is GitOps and how does Git become the source of truth for infrastructure?

CI/CD & GitOps

> 🎙 **Spoken answer · ~77 s**
>
> GitOps is an operating model where the desired state of infrastructure and applications is declared in Git, and automated agents continuously reconcile the real environment to match it. The four principles: the system is described declaratively; the desired state is versioned and immutable in Git; approved changes are applied automatically; and software agents ensure correctness and alert on drift. Git becomes the source of truth. All changes, whether an image version bump, a scaling change, or a firewall rule, go through pull requests, with review, CI validation, and policy checks, and the merged commit is the record. An agent such as Argo CD or Flux running in the cluster pulls from the repository and applies changes, and continuously compares live state with Git, reverting or flagging manual changes. For infrastructure, tools like Terraform driven by Atlantis or pipelines, or Crossplane, apply the same idea. Benefits: full audit trail of who changed what and why, easy rollback by reverting a commit, consistent environments, disaster recovery by re-applying the repository to a new cluster, less need for cluster credentials outside the cluster, and a familiar developer workflow.
>
> **Likely follow-ups**
>
> - How does the agent detect and correct drift?
> - What happens to secrets in a GitOps repository?

certs.eknathalabs.com

GitOps declares desired system state in Git; an automated controller continuously reconciles the live environment to match. Git becomes the single source of truth and the audit log for infrastructure.

CORE LOOP

```
Desired state (manifests/Helm/Kustomize) committed to a config repo
```

```
Controller (Argo CD / Flux) watches the repo
```

```
Controller diffs desired vs live cluster state and reconciles automatically
```

```
Drift (manual kubectl change) is detected and reverted to match Git
```

WHY IT IS POWERFUL

```
Every change is a reviewed, signed, auditable commit (who/what/when/why)
```

```
Rollback = git revert + auto-reconcile (no imperative kubectl needed)
```

```
No human has prod credentials; the controller pulls and applies
```

The shift is from **push** (CI runs kubectl/terraform against prod) to **pull** (an in-cluster controller pulls the declared state and applies it). Benefits: a complete audit trail in Git, trivial rollback via revert, automatic drift correction, and no CI system holding prod credentials. The config repo is typically separate from app code so deploys are decoupled from builds.

Platform relevance: GitOps is core to modern platform engineering. Mentioning the push→pull inversion, drift detection, and "rollback is just git revert" demonstrates current, production-grade knowledge.

📚 **Official docs**

- OpenGitOps principles
- Git documentation

Q80 Why should CI not use a long-lived Personal Access Token, and what replaces it?

CI/CD & GitOps

🎙 **Spoken answer · ~82 s**

A personal access token is tied to a human's identity, and typically carries broad permissions across everything that user can access. Using one in CI creates several problems. Security: it's long-lived, so if leaked from a log, a misconfigured job, or a compromised third-party action, an attacker gets the user's access until someone notices and revokes it. It often has more scope than the pipeline needs, violating least privilege. Lifecycle fragility: when the employee leaves or their account is deactivated, pipelines suddenly break, or worse, tokens of departed employees remain active. Accountability: actions in the audit logs appear as that person, blurring who or what actually did it. Shared use of one personal token across many jobs prevents attribution and makes rotation painful. What replaces it. The CI platform's automatically issued, short-lived job token, such as GitHub's GITHUB_TOKEN with explicit minimal permissions, or GitLab's CI job token, scoped to the repository and expiring when the job ends. For cross-repository access, use GitHub Apps or fine-grained tokens/service accounts with narrowly scoped, repository-specific permissions, installed at the organisation level, generating short-lived installation tokens. Read-only deploy keys for single-repository checkouts. For cloud access, OIDC federation instead of stored keys.

Likely follow-ups

- What does the job token's scope look like?
- Why does a GitHub App beat a personal token for automation?

ceris.eknathalabs.com

Long-lived PATs are static, broad, and rarely rotated — a single leak (in logs, env dumps, a compromised runner) is a durable breach. The modern replacement is short-lived, identity-based credentials.

BETTER OPTIONS

```
OIDC federation: CI exchanges a signed identity token for short-lived cloud creds
```

```
  (GitHub Actions ↔ AWS/GCP/Azure Workload Identity — no stored secrets)
```

```
GitHub App installation tokens: scoped, auto-expiring (~1h), per-repo
```

```
Deploy keys: single-repo, read-only by default, instead of account-wide PATs
```

```
Fine-grained PATs with explicit repo + permission scope and short expiry (last resort)
```

OIDC federation is the gold standard: the CI job presents a cryptographically signed identity token, the cloud provider verifies it against a trust policy and issues credentials valid for minutes — nothing long-lived is ever stored. For Git operations, GitHub App tokens or deploy keys replace account-wide PATs with scoped, expiring access.

Leak blast radius: a leaked 1-hour OIDC credential expires before most attackers can use it; a leaked PAT with no expiry is exploitable indefinitely. Always prefer ephemeral, scoped identity over static secrets.

📚 **Start here (official docs)**

- Git documentation

certs.eknathalabs.com

Q81 How do you prevent secrets from ever entering Git history?

Security

🎙 Spoken answer · ~88 s

Once a secret is in Git history, you must assume it's compromised, so the aim is to stop it getting there, using overlapping barriers. Design so there's less to leak: use secret managers, workload identity, and short-lived credentials, and provide a template with configuration read from the environment at runtime. Developer side: .gitignore entries for env files, key files, and local config, a committed .env.example with placeholders, and pre-commit hooks running gitleaks, detect-secrets, or TruffleHog on staged changes, distributed through a hook framework with a committed configuration. Remote side, which can't be bypassed locally: platform push protection, such as GitHub's, rejects pushes containing recognised secret formats; server-side pre-receive hooks on self-hosted servers do the same. Pull request side: required CI checks scanning the diff and commit messages, failing the build on findings. Continuous: organisation-wide secret scanning of existing repositories, including full history, with alerts routed to owners and validity checks for detected credentials. Cultural: training, easy ways to get secrets properly, and a blameless reporting path so people disclose mistakes quickly. Tuning: maintain custom patterns for internal token formats, manage allowlists with expiry, and track false-positive rates, as noisy tools get disabled. Have a rehearsed response: revoke, rotate, investigate, then clean history. The combination makes leaks rare and quickly contained.

Likely follow-ups

- Why isn't a pre-commit hook enough by itself?
- What do you do when push protection blocks a legitimate test value?

Defense in depth: stop secrets at the developer's machine, catch them in CI, and scan continuously — because a single push to a remote is effectively permanent exposure.

LAYERS

```
Pre-commit: gitleaks / detect-secrets hook blocks the commit locally
```

```
Server-side: push protection (GitHub) rejects pushes containing secret patterns
```

```
CI: re-scan (local hooks can be bypassed with --no-verify)
```

```
Continuous: scheduled secret scanning across all repos + alerting
```

```
.gitignore for known secret files (.env, *.pem, credentials.json)
```

AND DO NOT STORE SECRETS IN GIT AT ALL

```
Use a secrets manager: Vault, AWS/GCP Secrets Manager, sealed-secrets for k8s
```

```
Inject at runtime via env/volume, never commit plaintext config
```

certs.eknathalabs.com

The most reliable prevention is architectural: secrets should live in a secrets manager and be injected at runtime, so there is nothing to leak into Git in the first place. Layer on pre-commit scanning (fast, local), server-side push protection (cannot be bypassed), and CI re-scanning (catches `--no-verify`). Treat any committed-and-pushed secret as compromised — rotate it.

Interview framing: lead with "the best fix is not to put secrets in Git at all," then layer scanning as defense in depth. That ordering shows you address root cause before symptom.

📚 **Official docs**

- Git documentation

Q82 **Explain how you would build a deployment pipeline triggered by Git events.** CI/CD & GitOps

🎙 **Spoken answer · ~85 s**

A Git-triggered deployment pipeline starts with events from the repository. A webhook or native integration sends push, pull request, and tag events to the CI system. On pull request: run validation: lint, unit tests, build, security scans, and IaC plan, report status checks to the pull request, and optionally create a preview environment. On merge to main: build the artefact once, tag it with the commit SHA, run integration tests, scan, sign, and publish to the registry; then deploy automatically to a development or staging environment and run smoke or end-to-end tests. Promotion to production is triggered by a tag, a manual approval in an environment protection rule, or an automated gate based on checks and metrics. Deployment can be push-based, where the pipeline runs helm or kubectl using short-lived credentials from OIDC, or pull-based GitOps, where the pipeline updates a configuration repository and Argo CD or Flux applies it. Use progressive delivery, such as canary or blue-green, with health checks and automatic rollback on alert. Controls: protected branches and required reviews, pinned and trusted actions, least-privilege secrets per environment, concurrency control so only one deployment runs per environment at a time, path filters for monorepos, and audit logs linking deployments to commits.

Likely follow-ups

- What triggers production promotion?
- How does GitOps change the deployment step?

Map Git events to pipeline stages so the workflow is automatic and traceable. The trigger model defines the deploy semantics.

EVENT → ACTION MAPPING

```
PR opened/updated → build + test + security scan + deploy ephemeral preview env
```

```
Merge to main     → build immutable artifact + deploy to staging
```

```
Tag vX.Y.Z        → promote the SAME artifact to production (canary → full)
```

certs.eknathlabs.com

```
Manual approval    → gate between staging and prod for high-risk changes
```

KEY PROPERTIES

```
Build once, promote the same artifact (don't rebuild per environment)
```

```
Every artifact tagged with its commit SHA for traceability
```

```
Automated rollback: redeploy previous tag on health-check failure
```

Webhooks (or the CI provider's native triggers) fire on push/PR/tag events. The disciplined design builds the artifact once on merge and **promotes** that exact artifact — PR previews validate features in isolation, staging tests the literal prod bytes, and a tag promotes to prod with a canary. Tagging artifacts by SHA makes "what is running in prod?" answerable and rollback deterministic.

Safety in the pipeline: required green checks before merge, ephemeral preview environments per PR, canary/progressive delivery to prod, and automated rollback on SLO breach. Git events drive it; observability gates it.

📚 **Official docs**

- AWS Builders' Library: Automating safe, hands-off deployments
- Martin Fowler: Continuous Delivery
- Git documentation

certs.eknathalabs.com

Q83 What is a signed/verified release and how does it fit into supply-chain security (SLSA)?

Security

🎙 Spoken answer · ~80 s

A signed or verified release is an artefact, such as a tag, binary, package, or container image, that carries a cryptographic signature proving who produced it and that it hasn't been modified since. Consumers verify the signature before trusting it. In supply-chain security, this addresses tampering and impersonation between build and use. Tools: GPG-signed tags, Sigstore cosign for images and binaries with keyless signing tied to the CI identity, and registries or package ecosystems that support signatures. SLSA, Supply-chain Levels for Software Artifacts, goes further by defining the assurance level of the build process itself. A key element is provenance: a signed attestation recording what source repository and commit, which builder, and what build steps produced the artefact. At SLSA build level 1 you have provenance; level 2 requires a hosted build platform that signs it, so it's hard to forge; level 3 requires a hardened, isolated builder so that even compromised build steps or maintainers can't tamper with provenance. A release pipeline therefore builds on a hosted runner, generates an SBOM and provenance, signs the artefact and the attestations, records the signing event in a transparency log, and publishes them together.

Likely follow-ups

- What does provenance record?
- What does SLSA level 3 add over level 2?

A verified release proves provenance: that an artifact was built from specific source by a specific trusted process, unmodified. It closes the gap where attackers tamper between source and deploy.

THE CHAIN OF TRUST

```
Signed commits     → prove who authored the source
```

```
Signed tags        → prove which exact source is the release
```

```
Signed artifacts   → cosign sign <image>  (Sigstore)
```

```
Provenance attestation → SLSA: who built it, from what source, with what builder
```

```
Verify at deploy   → cosign verify; admission controller blocks unsigned images
```

SLSA (Supply-chain Levels for Software Artifacts) formalizes this into levels: at higher levels the build runs on a hardened, isolated builder that emits signed provenance linking the artifact back to the exact source commit. Sigstore/cosign make signing keyless (tied to OIDC identity). At deploy, an admission controller (e.g. Kyverno, Connaisseur) refuses to run images lacking a valid signature and provenance.

Why it matters: attacks like SolarWinds and the xz backdoor exploited the gap between trusted source and shipped artifact. Signing the whole chain — commit → tag → artifact → provenance, verified at

certs.eknathalabs.com

admission — is how you defend it. Strong material for a platform/security role.

📚 **Official docs**

- GitHub: Artifact attestations (build provenance)
- SLSA specification

Q84 How do Git hooks work, and where do server-side vs client-side hooks fit?

CI/CD & GitOps

🎙 **Spoken answer · ~78 s**

Git hooks are executable scripts that Git runs at specific points, found in .git/hooks, or in a directory set by core.hooksPath. Client-side hooks run on a developer's machine: pre-commit before a commit is created, for linting, formatting, and secret scanning; prepare-commit-msg and commit-msg for templating or validating messages; pre-push for heavier checks like tests; pre-rebase; and post-checkout, post-merge, and post-commit for housekeeping such as installing dependencies. A non-zero exit from the pre-hooks aborts the operation. Server-side hooks run on the receiving repository: pre-receive runs once for a push and can reject the whole push based on policy, such as blocking force pushes, enforcing commit message formats, or scanning for secrets or large files; update runs per branch being updated; post-receive runs after acceptance, to trigger notifications, CI, or deployment. Client-side hooks aren't versioned or enforced: they live outside the tracked files, require each developer to install them, and can be bypassed with --no-verify, so they're for fast, friendly feedback. Server-side hooks are authoritative because every push must pass them, though on hosted platforms such as GitHub they're replaced by branch protection, required status checks, push protection, and webhooks.

Likely follow-ups

- Which hook can reject an entire push?
- Why can't client hooks be relied upon for policy?

Hooks are scripts Git runs at lifecycle events. Client-side hooks live in each clone (advisory); server-side hooks run on the remote (enforcing).

CLIENT-SIDE (LOCAL, BYPASSABLE)

```
pre-commit    # lint/format/secret-scan before a commit is created
```

```
commit-msg    # validate message format (Conventional Commits)
```

```
pre-push      # run quick tests before pushing
```

```
# Bypassable with --no-verify — fast feedback, not enforcement
```

SERVER-SIDE (AUTHORITATIVE)

```
pre-receive   # reject a push that violates policy (protected paths, signed commits)
```

certs.eknathalabs.com

```
update       # per-branch ref checks
```

```
post-receive  # trigger CI/CD, notifications after accepted push
```

Client hooks give fast feedback but cannot be trusted as gates — developers can skip them. Real enforcement is server-side: `pre-receive` can reject pushes that break policy. On hosted platforms (GitHub/GitLab) you usually do not write raw server hooks; you use branch protection and required CI checks, which serve the same enforcement role. Distribute client hooks via the `pre-commit` framework so they are versioned and consistent.

Common mistake: relying on client-side hooks for security/compliance. They are convenience, not control. Anything that must be guaranteed belongs in server-side enforcement or CI.

📚 **Official docs**

- githooks
- Git documentation

Q85 **A dependency was compromised via a malicious commit. How do you investigate using Git?** Security

🎙 **Spoken answer · ~83 s**

If a dependency was compromised via a malicious commit, I need to determine what changed, when, by whom, and what it affected in my systems. Scope: identify the compromised repository, the affected versions or commit range, and the advisory details if one exists. Investigate in Git: use git log with date and author filters on the dependency's history, git show for the suspect commits, and git diff between the last known-good tag and the compromised one, looking for added network calls, obfuscated code, install scripts, changed build or CI files, new maintainers or unusual signing. Check commit metadata: authorship versus committer mismatches, signature verification status, odd timestamps, and whether the commit arrived via a pull request or direct push; git blame and log on changed files show prior history. Check whether released artefacts match the source: compare the published package with the tagged source, as attackers sometimes change only the published artefact. Then assess my exposure: find which of my repositories and builds depend on the affected versions, using lockfiles in Git history, git log -S on the lockfile to see when the version was introduced, and SBOMs. Determine whether affected builds ran and where it was deployed.

Likely follow-ups

- How does git log -S help trace when a version was introduced?
- Why compare the published package with the source?

Git forensics: use history, blame, and signatures to establish what changed, when, by whom, and whether it was authentic.

certs.eknathalabs.com

```
git log --all --source --oneline -- <suspect-file>   # full history of the file
```

```
git log -p -- <file>                # see every diff to the file over time
```

```
git blame -w -C <file>              # who last touched each line (ignore whitespace/moves)
```

```
git log --show-signature            # which commits are cryptographically verified
```

```
git log --since='2 weeks ago' --author='...'   # scope by time/author
```

```
git diff <last-good-tag> HEAD -- <path>   # what changed since known-good
```

Start from the last known-good tag and diff forward to isolate the introduced change. `git blame -C` attributes lines (the `-C` detects moved/copied code, defeating obfuscation by relocation). `--show-signature` reveals whether the malicious commit was signed by a trusted key or spoofed — unsigned or unexpected-author commits in a signed repo are red flags. The reflog on build machines can reveal what was actually checked out and built.

Lesson from xz: the attack used a trusted maintainer identity over time, so signature checks alone were insufficient — provenance and behavioral review matter too. Mention that detection combines Git forensics with build provenance and anomaly detection, not history inspection alone.

📚 **Official docs**

- Git documentation

certs.eknathalabs.com

Q86 How do you handle database migrations safely in a Git-driven deploy?

CI/CD & GitOps

🎙 Spoken answer · ~85 s

In a Git-driven deployment, the migration scripts are versioned with the code and deployed by the pipeline, so safety depends on process and design. Keep migrations in the repository as ordered, immutable files managed by a tool like Flyway, Liquibase, Alembic, or the framework's migrator, and never edit one that has been applied. Design every change as backward compatible with expand and contract, since during a rolling deployment both the old and new application versions run against the same schema: add columns or tables first, deploy code that uses them, backfill data in batches, and remove old structures in a later release. Run migrations as a single, controlled pipeline step before the application rollout, not at startup of each replica, using a lock so only one runs. Validate in CI by applying all migrations to a fresh database and to a production-like copy, measuring duration and locks; lint risky operations. Use lock timeouts and online techniques, such as concurrent index creation, and batched updates for large tables. Gate production with review of the migration pull request, especially destructive operations, and take a snapshot or backup first. Plan rollback as roll forward or application rollback, since reversing schema changes is risky; ensure old code remains compatible.

Likely follow-ups

- Why must migrations be backward compatible in rolling deploys?
- Why not run migrations on every pod start?

Migrations are the riskiest part of a deploy because they are stateful and often irreversible. The pattern is versioned, ordered, expand/contract migrations decoupled from code deploys.

PRINCIPLES

```
Versioned migration files in Git, applied in order (Flyway/Liquibase/Alembic)
```

```
Expand/contract: add column → deploy code using it → backfill → drop old (separate deploys)
```

```
Backward-compatible: new code must work with old schema and vice versa during rollout
```

```
Migrations run as a distinct, gated pipeline step — not silently in app startup
```

The core technique is **expand/contract** (parallel change): never make a breaking schema change in one step. Add the new structure (expand), ship code that writes to both/reads new, backfill data, then later remove the old (contract) — each as a separate, independently-rollbackable deploy. This keeps old and new app versions both functional during a rolling deploy, which is what makes zero-downtime and rollback possible.

The trap: a migration that drops a column in the same deploy that stops using it means you cannot roll back the code without a forward fix — the old code expects the column that is now gone. Decouple schema changes from code changes across multiple deploys.

certs.eknathalabs.com

📚 **Official docs**

- Git documentation

Q87 What does git fsck check, and how is it used for integrity verification?

Security

🎙 **Spoken answer · ~83 s**

git fsck, file system check, verifies the integrity and connectivity of the object database. It checks that objects are well-formed and match their hashes, that references point to existing objects, that trees reference valid blobs and subtrees, that commits reference valid trees and parents, and it reports problems such as corrupt, missing, or malformed objects, broken links, and dangling objects. Dangling objects are unreachable objects, like commits left after a reset or deleted branch, which isn't an error but is useful for recovery. Useful options: --full checks packfiles and alternates, --strict is stricter about format issues, --unreachable lists objects not reachable from any reference, --lost-found writes dangling commits and blobs to .git/lost-found for recovery, and --connectivity-only skips content checks for speed. Integrity verification use: after suspected disk problems, interrupted operations, or a crash; before and after migrating or repacking repositories; when a clone or fetch complains about missing or bad objects; as a scheduled check on servers hosting repositories; and in forensic work to verify that history hasn't been tampered with. Because object IDs are content hashes, fsck effectively verifies the whole Merkle structure. Settings like transfer.fsckObjects, fetch.fsckObjects, and receive.fsckObjects make Git validate objects during transfers, rejecting malformed data.

Likely follow-ups

- What is a dangling object?
- What does transfer.fsckObjects protect against?

`git fsck` (file system check) verifies the integrity and connectivity of the object database — confirming objects are not corrupt and references resolve.

```
git fsck                          # verify object integrity + connectivity
```

```
git fsck --full                   # check all objects including packed
```

```
git fsck --unreachable            # list objects not reachable from any ref
```

```
git fsck --lost-found             # write dangling commits/blobs for recovery
```

```
git fsck --connectivity-only      # faster: skip content hash verification
```

It validates that each object's content still matches its SHA (detecting bit-rot or tampering, since the hash is the address) and that the commit/tree/blob graph is fully connected. Two main uses: **integrity** — detect repository corruption, e.g. after a disk issue; and **recovery** — `--lost-found` surfaces dangling objects (orphaned commits from a bad reset, deleted branches) so you can rescue lost work.

certs.eknathalabs.com

Security angle: because Git is content-addressed, fsck detecting a hash mismatch means an object was altered — a tamper signal. SHA-1 collision concerns drove the move toward SHA-256 repos; mentioning that shows depth.

📚 **Official docs**

- Linux kernel docs: ext4
- Git documentation

Q88 How would you implement automatic rollback tied to Git in a deploy system?

CI/CD & GitOps

🎙 **Spoken answer · ~85 s**

Automatic rollback tied to Git means that when a deployment is unhealthy, the system returns to the last known-good commit or version without manual steps, and Git records it. Components. Immutable, versioned artefacts tagged by commit SHA, so the previous good version is always available. Health signals: readiness probes, smoke tests, and monitoring metrics such as error rate and latency compared with a baseline, evaluated for a bake period after deployment. A controller that watches those signals: Argo Rollouts or Flagger analyse canary metrics from Prometheus and automatically abort and revert traffic or replica sets when thresholds are exceeded; Kubernetes alone stops a rollout if new pods don't become ready but needs a progress deadline plus an external trigger to undo. For GitOps, the rollback must be reflected in Git so the declared state matches reality, otherwise Argo CD would reapply the bad version: so the pipeline or a bot creates a revert commit in the configuration repository, or pins the previous version, and the agent syncs it. Alternatively, pause automated sync and use the tool's rollback to a known revision while a revert is prepared. In push-based pipelines, a failure step runs a rollback job redeploying the previous digest and opens an incident.

Likely follow-ups

- Why must the rollback be reflected in Git?
- What limits automatic rollback when migrations are involved?

Rollback should be a fast, deterministic Git/artifact operation triggered by health signals — not a manual scramble. The design depends on building immutable, versioned artifacts.

GITOPS-STYLE ROLLBACK

```
git revert <deploy-commit>          # revert the config commit that bumped the image
```

```
# controller (Argo/Flux) auto-reconciles cluster back to previous state
```

```
argocd app rollback <app> <revision>   # explicit rollback to a prior synced revision
```

certs.eknathalabs.com

PIPELINE-DRIVEN ROLLBACK

```
Health check / SLO breach after deploy → redeploy previous tag's artifact
```

```
Progressive delivery (Argo Rollouts/Flagger): auto-abort canary on metric regression
```

```
Keep last-known-good tag/SHA recorded for instant redeploy
```

Because you build the artifact once and tag it by SHA, rollback is simply "redeploy the previous tag" — no rebuild, fully deterministic. In GitOps, rollback is `git revert` of the config change, and the controller reconciles automatically, preserving the audit trail. Progressive delivery tools watch metrics during a canary and auto-abort/rollback on regression before full traffic shifts.

> **Make rollback boring:** the best rollback is automatic and metric-driven (canary abort) so a human never has to decide under pressure. Pair Git-based rollback with health gates and you get safe deploys by default.

> 📚 **Official docs**
>
> - Kubernetes: Deployments
> - Git documentation

certs.eknathalabs.com

Q89 Explain ephemeral preview environments per pull request and how Git drives them.

CI/CD & GitOps

🎙 Spoken answer · ~85 s

Ephemeral preview environments are temporary, isolated deployments created automatically for each pull request, so reviewers and testers can see and use the change running live before it merges. Git drives the lifecycle through events: when a pull request is opened or updated, the pipeline builds the branch's artefact, tagged with the commit SHA, and deploys it to a uniquely named environment, for example a Kubernetes namespace pr-1234, with a unique URL like pr-1234.preview.example.com; it posts the link as a comment on the pull request. New pushes redeploy it. When the pull request is merged or closed, a cleanup job destroys the environment and its resources. Implementation approaches: a CI workflow using Helm or Kustomize with the PR number as a variable; Argo CD ApplicationSets with the pull request generator, which creates and removes an Application for each open PR automatically; or platform features like Vercel, Netlify, and Render previews. Considerations: data, using seeded, anonymised, or ephemeral databases rather than production data; shared dependencies, either mocked or pointed to a shared dev stack; secrets, scoped and minimal; cost control, with resource limits, automatic expiry, and scale-to-zero; wildcard DNS and TLS certificates; authentication so previews aren't public; and security for fork PRs, which shouldn't receive secrets.

Likely follow-ups

- How does Argo CD's pull request generator work?
- How do you control cost for preview environments?

A preview environment is a temporary, fully-functional deployment spun up automatically for each PR, letting reviewers and stakeholders test the change in isolation before merge.

LIFECYCLE DRIVEN BY GIT EVENTS

```
PR opened   → CI builds the branch + deploys a namespaced env (pr-123.preview.app)
```

```
PR updated  → redeploy the env with the new commit
```

```
PR merged/closed → tear down the env (free resources)
```

IMPLEMENTATION

```
Kubernetes namespace per PR, or vcluster, or platform (Vercel/Netlify) preview
```

```
Seeded/ephemeral data; isolated dependencies; unique URL posted back to the PR
```

Git events are the triggers: opening a PR creates the env, pushing updates it, closing it tears it down. The value is testing the actual running change — not just reading a diff — with a shareable URL for designers, PMs, and QA. The hard parts are data isolation (each env needs realistic but safe data) and cost control (auto-teardown is essential).

certs.eknathalabs.com

Platform value: preview environments dramatically improve review quality and shift testing left. Building this as a self-service capability is a strong platform-engineering deliverable to cite.

📚 **Official docs**

- Terraform: Sensitive data in state
- Git documentation

Q90 How do you audit who changed what in a repository for compliance?

Security

🎙 **Spoken answer · ~86 s**

Compliance auditing requires reliable answers to who changed what, when, why, and who approved it. Git itself provides much of this: commit history records author, committer, timestamp, message, and diff, and signed commits prove identity. git log with file and author filters, and blame, answer who changed a line. But Git metadata alone can be forged or rewritten, so you need controls around it. Protect history: branch protection blocks force pushes and deletions on main, so history is immutable in practice. Enforce review: required pull requests with approvals, code owner review, and status checks, so every change has an independent approver and passing tests; the pull request records discussion, approvers, and linked tickets. Require signed commits for non-repudiation. Platform audit logs record administrative actions: access changes, permission changes, branch protection edits, token creation, repository visibility changes, and force-push attempts; stream these to a SIEM with long retention. Link changes to work items by requiring ticket IDs in commit messages or PR titles. For deployments, connect commits to what ran in production through artefact tags and deployment records, so you can trace production back to approved changes. Evidence collection: automate reports of changes in a period, approvals, and exceptions, to satisfy SOC 2 or ISO 27001 auditors.

Likely follow-ups

- Why isn't Git history alone sufficient evidence?
- How do you trace production back to approved changes?

certs.eknathalabs.com

Git itself records authorship and content; the hosting platform adds the access and administrative audit layer. Compliance needs both.

FROM GIT

```
git log --all --pretty=fuller       # author AND committer + timestamps
```

```
git log --show-signature            # verified identity (not just claimed)
```

```
git blame -w <file>                 # line-level attribution
```

```
git log --follow -- <file>          # history across renames
```

FROM THE PLATFORM (THE REAL AUDIT TRAIL)

```
Audit log: who merged, who changed branch protection, who added access
```

```
Stream audit events to SIEM / immutable storage (tamper-evident)
```

```
Required signed commits so 'author' is verified, not spoofable
```

Git's author field is self-reported and forgeable, so for compliance you require **signed commits** (verified identity) and rely on the platform's **audit log** for who-merged-what, who-changed-settings, and access grants. Streaming those audit events to a SIEM or immutable store gives a tamper-evident record. `git log --pretty=fuller` distinguishes author (wrote it) from committer (applied it) — relevant after rebases/cherry-picks.

Key insight for auditors: unsigned commit metadata is not evidence — it can be forged. Verified signatures + platform audit logs are the defensible record. State this explicitly; it shows you understand the trust model.

📚 **Official docs**

- AWS CloudTrail

Q91 What is the difference between merge, squash-merge, and rebase-merge for integrating PRs? CI/CD & GitOps

🎙 **Spoken answer · ~76 s**

Platforms offer three ways to integrate a pull request, each producing a different history. A merge commit preserves every commit from the branch and adds a merge commit with two parents. It keeps the full, true history, including the branch structure, but main becomes non-linear and noisy with 'fix typo' commits and merge bubbles; reverting the whole feature means reverting the merge with -m 1. A squash merge collapses all the branch's commits into one new commit on main, whose message is usually the PR title and description. History is linear, with one commit per pull request, easy to read, bisect, and revert; but individual commits and their granularity are lost, authorship becomes the PR author with co-author trailers, and later branches stacked on the original will conflict. A rebase merge replays the branch's commits on top of main individually, without a merge commit, creating new commits with new hashes: linear history while retaining the individual commits. It requires well-crafted, atomic commits, and it rewrites hashes, so the commits on main differ from those on the branch and signatures are not preserved.

Likely follow-ups

- Which strategy loses individual commit granularity?
- Why does a rebase merge change commit hashes?

Three ways a PR lands on main, each shaping history differently. The choice is a team policy trade-off between traceability and tidiness.

coris.eknathalabs.com

THE THREE STRATEGIES

```
Merge commit   : preserves all PR commits + a merge commit; full history, busier graph
```

```
Squash-merge   : collapses the PR into ONE commit on main; clean linear history
```

```
Rebase-merge   : replays PR commits onto main individually; linear, no merge commit
```

Squash-merge is the most common default: main gets one tidy, revertible commit per PR, and messy WIP commits vanish — great for clean history and easy reverts, at the cost of losing intra-PR granularity (bisect lands on big commits). **Merge commits** preserve everything and clearly delineate PR boundaries, but clutter the graph. **Rebase-merge** keeps individual commits in a linear history but rewrites their SHAs and can be confusing if PR commits were not curated.

> **Recommendation to state:** squash-merge for most product teams (clean, revertible, one-commit-per-feature), reserving merge commits for cases where preserving individual commits matters (e.g. a curated series). Consistency per repo matters more than the specific choice — and enforce it in branch settings.

> 📚 **Official docs**
> - git-rebase

certs.eknathalabs.com

Q92 How do you handle access control and least privilege across many repositories?

Security

🎙 Spoken answer · ~87 s

Across many repositories, access control must be systematic. Principles. Authenticate centrally with SSO and enforce MFA; manage membership through your identity provider's groups synced to teams on the Git platform, so joiners, movers, and leavers are handled automatically. Grant access to teams, never individuals, with roles like read, triage, write, maintain, and admin, assigned at the lowest necessary level; most people need read or write, very few need admin. Base permissions for the organisation should be none or read, with write granted per repository through teams. Use rulesets or organisation-level policies to apply the same branch protection baseline everywhere, so nobody can push directly to main, and CODEOWNERS for sensitive paths. Control tokens and apps: disable classic personal access tokens, prefer fine-grained tokens and GitHub Apps with minimal, repository-scoped permissions, and review installed OAuth apps and deploy keys periodically. Restrict who can create repositories, change visibility, or fork private repositories. Separate duties: production deployment credentials and environment approvals owned by distinct groups. Automate: manage repository settings and team access as code with Terraform or a settings-as-code tool, so drift is detected and changes are reviewed. Review: scheduled access recertification, removal of inactive accounts and stale deploy keys, and audit log monitoring for privilege changes, new tokens, and bulk cloning.

Likely follow-ups

- Why assign access to teams instead of individuals?
- How do you manage repository settings as code?

Scale access control through groups, roles, and automation — never per-user, per-repo grants, which drift into over-permissioning.

PRINCIPLES & MECHANISMS

```
Teams/groups map to roles; assign repos to teams, users to teams (not repos directly)
```

```
Least privilege: read by default, write/maintain/admin only where needed
```

```
CODEOWNERS for review authority ≠ write access (separate concerns)
```

```
Time-bound / just-in-time elevation for admin actions
```

```
Automate via IaC: Terraform GitHub provider manages teams + repo access in Git
```

```
Periodic access reviews; remove dormant access; SSO + SCIM deprovisioning
```

The scalable model is role/group-based: define teams that map to responsibilities, grant repos to teams, and add people to teams — so offboarding (remove from group / SCIM deprovision) instantly revokes everywhere. Manage all of it as code (Terraform GitHub provider) so access itself is reviewed via PR and audited in Git. Default to read; grant write/admin narrowly; review periodically to catch privilege creep.

certs.eknathalabs.com

Managing access as code is the platform-engineering move: access changes become reviewable, auditable commits, and there is no manual console drift. Pair with SSO/SCIM so identity lifecycle drives access automatically.

📚 **Start here (official docs)**

- Git documentation

Q93 Explain the three-way merge algorithm Git uses.

Advanced & internals

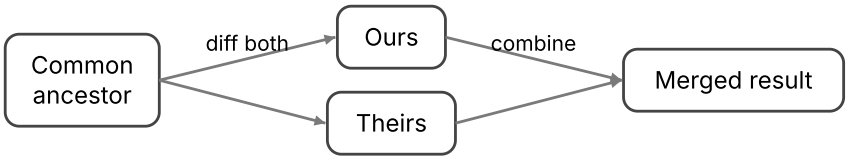
🎙 **Spoken answer · ~74 s**

A three-way merge combines two versions of a file by using their common ancestor as a reference. When merging branches A and B, Git finds the merge base, the most recent common ancestor commit, and compares each side against it. For each region of the file: if only A changed it, take A's change; if only B changed it, take B's; if both made identical changes, take that; if neither changed it, keep the original; and if both changed the same region in different ways, that's a conflict, and Git inserts conflict markers for a human to resolve. The ancestor is what makes this smarter than a two-way comparison, which can't tell who changed what, and would show every difference as conflict. Git's default strategy for merging two branches is ort, which replaced recursive; it handles renames by detecting similar files, and when there are multiple merge bases, as in criss-cross merges, it merges the bases to create a virtual ancestor. The same three-way logic powers rebase and cherry-pick, where the commit's parent plays the base role.

Likely follow-ups

- Why does the common ancestor matter?
- What is a criss-cross merge?

🗺 **Three-way merge**



Git merges using three inputs: the two branch tips and their common ancestor (merge base). The ancestor is what lets Git tell "added" from "deleted" and minimize false conflicts.

THE THREE POINTS

```
git merge-base main feature        # find the common ancestor commit
```

```
# Git compares:  BASE ↔ OURS  and  BASE ↔ THEIRS
```

certs.eknathalabs.com

```
# For each hunk: if only one side changed it vs base → take that side
```

```
#               if both sides changed the same region → CONFLICT
```

A two-way diff (just comparing the two tips) cannot tell whether a line was added on one side or deleted on the other. The **merge base** resolves this: changes present on one side but not the base are applied cleanly; only regions changed on *both* sides relative to the base become conflicts. This is why Git auto-merges most concurrent edits to different parts of a file.

Recursive/ort strategy: when branches have multiple merge bases (criss-cross merges), Git's default `ort` strategy merges the bases themselves first to synthesize a virtual ancestor. Mentioning `ort` (which replaced `recursive` as default) signals current knowledge.

📚 **Official docs**

- RFC 9293: TCP
- git-merge
- Git documentation

cons.EknathaLabs.com

Q94 How do you resolve a complex merge conflict methodically?

Merging & rebasing

🎙 **Spoken answer · ~88 s**

For a complex conflict I work methodically rather than editing markers hastily. First, understand the context: git status shows the conflicted files; git log --merge --oneline shows the commits on each side touching them; and git diff --name-only --diff-filter=U lists unresolved paths. Set merge.conflictStyle to zdiff3 so markers include the common ancestor. For each file, understand what each side was trying to achieve, by reading the commit messages, linked pull requests, and talking to the other author if the intent is unclear. Sometimes it's best to resolve in stages: handle the mechanical, renamed, or moved parts first, then the semantic ones; for big conflicts, merge main into the branch in smaller increments. Use a three-pane merge tool, with git mergetool, showing local, remote, base, and result. Resolve by combining intent rather than picking sides, and use git checkout --ours or --theirs only when one side should win wholesale, remembering these names swap in a rebase. For generated files or lockfiles, regenerate them after resolving the source rather than hand-editing. After resolving, remove all markers, build, run tests and linters, and review the final diff with git diff against both parents. Stage and continue the operation. If it's becoming a mess, abort and rethink. Enable rerere so repeated resolutions are reused.

Likely follow-ups

- What do ours and theirs mean during a rebase?
- Why regenerate lockfiles instead of hand-merging?

Conflicts are resolved by understanding intent on both sides, not by blindly picking ours/theirs. Tooling and discipline keep it safe.

```
git status                          # list conflicted files
```

```
git diff                            # see conflict markers in context
```

```
git checkout --conflict=diff3 <file>  # show BASE too, not just ours/theirs
```

```
git mergetool                       # launch a visual 3-way merge tool
```

```
git add <file>                      # mark resolved
```

```
git merge --continue                # or commit
```

```
git checkout --ours <file>          # take our version wholesale (use deliberately)
```

```
git checkout --theirs <file>        # take their version wholesale
```

Enable `diff3` conflict style so you see the common **base** alongside both sides — understanding what each side changed *from* is essential to merging intent rather than guessing. Resolve, then re-run the tests before continuing; a syntactically-resolved conflict can still be semantically wrong. For repetitive conflicts across rebases, enable `git rerere` to record and replay resolutions.

> **Do not blindly --theirs/--ours** a whole file to make conflicts disappear — that silently drops the other side's changes. The most dangerous conflict resolution is one that compiles but loses someone's work.

> 📚 **Official docs**
> - git-merge

certs.eknathalabs.com

Q95 What is git rerere and when does it pay off?

Advanced & internals

🎙 Spoken answer · ~69 s

git rerere, reuse recorded resolution, remembers how you resolved a particular conflict and automatically applies the same resolution when it sees the same conflict again. Enable it with git config rerere.enabled true. When a conflict occurs, Git records the preimage, the conflicted state; when you resolve and commit, it records the postimage, the resolution, keyed by a hash of the conflict. On later encounters of an identical conflict, Git applies the recorded resolution automatically, though you still need to stage and review the result. Cases where it pays off. Long-running branches that you repeatedly rebase or merge main into: the same conflicts reappear at each sync, and rerere resolves them once. Rebasing a stack of commits, where the same conflict recurs at several commits. Trying a merge, aborting, and retrying; testing a merge in a throwaway branch to learn the resolution, then redoing it for real; maintainers who carry a topic branch across many upstream updates; and when rebase is redone after rewording or reordering.

Likely follow-ups

- Where are recorded resolutions stored?
- How do you undo a bad recorded resolution?

rerere = "reuse recorded resolution." Git remembers how you resolved a specific conflict and auto-applies the same resolution if that exact conflict recurs.

```
git config rerere.enabled true
```

```
# Resolve a conflict once; Git records the before/after
```

```
# Next time the identical conflict appears (e.g. on re-rebase), Git auto-resolves it
```

```
git rerere status                  # show what rerere is tracking
```

```
git rerere diff                    # show recorded resolutions
```

It pays off in two situations: rebasing a long-lived branch repeatedly onto a moving main (you would otherwise re-resolve the same conflicts each time), and maintaining a long-running merge between two diverging branches. Without rerere, every rebase re-presents conflicts you already solved; with it, the resolution is replayed automatically.

Subtle but loved: few engineers know rerere. Citing it for "I keep re-resolving the same conflict on each rebase" shows you have done real long-branch maintenance and know the ergonomic fixes.

📚 Official docs

- Git documentation

certs.eknathalabs.com

Q96 Explain packfiles, loose objects, and what git gc does.

Advanced & internals

🎙 Spoken answer · ~82 s

Git stores objects in two forms. Loose objects are individual files in .git/objects, named by hash, each zlib-compressed; every new blob, tree, and commit starts life this way. They're simple and quick to write but inefficient at scale: many small files, no cross-object compression. Packfiles store many objects in a single file with an accompanying index for fast lookup. Inside a pack, similar objects are stored as deltas against a base object, so successive versions of a file take little space; Git chooses delta bases heuristically, often by file name and size. Packs are created when you push or fetch, since transfers use packs, and by git gc and git repack. What git gc does: packs loose objects into packfiles, consolidates multiple packs, compresses with deltas, packs references into the packed-refs file, removes unreachable objects older than the expiry, normally two weeks, expires old reflog entries, and updates auxiliary data such as the commit-graph. git gc --auto runs automatically when there are too many loose objects or packs, and servers should schedule git maintenance. Why it matters: repository size on disk, clone and fetch performance, and operation speed. Aggressive repacking is slower but yields smaller packs.

Likely follow-ups

- How do packfiles save space?
- What do loose objects look like on disk?

Git stores objects two ways: loose (one file per object, fast to write) and packed (many objects delta-compressed into a packfile, efficient to store/transfer). gc converts and optimizes.

```
ls .git/objects/                    # loose objects in 2-char-prefixed dirs
```

```
ls .git/objects/pack/               # packfiles (.pack) + indexes (.idx)
```

```
git gc                              # pack loose objects, prune, build commit-graph
```

```
git gc --aggressive                 # recompute deltas more thoroughly (slow)
```

```
git count-objects -vH               # see loose vs packed sizes
```

```
git repack -ad                      # repack everything into one pack
```

New objects start **loose** (cheap to create). Over time `git gc` (run automatically) packs them into **packfiles** using delta compression — storing similar objects as deltas against each other, which dramatically shrinks repo size and speeds up clones/fetches (the network transfers packs). gc also prunes unreachable objects past the grace window and builds the commit-graph for fast history traversal.

Operational note: `git maintenance start` schedules background gc, prefetch, and commit-graph updates — the modern, incremental replacement for periodic `gc --aggressive`, keeping large repos fast without disruptive full repacks.

certs.eknathalabs.com

📚 **Official docs**

- Pro Git: Git internals (objects)
- Git: Maintenance & garbage collection
- Git documentation

Q97 **How does Git handle large binary files, and what is Git LFS doing under the hood?**

Advanced & internals

🎙 **Spoken answer · ~83 s**

Git handles binary files poorly for two reasons: it stores a full copy of each version, since binaries rarely delta-compress well, so the repository balloons with every change, and it cannot merge them, so concurrent edits cause unresolvable conflicts. Every clone also downloads all historical versions. Git LFS, Large File Storage, solves the size problem by storing large files outside the repository's object database. Under the hood it uses Git's clean and smudge filters, configured via .gitattributes patterns set by git lfs track. When you git add a tracked file, the clean filter replaces the file's content with a small text pointer file, containing the LFS spec version, the sha256 oid of the content, and the size; the real content is saved in the local .git/lfs/objects cache. On git push, a pre-push hook uploads the actual content to the LFS server over an HTTP API, while the pointer goes into Git history. On checkout, the smudge filter reads the pointer, downloads the content from the LFS server if it isn't cached, and puts the real file in the working tree. So history remains small, and clones fetch only the large files needed for the commit being checked out.

Likely follow-ups

- What do the clean and smudge filters do?
- What does a pointer file look like?

Git is built for text: it stores full snapshots and delta-compresses similar objects. Binaries do not delta well, so each version bloats history permanently. Git LFS solves this with pointers.

THE PROBLEM

```
Every version of a 50MB binary is stored in full in history — forever
```

```
Clones must download all historical versions; repo balloons
```

certs.eknathalabs.com

HOW LFS WORKS

```
git lfs track '*.psd'              # writes patterns to .gitattributes
```

```
# On commit, the file is replaced in Git by a small text POINTER:
```

```
#   version, oid (sha256), size
```

```
# The real bytes go to a separate LFS store; fetched on checkout on demand
```

```
git lfs ls-files                   # list LFS-tracked files
```

LFS replaces the binary in Git history with a tiny pointer file (an OID + size); the actual content lives in a separate LFS server and is downloaded only for the versions you check out. So history stays small and clones are fast, because you do not drag every historical revision of every asset. `.gitattributes` records which patterns are LFS-managed.

Caveat: LFS needs server support and adds operational complexity (storage quotas, a separate store, migration of existing history with `git lfs migrate`). For truly large or non-versioned assets, an object store (S3/GCS) referenced by URL is sometimes the better answer than putting them in Git at all.

📚 **Official docs**

- Git LFS
- Git documentation

certs.eknathalabs.com

Q98 What is the commit-graph file and how does it speed up Git operations?

Advanced & internals

🎙 Spoken answer · ~83 s

The commit-graph file is a precomputed, compact binary cache, stored in .git/objects/info/commit-graph, of the commit history structure: for each commit it holds the commit's parents, root tree ID, commit date, and a generation number, in an efficient format. Normally, to traverse history, Git has to read and decompress each commit object from the object database, which is slow in large repositories with hundreds of thousands of commits. With the commit-graph, walking history reads this compact file instead, speeding up commands that traverse the graph, such as git log with graph and filters, git merge-base, reachability checks like git branch --contains and git tag --contains, and fetch and push negotiation. The generation number, a value derived from the longest path to a root, makes reachability queries much faster because Git can stop walking early when a commit's generation is lower than the target's. Newer versions also support changed-path Bloom filters, which record which paths each commit modified, greatly accelerating git log -- path and blame. Writing: git commit-graph write --reachable, or automatically via fetch.writeCommitGraph and gc.writeCommitGraph; git maintenance schedules incremental updates. Benefits are most visible on large monorepos and servers; hosting platforms rely on it heavily. Verify with git commit-graph verify.

Likely follow-ups

- What do Bloom filters in the commit-graph speed up?
- What is a generation number?

The commit-graph is an auxiliary file caching commit metadata and a precomputed reachability structure, so Git avoids walking and parsing commit objects one by one.

```
git commit-graph write --reachable   # build it
```

```
git config core.commitGraph true     # enable usage (default in modern Git)
```

```
git config fetch.writeCommitGraph true
```

```
# generation numbers enable fast 'is A an ancestor of B?' answers
```

Operations like `git log --graph`, `git merge-base`, `git tag --contains`, and ahead/behind counts require traversing and parsing many commit objects. The commit-graph stores parents, commit dates, and **generation numbers** in a single memory-mappable file, turning expensive walks into fast lookups. On large repos this is the difference between sub-second and multi-second history queries.

Where it shows up: it is part of the modern large-repo toolkit alongside partial clone, sparse-checkout, and `git maintenance`. Mentioning generation numbers (fast ancestry tests) demonstrates real internals knowledge.

certs.eknathalabs.com

📚 **Official docs**

- Git documentation

Q99 **When and why would you use git merge --no-ff vs a fast-forward merge?**

Merging & rebasing

🎙 **Spoken answer · ~85 s**

When merging, Git can fast-forward if the target branch hasn't diverged: the target's tip is an ancestor of the branch being merged, so Git just moves the pointer forward, with no merge commit, producing linear history in which the branch's commits appear as if made directly on main. git merge --no-ff forces a merge commit even when a fast-forward is possible, creating an explicit commit with two parents that records that a branch existed and was integrated. Why use no-ff. It preserves the grouping of a feature's commits under one merge commit, so you can see where the feature started and ended, and revert the entire feature with a single git revert -m 1 on the merge commit. It documents integration points, helpful for release notes and audits, with the pull request referenced in the merge message. Why use fast-forward. It keeps history linear and simple, free from clutter, and is natural when the branch has been rebased onto main beforehand; git merge --ff-only refuses to create merge commits, ensuring linear history and failing if main moved, which can be a protective default through merge.ff only. GitFlow's tradition is no-ff for feature merges; trunk-based teams generally prefer linear history using squash or rebase merges.

Likely follow-ups

- What does --ff-only protect against?
- How does no-ff make reverting a feature easier?

A fast-forward merge just moves the branch pointer forward when there is no divergence — leaving no record that a branch existed. `--no-ff` forces a merge commit to preserve that boundary.

```
git merge feature                  # fast-forwards if main hasn't diverged (no merge commit)
```

```
git merge --no-ff feature          # always create a merge commit, recording the branch
```

```
git merge --ff-only feature        # refuse to merge unless it can fast-forward
```

--no-ff preserves feature-branch boundaries in history — you can see "these commits were one PR" and revert the whole feature with a single `revert -m 1` of the merge commit. Useful for traceability and easy feature-level rollback. **Fast-forward** (or squash) keeps history linear and uncluttered. **--ff-only** is a safety setting (often on `git pull`) that refuses to silently create merge commits, forcing you to rebase intentionally.

Policy angle: `--no-ff` + meaningful merge commits gives feature-level grouping and one-shot reverts; squash gives the cleanest linear log. State the trade-off (traceability vs tidiness) and that consistency

certs.eknathalabs.com

per repo matters.

📚 **Official docs**

- git-merge
- Git documentation

Q100 Explain how git stash works internally and its common pitfalls.

Advanced & internals

🎙 **Spoken answer · ~86 s**

Internally, git stash saves your local modifications as commits and stores a reference to them. When you run git stash, Git creates a commit capturing the index state, with HEAD as its parent, and a second commit capturing the working tree, with both HEAD and the index commit as parents, so the stash is actually a merge-like commit; with -u, a third commit holds untracked files. The reference refs/stash points to the latest stash commit, and older entries are kept in the reflog of refs/stash, which is why stash@{n} syntax matches reflog syntax. After saving, Git resets the working tree and index to HEAD. git stash apply or pop does a three-way merge to reapply the changes, using the original base commit as the common ancestor, which means it works even if the branch has moved on, with potential conflicts. Pitfalls. Stashes are local and not pushed, so they're easily forgotten and not backed up. Untracked and ignored files aren't stashed by default, needing -u or -a. Pop on conflict doesn't drop the stash, leaving it on the list, which can confuse; always check. Old stashes become hard to apply as code drifts. They aren't tied to a branch, so you may apply to the wrong one.

Likely follow-ups

- What are the three commits in a stash?
- Why does a conflicting pop keep the stash?

A stash is not magic storage — it is real commits. Git creates commit objects capturing your work and parks them on a special ref, leaving your working tree clean.

```
git stash                          # stash tracked changes
```

```
git stash -u                       # include untracked files
```

```
git stash push -m 'wip: parser'    # named stash
```

```
git stash list                     # refs/stash + reflog of stashes
```

```
git stash show -p stash@{0}        # view a stash's diff
```

```
git stash pop                      # apply + drop
```

```
git stash apply                    # apply, keep in list
```

certs.eknathalabs.com

```
cat .git/refs/stash                # the stash ref (a commit)
```

Internally a stash builds (at least) two commits — one for the index, one for the working tree — with your current commit as parent, stored under `refs/stash` as a stack (reflog). Because they are real commits, **a dropped or popped stash is recoverable via the reflog / fsck** even though it is off the stash list.

Pitfalls: (1) `git stash` ignores untracked files unless you add `-u` — people lose new files this way. (2) `pop` can conflict and then NOT drop the stash, causing confusion. (3) stashes are easy to forget — prefer short-lived WIP commits or worktrees for anything beyond a quick context switch.

📚 **Official docs**

- git-stash
- Git documentation

Q101 What are Git refspecs and how do they control fetch/push behavior?

Advanced & internals

🎙 **Spoken answer · ~88 s**

A refspec tells Git how to map references between a remote and the local repository. The format is [+]<src>:<dst>. In fetch, src is the remote ref and dst the local ref where it is stored; the default fetch refspec in .git/config is +refs/heads/*:refs/remotes/origin/*, meaning that every branch on origin is fetched into a remote-tracking branch under refs/remotes/origin, with the plus sign allowing non-fast-forward updates of the tracking ref. In push, src is the local ref and dst is the remote ref: git push origin main:release pushes local main to the remote branch release; git push origin :old-branch, with an empty source, deletes the remote branch; git push origin HEAD:refs/heads/feature pushes the current commit regardless of the name. Wildcards allow patterns. You can customise fetch behaviour by adding refspecs: for example fetching GitHub pull request heads with +refs/pull/*/head:refs/remotes/origin/pr/*, which makes every pull request available locally as a branch, useful for review tooling and CI; or limiting the fetch to one branch for speed. Tags have their own default refspec behaviour: refs/tags/*:refs/tags/*. Settings such as push.default and remote.<name>.push define defaults. Understanding refspecs explains the confusing parts of Git: why origin/main is a separate local reference, how to mirror repositories with git push --mirror or +refs/*:refs/*, and how to delete or rename remote branches.

Likely follow-ups

- What does the + in a refspec mean?
- How do you fetch pull request heads?

A refspec maps refs between a remote and your local repo — `<src>:<dst>` with an optional leading + for force. It is the rule behind every fetch and push.

```
# Default fetch refspec for 'origin':
```

certs.eknathalabs.com

```
+refs/heads/*:refs/remotes/origin/*   # all remote branches → origin/* (force-update)
```

```
git fetch origin main:refs/remotes/origin/main
```

```
git push origin local-branch:remote-branch   # push local to a differently-named remote ref
```

```
git push origin :stale-branch         # empty src = DELETE the remote branch
```

```
git push origin HEAD:refs/for/main    # Gerrit-style review push
```

The + prefix allows non-fast-forward (force) updates — which is why your local `origin/*` tracking refs always update even after upstream rebases, while your own branches do not. Custom refspecs let you fetch/push to differently-named refs, fetch only specific branches, or delete remote branches (empty source). Understanding refspecs explains seemingly-magic behaviors like "why does fetch update origin/main but not my main?"

Configured in `.git/config` under each remote's `fetch =` line. Editing it (e.g. to fetch only one branch) is a real technique for limiting traffic on huge repos.

📚 **Official docs**

- Git documentation

certs.eknathalabs.com

Q102 How do you keep a fork in sync with upstream, and integrate upstream changes cleanly?

Merging & rebasing

🎙 Spoken answer · ~84 s

A fork is your own copy of someone else's repository on the hosting platform; keeping it current with upstream involves adding the original repository as a second remote. Setup: git remote add upstream <original-url>, with origin pointing to your fork; verify with git remote -v. To sync: git fetch upstream to retrieve the latest changes, then integrate into your local main. For a clean, linear approach when you don't commit directly to main: git switch main then git merge --ff-only upstream/main, or git reset --hard upstream/main if you want to discard divergence, then git push origin main to update your fork. Platforms also offer a sync fork button or API call. For feature branches on your fork, bring in upstream changes by rebasing: git rebase upstream/main, which replays your commits on top of the latest upstream, resolving conflicts as needed, then push with --force-with-lease since history changed; or merge upstream/main into the branch if you prefer to avoid rewriting. Best practices: never commit directly to your fork's main; do work on topic branches, so main can always fast-forward to upstream; sync before starting new work and before opening a pull request; keep pull requests small and up to date; and enable fetch.prune.

Likely follow-ups

- Why avoid committing directly to the fork's main?
- How do you rebase a feature branch onto upstream?

A fork needs both the upstream and your origin as remotes; you sync by fetching upstream and rebasing or merging your changes on top.

```
git remote add upstream <original-repo-url>
```

```
git remote -v                # origin (your fork) + upstream
```

```
git fetch upstream
```

```
git checkout main
```

```
git rebase upstream/main           # replay your changes on latest upstream (clean)
```

```
#   or:  git merge upstream/main    # if you prefer merge commits / shared branch
```

```
git push origin main               # update your fork
```

```
# For a feature branch you're contributing back:
```

```
git rebase upstream/main feature && git push --force-with-lease origin feature
```

certs.eknathalabs.com

Add the original as `upstream`, fetch it, and integrate. For a personal fork you control, **rebasing** onto `upstream/main` keeps your contribution a clean series on top of current upstream — exactly what maintainers want to review. Use `--force-with-lease` when re-pushing a rebased feature branch on your fork (safe force). Keep your fork's `main` a clean mirror of upstream and do work on feature branches.

Open-source etiquette: rebase your PR branch onto current upstream before review so it applies cleanly, and keep the commit series tidy. This is the workflow interviewers expect from anyone who contributes to OSS.

📚 **Official docs**

- Git documentation

Q103 What is git replace / git notes, and a scenario where each is useful?

Advanced & internals

🎙 **Spoken answer · ~85 s**

git replace and git notes are two lesser-known mechanisms. git replace creates a replacement reference that makes Git substitute one object for another when reading history, without rewriting commits or changing their hashes. For example, git replace <old-commit> <new-commit> stores a ref under refs/replace/. Everywhere Git reads the old object, it sees the new one instead. Scenario: grafting history together, such as joining a new repository to the archived history of a legacy system, by replacing the first commit of the new repository with a commit whose parent is the last commit of the old history, giving a continuous log without rewriting. Another is temporarily fixing a commit's content or parentage to test what a rewrite would look like, before running filter-repo to make it permanent. Replace refs aren't pushed by default, so collaborators won't see them unless they fetch refs/replace. git notes attach extra information to existing commits without altering them, stored in a separate notes ref, refs/notes/commits, keyed by commit hash. Scenario: CI results, code review links, deployment records, or audit annotations added after the fact, since commit messages are immutable. For instance, a deployment system adds a note 'deployed to prod on this date' to the commit, visible in git log.

Likely follow-ups

- Why would you graft history with git replace?
- Why add information as a note rather than amend the commit?

Both attach or substitute information without rewriting the original immutable history — useful when you must annotate or virtually edit history you cannot change.

GIT NOTES — ATTACH METADATA TO COMMITS

```
git notes add -m 'reviewed-by: ...' <sha>
```

```
git log --show-notes
```

```
# Use: CI results, review metadata, ticket links, attached to commits after the fact
```

certs.eknathalabs.com

GIT REPLACE — VIRTUAL OBJECT SUBSTITUTION

```
git replace <old-sha> <new-sha>     # transparently swap one object for another
```

```
# Use: graft histories, fix a broken object, or stitch a shallow/old history on
```

`git notes` attaches mutable annotations (review info, CI status, references) to immutable commits without changing their SHAs — handy for adding context after the fact. `git replace` creates a transparent substitution so tools see a different object in place of another, without rewriting history — classically used to graft a truncated/imported history onto an older archive, or to test a fix for a corrupt object.

Niche but telling: these are advanced plumbing. Knowing they let you annotate or virtually-edit history non-destructively (vs filter-repo which rewrites SHAs) signals deep familiarity.

📚 **Official docs**

- Git documentation

Q104 Explain octopus merges and when (or whether) to use them.

Merging & rebasing

🎙 **Spoken answer · ~85 s**

An octopus merge is a merge commit with more than two parents, created when merging three or more branches at once, such as git merge feature-a feature-b feature-c. It's Git's strategy for combining several heads in one operation, and the resulting merge commit has multiple parents. Limits: the octopus strategy only works if all the merges can be done automatically; if any conflicts arise, Git refuses and aborts, as it cannot ask a human to resolve in the middle of a multi-way merge. So it suits cases where the branches touch independent areas. Where it's used: the Linux kernel has some famous large octopus merges, and some maintainers merge many topic branches into an integration branch in one step; it can also be used in release preparation to combine independent topic branches neatly, or to keep history tidy when merging many small, unrelated changes. Whether to use it: for most teams, rarely. History with many-parent merges is hard to read and reason about, tools and visualisations handle them poorly, reverting one parent's changes out of an octopus is awkward since git revert -m selects only one parent, and bisecting through them is trickier. Sequential merges are clearer and let you resolve conflicts at each step.

Likely follow-ups

- Why does an octopus merge abort on conflicts?
- Why is reverting part of an octopus awkward?

An octopus merge combines more than two branches in a single merge commit (3+ parents). Git can do it, but only for conflict-free integrations.

```
git merge branch-a branch-b branch-c   # one merge commit with multiple parents
```

certs.eknathalabs.com

```
git log --graph                          # shows the multi-parent merge
```

Octopus merges are restricted to cases where the branches **do not conflict** — the default octopus strategy bails out if conflict resolution is needed, because resolving an N-way conflict is impractical. The legitimate use is integrating many independent, non-overlapping topic branches at once (historically used in some integration workflows). In practice they are rare and usually a smell.

Mostly avoid: octopus merges make history hard to read and bisect, and signal that you are batching integrations in a way that hides risk. Modern teams prefer merging PRs individually (often through a merge queue), which keeps each integration testable and revertible. Know it exists; rarely reach for it.

📚 **Start here (official docs)**

- Git documentation

Q105 How would you migrate a large legacy repository to a cleaner structure or remove bloat?

Advanced & internals

🎙 **Spoken answer · ~60 s**

Migrating a large legacy repository means first understanding what's wrong, then choosing a migration approach that minimises disruption. Assess: measure size and growth with git-sizer and git count-objects, and find the largest blobs and directories, long-lost binaries, vendored dependencies, generated files, committed secrets, and unwanted history. Define the goals: smaller clones, a restructured layout, split into several repositories, or merge into a monorepo. Tooling: git filter-repo is the recommended way to rewrite history: remove large files or paths, extract a subdirectory into its own repository with history using --subdirectory-filter or --path, rename directories, or rewrite author information with a mailmap; BFG is a simpler alternative for removing files and secrets. Move big binaries to LFS with git lfs migrate. To merge multiple repositories into a monorepo while preserving history, rewrite each into a subdirectory with filter-repo and merge with --allow-unrelated-histories, or use git subtree.

Likely follow-ups

- What does git filter-repo produce that helps after migration?
- What are the alternatives to rewriting history?

Rewriting large repos is done with git-filter-repo (filter-branch is deprecated and slow). The work is high-impact but disruptive, so plan the coordination.

```
pip install git-filter-repo
```

```
git filter-repo --strip-blobs-bigger-than 10M   # purge huge files from all history
```

```
git filter-repo --path keep/this/ --path also/this/   # keep only certain paths (extract a subdir)
```

certs.eknathalabs.com

```
git filter-repo --path-rename old/:new/          # restructure paths across history
```

```
git filter-repo --invert-paths --path secrets/   # remove a path entirely
```

```
# Then everyone re-clones; force-push the rewritten history
```

filter-repo can purge oversized blobs, extract a subdirectory into its own repo (preserving that path's history), rename/restructure paths across all of history, or remove sensitive files. Because it rewrites every commit SHA, the rollout is the hard part: communicate a freeze, do the rewrite, force-push, and have everyone re-clone (old clones and open PRs are invalidated).

Plan the cutover like a deploy: announce a window, freeze merges, rewrite, verify with `git fsck` and a diff of tip trees, force-push, then unfreeze. Half-migrated states (some forks on old history) are the failure mode — coordinate explicitly.

📚 **Official docs**

- PostgreSQL: Routine vacuuming

Q106 What causes "merge conflict" vs "semantic conflict," and how do you defend against each?

Merging & rebasing

🎙 **Spoken answer · ~61 s**

A textual merge conflict is when Git cannot automatically combine changes because two branches modified the same lines or one deleted what the other edited. Git detects it, stops, and asks a human. A semantic conflict is when the merge is textually clean, so Git succeeds without complaint, but the combined result is logically broken, for example one branch renames a function while another adds a new call to the old name, or two changes each valid separately violate an invariant together. Git cannot detect these, because it works on text, not meaning. Defences against textual conflicts: keep branches short-lived and merge or rebase frequently; make small, focused changes; communicate who is working on which areas, using CODEOWNERS and ownership; avoid sweeping reformatting mixed with logic changes, and run formatters before; modularise code so changes touch different files; use rerere, merge tools, and diff3 style for resolution.

Likely follow-ups

- Why can Git not detect semantic conflicts?
- How does a merge queue prevent them?

A textual merge conflict is when both sides edit the same lines — Git detects it. A semantic conflict is when both sides merge cleanly but the combined code is broken — Git cannot see it.

TEXTUAL CONFLICT

```
Both branches change the same region → Git inserts conflict markers
```

ceris.eknathalabs.com

```
Defense: small PRs, frequent integration, clear ownership of files
```

SEMANTIC CONFLICT (THE DANGEROUS ONE)

```
Branch A renames a function; Branch B adds a new call to the OLD name
```

```
Both merge cleanly (different lines) — but main no longer compiles/works
```

```
Defense: merge queue (test the COMBINED state), strong test suite, CI on main
```

Textual conflicts are visible and Git forces you to resolve them. Semantic conflicts are insidious precisely because each PR is green in isolation and they merge without markers — the breakage only appears in the integrated result. The structural defense is a **merge queue** that builds and tests the actual post-merge state, plus a comprehensive test suite that catches the integration failure.

Staff-level point: "passed CI but broke main" is almost always a semantic conflict. Naming the merge queue as the systemic fix (rather than "be more careful") shows you solve it at the platform level.

📚 **Official docs**

- git-merge

Q107 What exactly is HEAD, and what is a detached HEAD state?

Fundamentals

🎙 **Spoken answer · ~86 s**

HEAD is a special reference that identifies your current position in the repository: the commit that will become the parent of your next commit, and it determines what's checked out in your working tree. It normally contains the name of a branch, such as ref: refs/heads/main, a symbolic reference, so HEAD points to a branch, which points to a commit. When you commit, Git creates the commit and advances the branch that HEAD points to. Switching branches changes what HEAD refers to. Related references: HEAD~1 or HEAD^ is the first parent, HEAD~2 the grandparent, and HEAD@{1} is where HEAD was previously, from the reflog. Detached HEAD is the state in which HEAD contains a commit hash directly instead of a branch name. This happens when you check out a tag, a specific commit, or a remote-tracking branch such as origin/main, and also during bisect and in the middle of rebases. In this state you can look around and even commit, but new commits aren't on any branch; if you then switch away, those commits become unreachable, recoverable only through the reflog until garbage collection. Git warns you on entering it. To keep work done in a detached HEAD, create a branch there with git switch -c name.

Likely follow-ups

- What is the difference between HEAD~1 and HEAD@{1}?
- How do you keep commits made while detached?

certs.eknathalabs.com

HEAD is a reference to "where you are now" — normally a pointer to a branch name, which in turn points to a commit. Detached HEAD means HEAD points directly at a commit, not at a branch.

```
cat .git/HEAD                       # normally: ref: refs/heads/main
```

```
git checkout <sha>                  # detaches HEAD onto a specific commit
```

```
git checkout v2.3.0                 # checking out a tag also detaches
```

```
# In detached state, new commits belong to NO branch:
```

```
git switch -c rescue                # create a branch to keep detached-state commits
```

```
git switch -                        # return to previous branch
```

Normally HEAD → branch → commit, so committing advances the branch. In **detached HEAD**, HEAD → commit directly: you can look around and even commit, but those commits are not on any branch and become unreachable (garbage-collectable) once you leave — unless you create a branch to anchor them. It happens when you check out a SHA, tag, or remote-tracking ref directly.

The classic data-loss trap: making commits in detached HEAD, then switching away — the commits look gone. They are in the reflog for the grace period, so `git reflog` + `git branch rescue <sha>` recovers them. Always create a branch before committing in detached state.

📚 **Official docs**

- Git documentation

certs.eknathalabs.com

Q108 What is the difference between git switch/restore and git checkout?

Branching

🎙 Spoken answer · ~81 s

git checkout historically did several unrelated things, which made it confusing: switching branches, creating branches with -b, checking out a commit into a detached HEAD, and restoring files from the index or a commit. In Git 2.23, two newer commands split these responsibilities. git switch is for changing branches: git switch main, git switch -c new-branch to create and switch, git switch --detach <commit> for a deliberate detached HEAD, and git switch - to go back to the previous branch. It refuses to do file restores, so it's safer and clearer. git restore is for restoring file contents: git restore file discards working tree changes by taking the version from the index; git restore --staged file unstages; git restore --source=<commit> file brings a file from a specific commit into the working tree; and --staged --worktree restores both. Compared to checkout, which could ambiguously treat an argument as either a branch or a path, requiring '--' to disambiguate, these have clear semantics. checkout still works and remains in countless tutorials and scripts, so it's good to know both. Danger: git restore on working tree files permanently discards uncommitted changes, just as checkout -- file did.

Likely follow-ups

- What ambiguity did git checkout have?
- How do you unstage a file with the new commands?

`git checkout` was historically overloaded — it switched branches AND restored files, causing confusion and accidents. Git 2.23 split it into two focused commands.

```
# Switching branches:
git switch main                    # switch to an existing branch (clear intent)
git switch -c feature              # create + switch
git switch -                       # toggle to previous branch
# Restoring files:
git restore file.py                # discard working-tree changes to a file
git restore --staged file.py       # unstage (keep working-tree changes)
git restore --source=HEAD~2 file.py # restore file from an older commit
```

`switch` handles branch operations; `restore` handles file content operations. The old `git checkout file.py` silently discarded your changes with no undo — a notorious footgun — because the same verb also meant "switch branch." The split makes intent explicit and reduces accidental data loss. `checkout` still works for backward compatibility.

certs.eknathalabs.com

Why it matters: recommending `switch`/`restore` shows you keep current and care about reducing destructive ambiguity — a small but real signal of modern Git fluency.

📚 **Official docs**

- Git documentation

Q109 Explain the staging area (index) and why it exists.

Fundamentals

🎙 **Spoken answer · ~85 s**

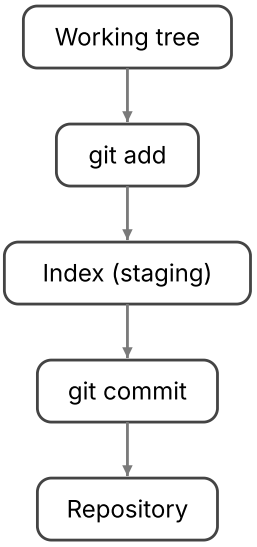
The staging area, or index, is a snapshot of what will go into your next commit. It sits between the working tree and the repository: you modify files, add the ones you want with git add, and commit the staged snapshot. Technically it's a binary file, .git/index, listing paths, file modes, blob hashes, and metadata for fast change detection. Why it exists. Crafting commits: you can stage only some files, or even individual hunks with git add -p, so each commit is a logical, reviewable unit even when your working tree contains several unrelated changes. It enables clean history without forcing you to work in tiny steps. Review before committing: git diff --staged shows exactly what you're about to commit. Merge support: during a merge, the index holds up to three versions of conflicted files, the base, ours, and theirs, in stages one to three, and tracks which conflicts are resolved. Performance: stat information in the index lets Git quickly detect which files changed without hashing everything. It also provides a place to build the tree object for the commit. Contrast with systems that commit all working changes directly. Some find the extra step confusing, and git commit -a stages tracked modifications automatically.

Likely follow-ups

- What do index stages one, two, and three hold during a merge?
- How does git add -p help build clean commits?

certs.eknathalabs.com

🗺 **Three areas**



The index (staging area) is an intermediate layer between your working tree and the repository — it holds the proposed contents of the next commit, letting you compose commits deliberately.

```
git add file.py                  # stage whole file
```

```
git add -p                       # stage selected hunks interactively
```

```
git status                       # staged vs unstaged vs untracked
```

```
git diff                         # working tree vs index (unstaged changes)
```

```
git diff --staged                # index vs HEAD (what will be committed)
```

```
git restore --staged file.py     # unstage
```

```
cat .git/index                   # binary index file (not human-readable)
```

The index lets you craft a commit that is a subset of your working changes — you can have edits in ten files but stage only the three that form one logical change, using `git add -p` to even select specific hunks. This is what enables clean, single-purpose commits. The two-stage model (working tree → index → commit) is core to Git and surprises newcomers from systems that commit the whole working state.

Interview depth: explaining `git diff` (tree vs index) vs `git diff --staged` (index vs HEAD) shows you truly understand the three states. The index is also why `reset --soft/--mixed/--hard` behave differently.

📚 **Official docs**

- PostgreSQL: Using EXPLAIN

certs.eknathalabs.com

Q110 How do remote-tracking branches work, and what is the relationship to local branches?

Branching

🎙 Spoken answer · ~87 s

A remote-tracking branch, such as origin/main, is a local, read-only reference that records where a branch on a remote was at the time of your last communication with it. Stored under refs/remotes/origin/, it's updated by git fetch, git pull, and git push, never by commits you make locally, and it lets you see what the remote has without network access, and compare with your own work using commands like git log main..origin/main. A local branch is your own editable branch that moves when you commit. They are related through the upstream configuration: a local branch can be set to track a remote-tracking branch, recorded in .git/config as branch.<name>.remote and branch.<name>.merge. Tracking gives git status its 'ahead by 2, behind by 1' output, and allows git pull and git push without arguments to know where to go. Creating a local branch from a remote one, for example git switch feature when origin/feature exists, sets up tracking automatically; git push -u origin branch sets upstream on the first push, and push.autoSetupRemote true automates that. Key idea: origin/main is only your last known copy of the remote; to refresh, you fetch. Stale remote-tracking branches for branches deleted on the remote remain until you run git fetch --prune, or set fetch.prune true.

Likely follow-ups

- What updates a remote-tracking branch?
- What does git fetch --prune do?

Remote-tracking branches (e.g. `origin/main`) are local, read-only bookmarks of where a remote branch was at your last fetch. They are not the remote, and not editable directly.

```
git branch -vv                      # show local branches + their upstream + ahead/behind
```

```
git branch --set-upstream-to=origin/main main
```

```
git fetch                           # updates origin/* tracking refs (not your branches)
```

```
git status                          # 'ahead 2, behind 1' compares to tracking branch
```

```
git push -u origin feature          # set upstream while first pushing
```

`origin/main` is a cache: `git fetch` updates it to the remote's latest, but your local `main` moves only when you merge/rebase or pull. The "ahead/behind" in `git status` compares your branch to its upstream tracking branch. This separation is deliberate — fetch is safe (only updates tracking refs), while integrating remote work into your branch is a separate, explicit step you control.

Mental model: three layers — the remote (server), `origin/*` (your last-known snapshot of it), and your local branches (your work). `fetch` syncs the middle layer; merge/rebase/pull bridges middle to local. This explains why fetch never surprises you with changed files.

certs.eknathalabs.com

📚 **Official docs**

- Git documentation

Q111 What is the difference between .gitignore, .git/info/exclude, and assume-unchanged?

Fundamentals

🎙 **Spoken answer · ~86 s**

These three mechanisms all stop Git from paying attention to files, but they work in different ways and have different intent. .gitignore is a versioned file, committed to the repository, listing patterns of untracked files that Git should ignore, such as build outputs, dependency folders, logs, and environment files. It's shared by the whole team, and nested .gitignore files apply to subdirectories. It only affects untracked files: a file already tracked continues to be tracked even if it matches a pattern, and must be removed from the index with git rm --cached. .git/info/exclude works the same as .gitignore but is local to your clone and not versioned or shared, ideal for personal files such as editor configurations, scratch notes, or local tooling that other developers don't have, and which don't belong in the project's shared ignore file. A global ignore file, set via core.excludesFile, serves personal patterns across all your repositories. assume-unchanged, git update-index --assume-unchanged file, is different: it tells Git to assume that a tracked file hasn't changed and skip checking it, as a performance hint for huge files. It is not a reliable way to ignore local modifications to tracked files, since Git may overwrite them during checkout or merge and silently lose your changes.

Likely follow-ups

- Why does .gitignore not stop tracking an already tracked file?
- What is skip-worktree for?

Three mechanisms to not track files, differing in scope and intent: shared, local-only, and "ignore changes to a tracked file."

```
.gitignore                          # committed, shared with the team (project-wide)
```

```
.git/info/exclude                   # local-only ignores, NOT committed (personal)
```

```
~/.config/git/ignore                # global ignores for all your repos
```

```
# For ALREADY-tracked files (.gitignore won't ignore those):
```

```
git rm --cached file                # stop tracking, keep on disk
```

```
git update-index --skip-worktree config.local   # ignore local changes to a tracked file
```

```
git update-index --assume-unchanged big.bin      # perf hint: assume file unchanged
```

certs.eknathalabs.com

`.gitignore` is committed and shared — for build artifacts, deps, OS files everyone should ignore. `.git/info/exclude` is your private, uncommitted ignore list. Critically, **.gitignore only affects untracked files** — to stop tracking something already committed you must `git rm --cached` it. `--skip-worktree` is the right tool for "keep this tracked file but ignore my local edits" (e.g. a config template you tweak locally); `--assume-unchanged` is only a performance hint, not a reliable ignore.

Common confusion: adding a file to .gitignore after it is already tracked does nothing — Git keeps tracking it. You must untrack with `git rm --cached` first. And do not use `--assume-unchanged` to hide local config edits; it is unreliable — use `--skip-worktree`.

📚 **Official docs**

- Git documentation

Q112 How would you design a branch naming and lifecycle convention for a large team? Branching

🎙 **Spoken answer · ~87 s**

A branch convention should make purpose, ownership, and lifecycle obvious and be enforceable by tooling. Naming: use prefixes by type, such as feature/, fix/ or bugfix/, hotfix/, release/, chore/, and docs/, followed by a ticket ID and a short description in lowercase with hyphens, for example feature/PAY-1234-add-refund-endpoint. Including the ticket ID ties code to work tracking and enables automation, such as linking and status updates. Avoid personal names, uppercase and special characters, spaces, and overly long names. Long-lived branches should be few and protected: main, plus release/x.y for supported versions if needed. Lifecycle: branches are created from an up-to-date main, short-lived, ideally merged within a day or two, with small pull requests; merged branches are deleted automatically on merge using the platform setting; stale branches with no activity after a defined period, such as thirty days, are flagged by a bot and then removed, after notifying the owner. Rules: protected branches enforce reviews and checks; release branches are cut only when needed and receive only fixes via cherry-picks or backports; hotfixes branch from the production tag. Enforce naming with a branch name check in CI or push rulesets on the platform, which can reject non-conforming branch names. Document the policy in CONTRIBUTING. Use tags, not branches, for releases.

Likely follow-ups

- Why include the ticket ID in the branch name?
- How do you clean up stale branches?

Conventions reduce cognitive load and enable automation. Encode type, scope, and traceability in the name, and define a clear lifecycle.

NAMING PATTERN

```
feature/JIRA-123-short-description
```

certs.eknathalabs.com

```
fix/JIRA-456-null-pointer
```

```
hotfix/CVE-2025-xxxx
```

```
release/2.4.0
```

```
# type/ticket-slug — parseable by automation, traceable to a ticket
```

LIFECYCLE

```
Short-lived: branch → PR → review → merge → auto-delete on merge
```

```
Auto-delete merged branches; flag/clean stale branches (>30 days) periodically
```

```
Branch → environment mapping handled by tags/artifacts, not long-lived branches
```

A `type/ticket-slug` convention makes branches self-documenting and machine-parseable (CI can route by type, link to tickets, derive changelogs). The lifecycle should favor **short-lived** branches with auto-deletion on merge, and a periodic sweep of stale branches — a repo with hundreds of dead branches is a navigation and tooling burden. Tie versioning to tags, not to permanent environment branches.

Platform framing: branch hygiene (auto-delete, stale cleanup, parseable names) is a small thing that compounds at scale. Owning it as policy + automation is the platform-engineering instinct interviewers look for.

📚 **Start here (official docs)**

- Git documentation

certs.eknathalabs.com

Q113 Explain the difference between author and committer on a commit.

Fundamentals

🎙 Spoken answer · ~83 s

Every Git commit records two identities with timestamps: the author and the committer. The author is the person who originally wrote the change; the committer is the person who applied it to the repository, that is, created this specific commit object. Often they are the same, but they differ when the commit is rewritten or applied by someone else: after git commit --amend or a rebase, the committer and committer date update to the person and time of the rewrite, while the author and author date stay as the original; when a maintainer applies a contributor's patch with git am, the contributor is the author and the maintainer the committer; cherry-picking preserves the author and sets you as the committer; and squash merges on platforms record the PR author as author, and the platform or merger as committer. Why it matters: credit and attribution stay with the original author through history rewrites; git log shows the author by default, with --pretty=fuller showing both; blame shows the author. For auditing and verification, note that both fields are just unauthenticated text set by configuration, user.name and user.email, or environment variables, so they can be forged; only signed commits give trustworthy identity.

Likely follow-ups

- Which of the two changes after a rebase?
- Why can author information be forged?

Git records two identities per commit: the **author** (who wrote the change) and the **committer** (who created this commit object). They are usually the same but diverge after history rewriting.

```
git log --pretty=fuller          # shows both Author/AuthorDate and Commit/CommitDate
```

```
git show --format=fuller <sha>
```

```
# When they differ:
```

```
#   rebase — committer becomes the rebaser, author stays original
```

```
#   cherry-pick — same: author preserved, committer is you
```

```
#   applying a patch / PR on someone's behalf
```

When you rebase, cherry-pick, amend, or apply someone else's patch, Git preserves the original **author** (and author date) but sets the **committer** to you with a new commit date. This is why a rebased branch shows old author dates but new commit dates. It matters for attribution (the author gets credit) and for audit (the committer shows who actually rewrote/applied it).

Audit relevance: `git log --pretty=fuller` distinguishing the two is useful in forensics — e.g. spotting that a commit was authored long ago but committed (rebased/applied) recently by someone else. Both fields are self-reported, hence the need for signing.

certs.eknathalabs.com

📚 **Official docs**

- Git documentation

Q114 What is a tracking branch vs an upstream branch, and how do you set them? Branching

🎙 **Spoken answer · ~88 s**

The terms overlap. A tracking branch, in the older terminology, is a local branch that is configured to track a remote branch: it has an upstream. The upstream branch is the remote branch that a local branch is linked to, recorded in config as branch.<name>.remote and branch.<name>.merge, and it determines the default targets for git pull, git push without arguments, and the ahead and behind counts in git status. The upstream is typically a remote-tracking ref such as origin/feature. Do not confuse these with remote-tracking branches, which are the local read-only copies like origin/feature that record the remote's state. Ways to set the upstream: when creating a local branch from a remote one, git switch feature or git checkout --track origin/feature sets it automatically; git branch --set-upstream-to=origin/feature sets it for the current branch (-u is a shortcut); git push -u origin feature pushes and sets upstream in one step; and push.autoSetupRemote true makes the first push set it up automatically. Inspect with git branch -vv, which shows each local branch, its upstream, and ahead and behind counts, and git rev-parse --abbrev-ref @{upstream}. Remove with git branch --unset-upstream. Also, push.default settings, with simple as the default, control what a plain git push does, usually pushing the current branch to the same-named upstream.

Likely follow-ups

- How do you see each branch's upstream and ahead/behind counts?
- What does push -u do?

An upstream (tracking) relationship links a local branch to a remote-tracking branch so Git knows what to compare against and where to push/pull by default.

```
git push -u origin feature          # push AND set origin/feature as upstream
```

```
git branch --set-upstream-to=origin/main main
```

```
git branch -vv                      # see each local branch's upstream + ahead/behind
```

```
git status                          # uses upstream to report ahead/behind
```

```
# Without upstream set, bare 'git push'/'git pull' may error or need full args
```

Setting an upstream lets you run bare `git push` / `git pull` (Git knows the target), and powers the "ahead 3, behind 1" status. `-u` on first push is the common way to establish it. `git branch -vv` shows the full picture: local branch, its upstream, and divergence. Without an upstream, Git relies on `push.default` config and may require explicit `origin branch` arguments.

certs.eknathalabs.com

Config to know: `push.default = simple` (the modern default) pushes the current branch to its upstream of the same name — safe behavior. Understanding upstreams explains a lot of "why did push go there / why this ahead-behind" confusion.

📚 **Official docs**

- Git documentation

Q115 **How does git diff work, and what are the most useful diff options in practice?** Fundamentals

🎙 **Spoken answer · ~87 s**

git diff shows differences between states of the repository. It computes the changes between two trees or between a tree and the working directory, using a diff algorithm, by default Myers, to find a minimal set of line insertions and deletions, grouped into hunks with context lines. The forms to know: git diff compares the working tree with the index, the unstaged changes; git diff --staged compares the index with HEAD, what you are about to commit; git diff HEAD compares the working tree with the last commit; git diff A B compares two commits, branches, or tags; git diff A...B, three dots, compares B with the merge base of A and B, showing what B introduced, which is what a pull request shows; and git diff A..B is the same as two arguments. Useful options. --stat for a summary of files and line counts, --name-only and --name-status for lists of paths, -w or --ignore-all-space to ignore whitespace noise, --word-diff or --color-words for prose, --diff-algorithm=histogram or patience for more readable diffs on code with moves, -M and -C to detect renames and copies, --check to find whitespace errors, -U<n> to change context size, -S and -G to search for changes, and a path after -- to limit the scope.

Likely follow-ups

- What is the difference between two-dot and three-dot diff?
- Which option helps ignore whitespace noise?

Git computes diffs on demand (it stores snapshots, not diffs) by comparing trees. The comparison endpoints depend on the flags.

```
git diff                          # working tree vs index (unstaged)
```

```
git diff --staged                 # index vs HEAD (what's about to commit)
```

```
git diff HEAD                     # working tree vs last commit (all changes)
```

```
git diff main..feature            # tip-to-tip difference
```

```
git diff main...feature           # changes on feature SINCE it diverged (merge-base)
```

```
git diff -w                       # ignore whitespace
```

certs.eknathalabs.com

```
git diff --stat                    # summary: files + insertions/deletions
```

```
git diff --color-moved             # distinguish moved code from added/deleted
```

The endpoints matter: no args = unstaged, `--staged` = staged, `HEAD` = everything since last commit. The two-dot vs three-dot distinction is a frequent interview point: `main..feature` is a raw tip comparison, while `main...feature` shows what feature added since branching off the merge base — usually what you want for "what does this PR change." `--color-moved` and `-w` make reviews of refactors far clearer.

Underused gem: `git diff --color-moved` highlights relocated code differently from genuine changes — invaluable when reviewing a refactor that moved blocks around. Mentioning two-dot vs three-dot semantics signals real fluency.

📚 **Official docs**

- Git documentation

Q116 How do you clean up local and remote branches safely at scale?

Branching

🎙 **Spoken answer · ~75 s**

Cleaning up branches safely means removing only those that are merged or confirmed abandoned, with a safety net. Local cleanup: first refresh and prune remote-tracking branches that no longer exist: git fetch --prune, or set fetch.prune true. List branches fully merged into main with git branch --merged main, excluding main and other protected names, then delete with git branch -d, which refuses to delete unmerged branches, unlike -D, which forces. A common one-liner pipes merged branches through grep to exclude protected ones into git branch -d. For squash-merged branches, git doesn't see them as merged, so use git branch -vv to find ones whose upstream is gone, and verify before force-deleting; helper tools like git-delete-squashed or checking against the PR state help. Remote cleanup at scale: enable the platform setting to automatically delete head branches after merge; use scripts or tooling via the API to find branches with no commits for a threshold such as ninety days and no open pull request, notify the owners and then delete after a grace period; skip protected and release branches by pattern.

Likely follow-ups

- Why is git branch -d safer than -D?
- How do you handle squash-merged branches?

Stale branches accumulate and clutter tooling. Clean them with awareness of what is merged and what is gone from the remote.

```
git fetch --prune                  # remove local origin/* refs for deleted remote
branches
```

certs.eknathalabs.com

```
git branch --merged main           # local branches already merged into main (safe to
delete)
```

```
git branch -d feature              # delete merged branch (refuses if unmerged)
```

```
git branch -D feature              # force-delete (use with care)
```

```
git branch --no-merged main        # branches NOT yet merged (review before deleting)
```

```
# Find stale branches by last-commit date:
```

```
git for-each-ref --sort=committerdate refs/heads/ --format='%(committerdate:short) %
(refname:short)'
```

```
git push origin --delete old-branch # delete a remote branch
```

`git fetch --prune` clears local tracking refs for branches deleted on the remote (otherwise `origin/*` accumulates ghosts). `--merged`/`--no-merged` separate safe deletions from ones needing review. The lowercase `-d` is safe (refuses unmerged branches); `-D` forces and can lose work. `git for-each-ref` sorted by date finds the truly stale ones programmatically.

> **Automate it:** enable "auto-delete head branches on merge" in the platform, run a periodic job listing branches older than N days with no open PR, and prune tracking refs on fetch. Branch hygiene as automation, not manual chores.

> 📚 **Official docs**
>
> - Git documentation

certs.eknathalabs.com

Q117 What happens, object by object, when you make a commit?

Fundamentals

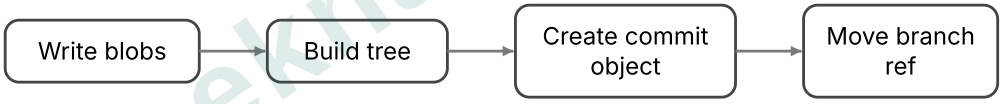
🎙 Spoken answer · ~88 s

When you run git commit, a series of objects are created. First, the index, the staging area, already contains the paths and blob hashes of the files to be committed. When you git add a file, Git hashes its content and writes a blob object, if one with that hash doesn't exist yet, and records it in the index. On commit, Git converts the index into tree objects: it builds a tree for each directory, bottom-up, with entries listing mode, name, and hash of blobs and subtrees; unchanged directories reuse existing tree objects, since identical content has identical hashes. The root tree's hash represents the whole snapshot. Next, Git creates a commit object containing the root tree hash, the parent commit hash, which is the current HEAD commit, or two for a merge, the author and committer identities with timestamps, and the commit message. This is hashed, giving the commit ID, and written to the object database as a loose object, zlib-compressed. Finally, Git updates the reference: the current branch file, such as refs/heads/main, is changed to the new commit's hash, and HEAD, which points at that branch, therefore points to the new commit; an entry is added to the reflog. Hooks run before and after: pre-commit, commit-msg, and post-commit.

Likely follow-ups

- Why are unchanged directories not duplicated?
- What does the commit object contain?

🗺 Making a commit



A commit triggers a cascade of object creation in the content-addressed store. Understanding it demystifies why commits are snapshots and why SHAs change on rewrite.

THE SEQUENCE

```
1. git add  → blob objects written for changed file contents (hashed by content)
```

```
2. git commit → tree objects built from the index (directory → blob/tree mapping)
```

```
3. a commit object is written: points to the root tree + parent commit(s) + author/msg
```

```
4. the current branch ref is updated to point at the new commit SHA
```

```
5. HEAD (via the branch) now resolves to the new commit
```

Each step produces immutable, SHA-addressed objects: blobs (file contents), trees (directory snapshots referencing blobs/subtrees), and a commit (root tree + parents + metadata). The commit is a full **snapshot** (via the tree), not a diff. Only the branch ref — a tiny file in `.git/refs/heads/` — is mutated to point at the

certs.eknathalabs.com

new commit. This is why amending or rebasing creates entirely new objects with new SHAs: you cannot edit an immutable object, only create a replacement and move the ref.

The payoff insight: "a branch is just a movable pointer; commits are immutable snapshots." Explaining the blob→tree→commit→ref chain is the single clearest demonstration that you understand Git's data model rather than just its commands.

📚 **Official docs**

- Git documentation

Q118 **Explain the tradeoffs of rebasing vs merging for keeping a feature branch current.** Branching

🎙 **Spoken answer · ~77 s**

To keep a feature branch current with main, you can merge main into it or rebase it onto main. Merging main in creates a merge commit on the feature branch, preserving the actual history and never rewriting commits. It's safe for shared branches, since nobody's commits change, and conflicts are resolved once, in the merge commit. The downside is a noisy history with repeated merge commits from main inside the branch, which makes the branch's own changes harder to review and bisect, and the pull request's commit list can be cluttered. Rebasing replays the branch's commits on top of the latest main, creating new commits, so the history is linear and reads as if the work was done on current main. It produces a clean pull request and simplifies bisect and log. The downsides: it rewrites history, so commits get new hashes and a force push is needed; it's dangerous on branches shared with others, because their copies diverge, causing duplication and confusion; conflicts may need resolving repeatedly, once per replayed commit, though rerere helps; and it loses the record of when the integration happened.

Likely follow-ups

- When is merging main into the feature branch preferable?
- What does rebasing do to commit hashes?

certs.eknathalabs.com

Both bring main's latest into your feature branch; they differ in resulting history and safety. The right choice depends on whether the branch is shared.

REBASE ONTO MAIN

```
git rebase main                    # replay your commits on top of latest main
```

```
+ Linear, clean history; no merge-bubble clutter
```

```
+ Your changes read as if written against current main
```

```
- Rewrites SHAs — unsafe if others share the branch
```

```
- Must resolve conflicts per-commit; force-push needed after
```

MERGE MAIN IN

```
git merge main                    # bring main into feature via a merge commit
```

```
+ Non-destructive; safe on shared branches; resolve conflicts once
```

```
- Adds merge commits; busier, non-linear history
```

Rebase gives a clean, linear history and is ideal for a *private* feature branch you will open as a PR — reviewers see a tidy series against current main. **Merge** is non-destructive and the safe choice when others share the branch, at the cost of merge bubbles. The cardinal rule: never rebase commits that others have already pulled (the "golden rule of rebasing") — it rewrites shared history and forces everyone to recover.

Golden rule of rebasing: only rebase commits that exist solely in your local/private branch. Once pushed and pulled by others, integrate with merge instead. Violating this is the most common cause of team-wide Git chaos.

📚 **Official docs**

- git-rebase

Q119 How do tags differ from branches, and lightweight vs annotated tags?

Fundamentals

🎙 **Spoken answer · ~73 s**

Branches and tags are both references to commits, but with different semantics. A branch is a movable pointer: when you commit while on it, the pointer advances. It represents an ongoing line of development. A tag is intended to be a fixed label on a specific commit, marking a point in history, usually a release such as v1.4.0; it doesn't move when you commit. Tags live under refs/tags and, unlike branches, aren't normally checked out for work; checking one out gives a detached HEAD. Lightweight tags are simply a ref to a commit, like a branch that never moves, with no extra metadata. Annotated tags are full Git objects, created with git tag -a v1.4.0 -m 'message', that store the tagger's name, email, date, a message, and optionally a GPG or SSH signature, and they point to the commit. Annotated tags are the right choice for releases because they carry who, when, and why, can be signed and verified, and are shown in git describe by default. Lightweight tags suit temporary or private markers.

Likely follow-ups

- Why use annotated tags for releases?
- Are tags pushed automatically?

Both are refs pointing at commits, but a branch **moves** as you commit while a tag is a **fixed** marker. Tags come in two flavors with different metadata and trust properties.

certs.eknathalabs.com

```
git tag v2.4.0                    # LIGHTWEIGHT: just a name → commit pointer
```

```
git tag -a v2.4.0 -m 'Release 2.4.0' # ANNOTATED: a real tag OBJECT (tagger, date, msg)
```

```
git tag -s v2.4.0                 # signed annotated tag (GPG/SSH)
```

```
git push origin v2.4.0            # tags are NOT pushed by default
```

```
git push origin --tags            # push all tags
```

```
git describe --tags               # nearest tag + distance (version strings)
```

A **branch** ref advances with each commit; a **tag** ref stays put — it marks a specific point (a release). **Lightweight** tags are just a name pointing at a commit (like a branch that never moves). **Annotated** tags are full objects storing the tagger, date, message, and an optional signature — and they are what you should use for releases, because they carry provenance and can be verified. Remember tags are not pushed automatically.

Always annotate releases: annotated (ideally signed) tags give you an auditable, verifiable release marker; lightweight tags are fine for private/local bookmarks. `git describe` turning tags into deterministic version strings is the basis of many CI versioning schemes.

📚 **Official docs**

- Git documentation

certs.eknathalabs.com

Q120 Capstone: design a complete Git strategy for a 50-engineer SaaS deploying many times a day.

Branching

🎙 Spoken answer · ~78 s

For a 50-engineer SaaS deploying many times a day, I'd choose trunk-based development with strong automation. Branching: main is the single long-lived branch, always deployable; engineers create short-lived topic branches, merged within a day, with pull requests of a few hundred lines; unfinished work is hidden behind feature flags; release branches only if a customer-facing version needs long-term support. Protection: rulesets on main requiring pull requests, one or two approvals, code owner review for sensitive paths, required status checks, a merge queue to keep main green, signed commits, no force pushes, linear history through squash merges with PR-title conventions. Quality gates: fast CI under ten minutes with caching and test sharding, linting, unit and integration tests, secret scanning with push protection, dependency and container scans, and pre-commit hooks for early feedback. Delivery: every merge to main builds an immutable artefact tagged with the commit SHA, deploys automatically to staging, runs smoke tests, then progressively to production using canary or blue-green with automated rollback; GitOps for configuration with environment folders; database migrations as backward-compatible, separate steps. Releases: semantic version tags generated from Conventional Commits, with automated changelogs.

Likely follow-ups

- Why a merge queue at this team size and cadence?
- How does a hotfix work in this model?

Tie together branching model, integration, deploy mapping, safety, and governance into one coherent system suited to high-velocity SaaS.

BRANCHING & INTEGRATION

```
Trunk-based: short-lived branches off main, merged within a day
```

```
Feature flags decouple deploy from release (ship dark, enable gradually)
```

```
Squash-merge to main: one clean, revertible commit per PR
```

```
Merge queue: serialize merges, test the combined state, keep main always green
```

DEPLOY & ROLLBACK

```
Build artifact ONCE on merge; promote same artifact: staging → (tag) → prod canary
```

```
GitOps (Argo/Flux) with a config repo as source of truth; rollback = git revert
```

```
Progressive delivery with auto-abort on SLO/metric regression
```

SAFETY & GOVERNANCE

certs.eknathalabs.com

```
Branch protection: required green checks, CODEOWNERS review, no force-push to main
```

```
Signed commits; secret scanning + push protection; pre-commit hooks shipped in-repo
```

```
OIDC (no long-lived PATs) for CI → cloud; access managed as code (Terraform)
```

```
Auto-delete merged branches; track PR cycle time as a health metric
```

The through-line: optimize for **small batches integrated continuously and shipped safely**. Trunk-based + feature flags + a merge queue keeps main releasable at high volume. Build-once/promote with GitOps gives traceable deploys and one-command (revert) rollback. Branch protection, signed commits, secret scanning, and OIDC enforce safety without slowing people down. Every guardrail is automated so contributing is easy and mistakes are caught by tooling, not heroics.

What ties it together: Git becomes the audit log and control plane — every change to code, config, and access is a reviewed, signed, traceable commit. That is the platform-engineering vision: a system where the safe path is the easy path, and the whole pipeline is driven and recorded by Git.

📚 **Official docs**

- Git documentation

Q121 **What is the difference between git restore, git reset and git checkout -- file for discarding changes?** Fundamentals

🎙 **Spoken answer · ~58 s**

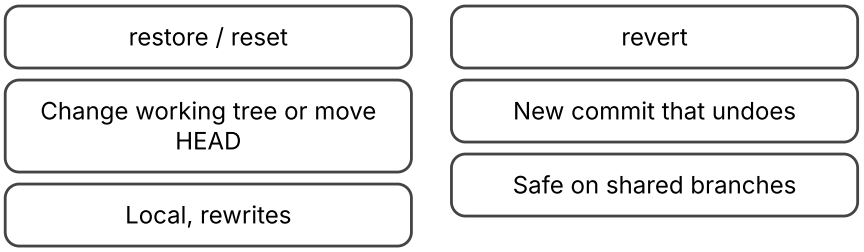
Git has three trees: the working directory, the index, and HEAD. The restore command was introduced to separate file-level operations from branch-level ones. Running restore on a path copies the version from the index into the working tree, discarding unstaged edits. Adding the staged option copies from HEAD into the index, which unstages the file without touching your edits. The source option lets me pull a file from any commit. The reset command, by contrast, mainly moves the current branch pointer, and its mixed default also resets the index. Checkout with double dash and a path does the same as restore but sits in a command that also switches branches, which caused many accidents. Discarding working tree changes is irreversible because they were never committed, so I check git diff first, or stash instead if I might want them back.

Likely follow-ups

- Which of these can lose data permanently?
- Why was restore introduced?

courses.eknathalabs.com

🗺 **restore vs reset vs revert**



restore is the modern, single-purpose command for files; reset moves branch pointers and optionally the index; checkout -- file is the older overloaded form of restore. Know which tree each one touches.

```
git restore file.txt
```

```
git restore --staged file.txt
```

```
git restore --source=HEAD~2 file.txt
```

```
git reset HEAD file.txt
```

Interview tip: restore without flags only touches the working tree. Add --staged to touch the index.

📚 **Official docs**

- git-reset
- Git documentation

certs.eknathalabs.com

Q122 What is the difference between git config at system, global and local levels, and how do includeIf rules help?

Fundamentals

🎙 **Spoken answer · ~60 s**

Git reads configuration from several files in order: system-wide, the user's global file, then the repository's own config, and optionally per-worktree. A narrower scope wins, so a local setting overrides a global one. I use local config for things that belong to a project, such as hooks path or line-ending rules, and global config for personal preferences like editor and aliases. A common production problem is committing with the wrong email, for example a personal address in a work repo. The includeIf directive solves this by loading an extra config file only when the repository lives under a given directory, so everything under my work folder automatically uses the work identity and signing key. When behaviour is unexpected, I run the list command with the show-origin flag, which prints the file and line that supplied every value, and that usually reveals the cause quickly.

Likely follow-ups

- How do you fix commits with the wrong author?
- What is a worktree config?

Config is layered: system, global (user), local (repo), plus worktree. Later levels override earlier ones. includeIf loads different identity or settings depending on the repo path.

```
git config --list --show-origin
```

```
git config --global user.email me@example.com
```

```
git config --local core.hooksPath .githooks
```

```
[includeIf "gitdir:~/work/"] path = ~/.gitconfig-work
```

Interview tip: Use show-origin to debug where a surprising setting comes from.

📚 **Official docs**

- Git documentation

certs.eknathalabs.com

Q123 What does git status show beyond changed files, and how do you read ahead/behind information?

Fundamentals

🎙 Spoken answer · ~62 s

Status is more than a file list. The header tells me which branch I am on and how it relates to its upstream, for example ahead by two and behind by one, which means I have local commits to push and remote commits to integrate, and a diverged state needs a rebase or merge. It also reports when I am in the middle of a merge, rebase, cherry-pick or bisect, and lists the file categories separately: staged for the next commit, modified but unstaged, and untracked. The short form with the branch flag gives a compact view I use daily. For scripts and prompts I use the porcelain format, which is designed to remain stable, rather than parsing human text. To see divergence numerically, I use rev-list with left-right and count between the upstream and HEAD. Reading these signals before pushing prevents most rejected pushes and surprise merges.

Likely follow-ups

- What does diverged mean?
- Why use porcelain output?

Status shows the branch, its upstream, ahead/behind counts, staged and unstaged changes, untracked files and in-progress operations like a rebase or merge. Short and branch flags make it scriptable.

```
git status -sb
```

```
git rev-list --left-right --count origin/main...HEAD
```

```
git status --porcelain=v2 --branch
```

```
git for-each-ref --format='%(refname:short) %(upstream:track)' refs/heads
```

Interview tip: Porcelain output is stable across versions, so scripts should use it instead of parsing the human format.

📚 Official docs

- Git documentation

certs.eknathalabs.com

Q124 How do you use git add -p to craft clean commits from messy working changes?

Fundamentals

🎙 Spoken answer · ~62 s

When I have been working for a while, my working tree often contains a bug fix, a refactor and a debugging print mixed together. Committing everything at once makes review and later bisecting harder. Patch mode walks through each hunk of change and asks whether to stage it. I answer yes or no, use the split option when a hunk contains two unrelated edits, and the edit option when I need to hand-trim a hunk. The result is a series of small commits that each do one thing and pass tests. After staging, I inspect the staged diff to verify it contains only what I intend, and run tests before committing. The same interactive approach exists for restore and stash, which lets me discard or stash only part of my changes. Clean commits make code review faster, make revert safe, and make bisect point to a precise cause.

Likely follow-ups

- How do you split a hunk further?
- Why do small commits help bisect?

Patch mode stages individual hunks, letting you split unrelated edits into separate logical commits. Use s to split hunks and e to edit them manually.

```
git add -p
```

```
git add -p src/api.py
```

```
git diff --staged
```

```
git restore -p file.txt
```

Interview tip: Review with git diff --staged before committing, so you know exactly what is in the commit.

📚 Official docs

- Google SRE Book: Managing Critical State (consensus)
- Git documentation

certs.eknathalabs.com

Q125 What are Git's merge-base and how do triple-dot and double-dot ranges differ?

Fundamentals

🎙 Spoken answer · ~61 s

The merge base is the most recent commit that two branches share, and it is the starting point for a three-way merge and for judging what each branch changed. Range syntax builds on it. Double dot in log means commits reachable from the right side that are not reachable from the left, so main dot dot feature lists what the feature adds. Triple dot in log shows the symmetric difference: commits on either side but not both, and with the left-right flag it marks which side each came from, which is useful to see divergence. In diff the meaning changes: triple dot compares the right side against the merge base, which shows only what the branch introduced, even if main has moved on. That is exactly what a reviewer wants for a pull request. Mixing up the two forms is a classic source of confusing output.

Likely follow-ups

- Why does diff triple-dot differ from log?
- What is a symmetric difference?

merge-base is the best common ancestor of two commits. A..B means commits reachable from B but not A; A...B in log means those in either but not both; in diff, A...B compares B to the merge-base.

```
git merge-base main feature
```

```
git log main..feature --oneline
```

```
git log main...feature --left-right --oneline
```

```
git diff main...feature
```

Interview tip: For reviewing a PR, diff with triple dots shows only the branch's own changes, ignoring main's newer commits.

📚 Official docs

- Cloudflare Learning: DNS security
- Git documentation

certs.eknathalabs.com

Q126 How does git switch -c and git branch --set-upstream-to differ from push -u when creating feature branches?

Branching

🎙 **Spoken answer · ~58 s**

Creating a branch is purely local: switch with the create flag makes a new branch at the current commit and moves HEAD to it. The remote does not know about it until I push. A tracking relationship stores which remote branch is the counterpart, so status can show ahead and behind counts, and pull and push work without arguments. I normally establish it with push and the set-upstream flag the first time I publish, which pushes and records the link in one step. If the branch already exists on the remote, I can attach it afterwards with the set-upstream-to option on the branch command. To avoid forgetting, I enable the autoSetupRemote setting so the first push sets tracking automatically. Naming matters as well: a convention with a type prefix and ticket ID makes branches easy to filter and automate.

Likely follow-ups

- What does push.default control?
- How do you see tracking info for all branches?

switch -c creates and checks out a local branch. Upstream tracking links it to a remote branch so status, pull and push know the counterpart. push -u does the push and sets tracking in one step.

```
git switch -c feature/login
```

```
git push -u origin feature/login
```

```
git branch --set-upstream-to=origin/feature/login
```

```
git config push.autoSetupRemote true
```

Interview tip: push.autoSetupRemote removes the need for -u on the first push.

📚 **Official docs**

- Atlassian: Gitflow workflow
- Git documentation

certs.eknathalabs.com

Q127 What is a release branch strategy and how do you backport a fix to several maintained versions?

Branching

🎙 Spoken answer · ~62 s

Teams that support several versions at once keep a long-lived release branch per supported line, such as release 2.4 and release 2.3. A bug fix is first made on the main branch, so future releases include it, then it is backported to each supported branch. I cherry-pick the fix commit using the x flag, which appends the original commit hash to the message, creating traceability. Conflicts are possible because the older branch has diverged, so I resolve them and rerun the tests for that version rather than assuming the same fix works. After merge I tag a patch release with an annotated tag and publish artifacts. To scale this, many teams use bot automation where a label on the pull request opens backport pull requests automatically. I use branch contains to verify which branches already include a fix, and I retire release branches according to a published support policy.

Likely follow-ups

- Why fix main first?
- What does cherry-pick -x add?

Maintain release/x.y branches for supported versions. Fix on main first, then cherry-pick with -x to each supported release branch, test, and tag patch releases. Automation labels PRs for backports.

```
git switch release/2.4
```

```
git cherry-pick -x abc1234
```

```
git tag -a v2.4.3 -m 'Patch release'
```

```
git branch --contains abc1234 -r
```

Interview tip: Always fix forward on main first so the bug cannot reappear in the next release.

📚 Official docs

- git-cherry-pick

certs.eknathalabs.com

Q128 How do feature flags reduce the need for long-lived branches?

Branching

🎙 Spoken answer · ~59 s

Long-lived branches exist because a feature cannot ship until it is finished, so developers isolate it, and the longer it lives, the worse the merge pain and integration surprises become. Feature flags break that coupling. I merge small increments to the main branch continuously, with the unfinished code hidden behind a runtime switch that defaults to off. Deployment then no longer equals release: the code is in production but dormant, and I enable it for internal users, then a percentage of traffic, and can turn it off instantly without a redeploy if metrics worsen. This supports trunk-based development and quick reviews of small changes. The costs are extra code paths to test, the need for flag management tooling, and the risk of stale flags, so each flag gets an owner and a removal date and is cleaned up soon after full rollout.

Likely follow-ups

- What are the risks of flags?
- How do you test both flag states?

Flags let incomplete work merge to main behind a switch, decoupling deploy from release. Branches stay short, integration is continuous, and rollout can be gradual and instantly reversible.

```
if (flags.isEnabled('new-checkout', user)) { newFlow() } else { oldFlow() }
```

```
git log --since=2.weeks --merges --oneline | wc -l
```

```
git branch --sort=committerdate --format='%(committerdate:short) %(refname:short)'
```

Interview tip: Flags are debt. Track and delete them after rollout, or the codebase fills with dead paths.

📚 Official docs

- Martin Fowler: Feature Toggles
- Trunk Based Development

certs.eknathalabs.com

Q129 How do you rename the default branch from master to main without breaking clones and CI?

Branching

🎙 **Spoken answer · ~58 s**

The rename itself is trivial, but the references to the old name are everywhere. My plan starts with an inventory: CI pipeline triggers, deployment scripts, branch protection rules, merge queue settings, code owners, badges, documentation and any automation that hard-codes the name. Then I rename on the hosting platform, which usually retargets open pull requests and offers redirects for web links. I update the protection rules and pipeline conditions at the same time, ideally using a window with low activity. Each developer then updates their clone by renaming the local branch, fetching, setting the new upstream and refreshing the remote HEAD pointer. I send a short note with the exact commands. Afterwards I watch for pipelines that silently stopped triggering, and keep the old branch name as a protected alias for a short period if integrations are slow to update.

Likely follow-ups

- What breaks silently after a rename?
- How do forks get updated?

Rename on the host, which can set redirects; update local clones with branch -m and upstream changes; fix CI triggers, protection rules, deploy configs, docs and webhooks; communicate with a short checklist.

```
git branch -m master main
```

```
git fetch origin
```

```
git branch -u origin/main main
```

```
git remote set-head origin -a
```

Interview tip: Search the whole organisation for hard-coded master references in pipelines, badges and scripts before cutting over.

📚 **Official docs**

- Git documentation

certs.eknathalabs.com

Q130 What is stacked branching (stacked PRs) and when is it worth the complexity?

Branching

🎙 **Spoken answer · ~59 s**

A big feature is hard to review as one pull request, so I split it into a chain where branch two is based on branch one, and branch three on branch two, each a small reviewable change that targets its predecessor. Reviewers can approve each piece independently, and merging proceeds from the bottom up. The difficulty arises when a lower branch changes after review feedback, because the branches above it must be rebased on the new base. Newer Git versions help with the update-refs option, which moves every branch in the stack during a single rebase, and there are tools that automate restacking and opening pull requests. I always push with force-with-lease because rebased branches need force. Stacking is worth it for large, naturally sequential work, but for small changes it adds overhead, so I prefer short independent pull requests when possible.

Likely follow-ups

- What is force-with-lease?
- When is stacking not worth it?

Stacked branches build each change on top of the previous one so large work is reviewed as small dependent pull requests. Tools automate rebasing the stack when a lower branch changes.

```
git switch -c part1 && git switch -c part2
```

```
git rebase --onto part1 old-part1 part2
```

```
git rebase --update-refs main
```

```
git push --force-with-lease origin part1 part2
```

Interview tip: The update-refs flag moves all intermediate branch pointers during a rebase, which makes stacks far easier to maintain.

📚 **Official docs**

- Git documentation

certs.eknathalabs.com

Q131 What merge strategies and options does Git offer (ort, recursive, ours, theirs, subtree), and when do they matter?

Merging & rebasing

🎙 Spoken answer · ~59 s

A merge strategy is the algorithm that combines trees. Modern Git uses ort by default, which is fast and handles renames well. Strategy options adjust behaviour: the X theirs and X ours options resolve only the conflicting hunks in favour of one side while still merging the non-conflicting changes from both, which is sometimes useful for generated files. That is very different from the ours strategy, which creates a merge commit that keeps my tree exactly as it was and just records the other branch as merged, which I use to mark an obsolete branch as integrated without taking its content. The subtree strategy merges into a subdirectory for vendored projects. The danger is that automatic preference hides real conflicts and silently drops someone's work, so I use these sparingly, review the result with diff, and prefer manual resolution for source code.

Likely follow-ups

- What is the difference between -X ours and -s ours?
- Why is patience diff useful?

The default ort strategy handles most merges. -X ours/theirs resolve conflicting hunks automatically in favour of one side; -s ours discards the other branch's changes entirely. Choose carefully because both can hide mistakes.

```
git merge -s ort -X patience feature
```

```
git merge -X theirs feature
```

```
git merge -s ours obsolete-branch
```

```
git merge -s subtree vendor/lib
```

Interview tip: -X ours only affects conflicting hunks. -s ours keeps your tree and records the merge, ignoring all of theirs.

📚 Official docs

- git-merge
- Pro Git: Submodules
- PostgreSQL: Indexes

certs.eknathalabs.com

Q132 How do you configure and use a mergetool and diff3/zdiff3 conflict style to make resolution easier?

Merging & rebasing

🎙 **Spoken answer · ~58 s**

By default, conflict markers show only my side and theirs, which forces guessing about intent. Setting the conflict style to diff3, or the cleaner zdiff3, adds a third section with the original common ancestor, so I can see what each side changed relative to the base and combine both intents correctly. For bigger conflicts I configure a graphical or terminal mergetool that opens base, local, remote and the merged result in a four-pane view. Git also exposes the three versions as stages in the index, which I can inspect with the checkout options for ours and theirs, or the diff against base. After resolving I search for leftover markers, run the build and tests, and only then add and continue. With rerere enabled, Git remembers my resolutions so repeated rebases do not require resolving the same conflict again.

Likely follow-ups

- What does zdiff3 improve?
- What are index stages 1, 2 and 3?

The diff3 and zdiff3 styles include the common ancestor in conflict markers, so you can see what each side changed. A three-way mergetool shows base, local and remote side by side.

```
git config --global merge.conflictStyle zdiff3
```

```
git mergetool
```

```
git checkout --ours file && git checkout --theirs file
```

```
git diff --base file
```

Interview tip: Seeing the base version often makes the right resolution obvious, instead of guessing between two competing blocks.

📚 **Official docs**

- Git documentation

certs.eknathalabs.com

Q133 How do you merge a branch that has already been squash-merged elsewhere without getting duplicate conflicts?

Merging & rebasing

🎙 Spoken answer · ~59 s

A squash merge creates one new commit on main containing the content of the whole branch, but it does not record the branch's commits as ancestors. If another branch was built on top of that original branch, Git later thinks the original commits are still unmerged, and a regular rebase or merge replays them, producing conflicts with changes that main already contains. The fix is to tell Git exactly which commits to carry over. I use rebase with the onto option, giving the new base as main, the old branch tip as the upstream to exclude, and the child branch to move. This replays only the child's own commits. The cherry command helps verify what is considered new. To avoid the problem, I rebase child branches immediately after the parent is squashed, or avoid squash merging when stacked work is expected.

Likely follow-ups

- Why does squash lose ancestry?
- What does git cherry show?

After a squash merge, the original commits are not ancestors of main, so Git sees them as unmerged. Rebase the child branch with --onto to drop the already-squashed commits, or delete and recreate from main.

```
git rebase --onto main old-feature child-feature
```

```
git log main..child-feature --oneline
```

```
git cherry -v main child-feature
```

Interview tip: git cherry marks commits already applied upstream with a minus, which helps identify what is truly new.

📚 Official docs

- git-rebase
- git-merge

certs.eknathalabs.com

Q134 What is git merge --abort versus git reset --merge, and how do you safely back out of a merge in progress?

Merging & rebasing

🎙 **Spoken answer · ~63 s**

If a merge stops with conflicts and I decide it is not the right time, the abort option returns the working tree and index to how they were before the merge started, as long as I did not have uncommitted changes that got mixed in, which is why I start merges from a clean tree. The reset with the merge option is the underlying mechanism and works when abort refuses. If I already completed the merge commit locally but have not pushed, I can move the branch back using ORIG_HEAD, which Git sets to the previous tip before risky operations. If the merge is already pushed and others may have pulled it, I never rewrite history; I revert the merge commit and specify the mainline parent, which makes a new commit undoing the changes. Note that reverting a merge affects later re-merging of that branch, which I handle by reverting the revert.

Likely follow-ups

- What is ORIG_HEAD?
- Why does reverting a merge need -m?

merge --abort restores the pre-merge state when no uncommitted changes would be lost. reset --merge is the lower-level equivalent. If the merge was already committed locally, use reset to ORIG_HEAD or revert if it was pushed.

```
git merge --abort
```

```
git reset --merge
```

```
git reset --hard ORIG_HEAD
```

```
git revert -m 1 <merge-sha>
```

Interview tip: Commit or stash your local edits before merging so an abort can cleanly restore the starting point.

📚 **Official docs**

- git-reset
- Git documentation

certs.eknathalabs.com

Q135 How does git rebase --autosquash with fixup commits streamline addressing review feedback?

Merging & rebasing

🎙 Spoken answer · ~58 s

During code review I often need to change something that belongs in an earlier commit of my branch. Rather than amending blindly, I commit the correction with the fixup option pointing at the target commit, which creates a commit whose message starts with fixup and the original title. Reviewers can read the small incremental commit, which makes re-review easy. When the review is finished, I run an interactive rebase against the base branch with the autosquash option, and Git automatically reorders each fixup right after its target and marks it to be squashed in, so I only need to save the todo list. The result is a clean, logical history. Because the commits are rewritten, I push with force-with-lease, and only do this on my own feature branch, never on a shared branch that others are building on.

Likely follow-ups

- What is the difference between fixup and squash?
- Why force-with-lease and not force?

Create fixup! commits targeting earlier commits with git commit --fixup, then run an interactive rebase with --autosquash to reorder and squash them automatically into their targets.

```
git commit --fixup=abc1234
```

```
git rebase -i --autosquash main
```

```
git config --global rebase.autoSquash true
```

```
git push --force-with-lease
```

Interview tip: Reviewers can see the incremental fixup commits during review, and you squash them away just before merge.

📚 Official docs

- git-rebase
- Git documentation

certs.eknathalabs.com

Q136 How do you use git log -S and -G (pickaxe) to find when a piece of code was added or removed?

History & commits

🎙 Spoken answer · ~58 s

When I need to know when a constant or function appeared, disappeared or changed, searching the current file is not enough. The S option, known as the pickaxe, lists commits where the count of a given string changed, so it finds the commit that introduced or deleted it. The G option takes a regular expression and finds commits whose diff lines match, which also catches modifications. Adding the patch flag shows the actual change, and restricting to a path speeds it up. For tracking a function across refactors, the L option follows a function name or line range through history. If a file was renamed, the follow option keeps tracing before the rename. Together these answer why a line exists and who changed it, which is essential for incident investigation and for understanding a legacy decision before changing it.

Likely follow-ups

- Difference between -S and -G?
- What does -L do?

-S finds commits where the number of occurrences of a string changed; -G finds commits whose diff matches a regex. Combine with -p and --follow to trace code through history.

```
git log -S'MAX_RETRIES' --oneline
```

```
git log -G'timeout\s*=\s*\d+' -p -- src/
```

```
git log -L :parseConfig:src/config.py
```

```
git log --follow -p -- path/to/file
```

Interview tip: -L traces the history of a function or line range, which often beats blame for understanding how code evolved.

📚 Official docs

- Git documentation

certs.eknathalabs.com

Q137 How do you rewrite history across an entire repository using git filter-repo, and what are the consequences?

History & commits

🎙 Spoken answer · ~58 s

Sometimes a repository needs a global history change: remove a leaked credential file, strip a huge binary, correct author emails, or split a directory out into its own repository. The filter-repo tool does this quickly and safely, and replaced the old filter-branch command, which was slow and error-prone. I run it in a fresh mirror clone, specify the path to remove or the text to replace, and it rewrites all commits and refs. The consequences are significant: every affected commit gets a new hash, so existing clones, forks, open pull requests, tags and signatures no longer match. I coordinate a freeze, force-push the rewritten refs, ask everyone to reclone rather than merge, and ask the hosting provider to purge cached objects. For leaked secrets, rewriting history is secondary to rotating the credential, because copies may already exist elsewhere.

Likely follow-ups

- Why must everyone reclone?
- Why is rotation more important for leaks?

filter-repo is the supported replacement for filter-branch. It rewrites every commit to drop paths, replace text or change authors. All hashes change, so everyone must reclone and open PRs break.

```
git filter-repo --path secrets.env --invert-paths
```

```
git filter-repo --replace-text expressions.txt
```

```
git filter-repo --mailmap mailmap.txt
```

```
git clone --mirror repo.git && cd repo.git
```

Interview tip: Always work on a fresh mirror clone and keep a backup, because the operation is destructive and rewrites everything.

📚 Official docs

- git-filter-repo
- Git documentation

certs.eknathalabs.com

Q138 What is a good strategy for using git blame effectively, including ignoring formatting-only commits?

History & commits

🎙 Spoken answer · ~60 s

Blame annotates each line with the last commit that changed it, which is a starting point, not an answer. A formatting run or a rename across the repository makes every line point at one uninteresting commit. To see through that, I use the whitespace flag, and the move and copy detection flags, which attribute lines to their original commit even if they were moved between files. For team-wide cleanliness, I keep a file listing the hashes of bulk formatting commits and configure blame to ignore them, and hosting platforms read the same file. Once I find the commit, I read its message and linked pull request to understand the reasoning, then use pickaxe search or line history if the code has been reworked several times. Blame should drive better questions, never blame people, so I frame investigations around the change, not the author.

Likely follow-ups

- What does -C detect?
- How do ignore-revs files work?

Use blame with -w, -M and -C to ignore whitespace and detect moved code, and configure blame.ignoreRevsFile so bulk reformatting commits don't hide the real author and reason.

```
git blame -w -M -C file.py
```

```
git blame -L 40,60 file.py
```

```
echo <format-commit-sha> >> .git-blame-ignore-revs
```

```
git config blame.ignoreRevsFile .git-blame-ignore-revs
```

Interview tip: Blame tells you who last touched a line, not who wrote the logic. Follow the commit message and PR for the why.

📚 Official docs

- Git documentation

certs.eknathalabs.com

Q139 How do you generate a changelog and release notes from Git history reliably?

History & commits

🎙 Spoken answer · ~60 s

A reliable changelog starts with consistent input. I adopt a convention such as conventional commits, where each message begins with a type like feat, fix or chore, and an optional scope, and I enforce it with a commit-msg hook and a CI check on pull request titles. Then notes can be generated from the range between the previous tag and the new one, grouped by type, with breaking changes called out from a marker in the message. I find the previous tag with describe, list non-merge commits in the range, and render them via a release tool or a simple script. For squash-merge workflows, the pull request title becomes the commit message, so I enforce title format instead. A human still reviews the generated text to remove noise and add context for users, and the release notes link to issues and upgrade guidance where needed.

Likely follow-ups

- What are conventional commits?
- How do squash merges affect changelogs?

Use conventional commit messages or PR labels, then generate notes from the range between tags with git log or tooling. Consistency in commit format is what makes automation trustworthy.

```
git log v1.4.0..v1.5.0 --pretty='- %s (%h)' --no-merges
```

```
git describe --tags --abbrev=0
```

```
git log --grep='^feat' v1.4.0..HEAD --oneline
```

```
git shortlog -sn v1.4.0..HEAD
```

Interview tip: Generated notes are only as good as the commit messages, so enforce a convention with a commit-msg hook or CI check.

📚 Official docs

- Git documentation

certs.eknathalabs.com

Q140 How can you find which commits touched a file but are not in a release, using git cherry, branch --contains and tag --contains?

History & commits

🎙 **Spoken answer · ~57 s**

A frequent operational question is whether a particular fix is included in the version running in production. If I have the commit hash, I ask Git which tags contain it, and which remote branches contain it; no output for the release tag means it is not included. To find what is waiting for the next release, I list commits reachable from main but not from the last release tag, optionally limited to a path or a team's directory. When release branches get cherry-picked fixes, the hashes differ, so the cherry command compares patch contents and marks which commits are already applied. For traceability, I include the original hash in backport messages and use annotated tags on each release. These queries replace guessing and spreadsheets, and they are quick to run before a release or during an incident.

Likely follow-ups

- How to check what is in the deployed build?
- What does git cherry compare?

Use contains queries to see which branches or tags include a commit, and git log with ranges to list what is on main but not yet in a release tag.

```
git tag --contains abc1234
```

```
git branch -r --contains abc1234
```

```
git log v2.3.0..origin/main --oneline -- path/
```

```
git cherry -v release/2.3 main
```

Interview tip: This answers the common production question: is my fix in the build that is deployed?

📚 **Official docs**

- Git documentation

certs.eknathalabs.com

Q141 How do you design a code review workflow using draft pull requests, required checks and CODEOWNERS?

Workflows

🎙 Spoken answer · ~58 s

I want review to be fast, meaningful and enforced by the system. Authors open a draft pull request early to get CI running and feedback on direction, then mark it ready when complete. Required status checks, such as build, tests, linting and security scans, must pass before merge. A CODEOWNERS file maps directories to owning teams so the right reviewers are requested automatically, and branch protection requires at least one approval, dismisses stale approvals when new commits arrive, and blocks direct pushes to the main branch. To keep reviews quick, I encourage small, focused changes with a clear description of what and why, link the ticket, and include test evidence. I track metrics such as time to first review and pull request size, because chronic delays usually indicate large changes or unclear ownership. Auto-merge after approval removes idle waiting.

Likely follow-ups

- Why dismiss stale approvals?
- How do you keep PRs small?

Draft PRs signal work in progress, required status checks prevent merging broken code, CODEOWNERS routes review to the right people, and branch protection enforces it all. Keep PRs small to keep review fast.

```
gh pr create --draft --fill
```

```
gh pr checks --watch
```

```
gh pr ready
```

```
gh pr merge --squash --auto
```

Interview tip: Measure time-to-first-review and PR size. Long reviews usually mean oversized changes, not slow reviewers.

📚 Official docs

- Google SRE Book: Managing Critical State (consensus)
- GitHub: About code owners

certs.eknathalabs.com

Q142 What is the forking workflow and when is it used instead of shared-repo branches?

Workflows

🎙 **Spoken answer · ~57 s**

In the forking workflow each contributor has their own server-side copy of the repository. They clone their fork, add the original project as a second remote usually named upstream, create a branch for their change, push it to their fork, and open a pull request against the upstream main branch. Maintainers need to grant write access only to a small group, which is why open source projects and cross-company collaborations use it. The contributor must keep their fork current by fetching upstream and rebasing or merging, otherwise their pull request conflicts or lacks recent changes. Inside a single company with trusted teams, the shared repository with short-lived branches is simpler, avoids duplicate storage and fork drift, and lets CI use repository secrets safely. Pull requests from forks usually run CI with reduced permissions for security reasons.

Likely follow-ups

- Why do fork PRs get limited secrets?
- How do you sync a fork?

Contributors fork the upstream repo, push to their fork, and open PRs back. It suits open source and cross-organisation work where contributors lack write access, with upstream as a read-only remote.

```
git remote add upstream https://host/org/repo.git
```

```
git fetch upstream && git rebase upstream/main
```

```
git push origin feature
```

```
git remote -v
```

Interview tip: Keep the fork's main in sync with upstream, and always branch from the updated main, not from your fork's stale copy.

📚 **Official docs**

- Git documentation

certs.eknathalabs.com

Q143 How would you set up Git workflows for a team using conventional commits, semantic-release and protected tags?

Workflows

🎙 Spoken answer · ~57 s

The idea is that the history itself drives versioning. Developers write commit messages that follow a convention: feat for a new feature, fix for a bug fix, and an exclamation mark or a breaking change footer for incompatible changes. A CI job on the main branch analyses the commits since the last release tag and computes the next semantic version: a major bump for breaking changes, minor for features, patch for fixes. It then creates the tag, builds artifacts, publishes the package and writes the changelog automatically. To keep it trustworthy, I validate commit messages or pull request titles in CI, require squash merging with a well-formed title, and use protected tag rules so only the release pipeline identity can create version tags. A dry run on pull requests shows maintainers the version impact before merging.

Likely follow-ups

- What triggers a major bump?
- Why protect tags?

Enforce commit format in PRs, let a release job read commits since the last tag to compute the next semver, create the tag and changelog, and protect tags so only the release job can create them.

```
git commit -m 'feat(api): add pagination'
```

```
git commit -m 'fix!: change error format' -m 'BREAKING CHANGE: ...'
```

```
npx semantic-release --dry-run
```

```
git tag -l 'v*' --sort=-v:refname | head
```

Interview tip: Dry runs in pull requests show the version bump a change would cause before it merges.

📚 Official docs

- Conventional Commits
- Git documentation

certs.eknathalabs.com

Q144 How do you handle a merge freeze or release train in a busy monorepo?

Workflows

🎙 Spoken answer · ~60 s

Freezing the main branch stops all progress and creates a rush afterwards, so I prefer a release train model. At a scheduled time or when criteria are met, I cut a release branch from a known good commit on main. Main stays open for new work, and the release branch accepts only approved fixes, cherry-picked from main with traceability, with a stricter protection rule and the full test suite required. The release is tagged and deployed from that branch. If a real freeze is needed, for example during a holiday, I enforce it with branch rules or a status check that fails unless the change has an exception label, rather than relying on emails. A merge queue keeps the main branch green by testing each change on top of the latest queue before merging. Clear communication of dates, exceptions and an on-call approver prevent confusion.

Likely follow-ups

- Why a release branch over freezing main?
- What does a merge queue test?

Use a merge queue or protected release branch, cut the release at a known commit, allow only approved fixes after the cut, and communicate freeze windows clearly with automation enforcing them.

```
git switch -c release/2026-10 <cut-sha>
```

```
git cherry-pick -x <approved-fix>
```

```
gh api repos/org/repo/rulesets
```

```
git log release/2026-10..main --oneline | wc -l
```

Interview tip: Prefer cutting a release branch over freezing main, so everyone else can keep merging.

📚 Start here (official docs)

- Git documentation

certs.eknathalabs.com

Q145 What Git practices help with working across time zones and asynchronous teams?

Workflows

🎙 Spoken answer · ~57 s

Asynchronous work fails when pull requests need conversation to be understood. So I write descriptions that say what changed, why, how it was tested, and what to look at first, with screenshots or logs where helpful. Automated checks provide instant feedback so authors do not wait for humans to find trivial issues. Pull requests stay small, so a reviewer can finish in one sitting, and I use CODEOWNERS plus a rota so each change has a named reviewer who covers the other time zone. Branch conventions and the rule of integrating often reduce conflicts when people hand work over. Settings like automatic pruning on fetch and a consistent pull strategy keep local clones tidy. Decisions are recorded in the pull request or in short architecture decision records, so the next person understands the reasoning without asking.

Likely follow-ups

- How do you reduce review latency across zones?
- What belongs in a PR description?

Small PRs with clear descriptions, branch naming and ownership conventions, automated checks to give instant feedback, rebasing discipline, and written decision records so work does not block on someone being online.

```
git log --since='yesterday' --format='%an %s' --no-merges
```

```
gh pr list --search 'review:required'
```

```
git config --global pull.rebase true
```

```
git config --global fetch.prune true
```

Interview tip: The goal is that a reviewer in another time zone can approve without a call, from the description and CI results alone.

📚 Official docs

- Git documentation

certs.eknathalabs.com

Q146 How do you recover lost commits that are no longer reachable from any branch, such as after a reset or a dropped stash?

Recovery & undo

🎙 **Spoken answer · ~60 s**

Git rarely deletes objects immediately. After a hard reset, a bad rebase or dropping a stash, the commits still exist in the object database, just unreferenced. First I check the reflog, which records where HEAD and each branch pointed over time; I find the entry from before the mistake and create a branch there. If the reflog does not help, for example because the commit was never checked out, I run fsck with the lost-found option, which lists dangling commits and blobs, and I inspect candidates with show to identify the right one. A dropped stash is a dangling commit, and the fsck output shows its hash so I can apply it. As soon as I find the work, I create a branch or tag pointing at it so garbage collection cannot remove it. The longer I wait, the greater the risk after automatic pruning.

Likely follow-ups

- How long do unreachable objects live?
- How do you recover a dropped stash?

Use reflog for recent HEAD positions, git fsck --lost-found for dangling commits, and unreachable stash commits via fsck. Create a branch at the recovered SHA before gc prunes it.

```
git reflog
```

```
git fsck --lost-found --no-reflogs
```

```
git branch rescue <sha>
```

```
git stash apply <dropped-stash-sha>
```

Interview tip: Unreachable objects survive until gc prunes them, typically after at least two weeks, so act promptly.

📚 **Official docs**

- git-reflog
- git-stash

certs.eknathalabs.com

Q147 You pushed to the wrong branch (for example main) by mistake. What do you do?

Recovery & undo

🎙 Spoken answer · ~58 s

First I stop and tell the team, so nobody pulls or deploys the bad state, and check whether a pipeline already started. Then I save the work: create a new branch at the commit so the changes are not lost, and I can open a proper pull request from it. To remove the mistake from the shared branch, the safest approach is to revert the commits, which adds new commits that undo them and is safe for everyone who has already fetched. If the push was seconds ago, protection rules allow it and nobody has pulled, I might reset the branch to the previous good commit and force push with a lease, but I treat that as exceptional and coordinate it. Afterwards, I fix the root cause, usually missing branch protection, and write a brief review so the guardrail exists.

Likely follow-ups

- Why is revert safer than reset here?
- What guardrail prevents this?

Act quickly: tell the team, check whether CI or deploys have picked it up, then revert on the shared branch (or force-reset only if policy allows and nobody has pulled), and move the commits to the right branch.

```
git revert <sha>
```

```
git branch correct-branch <sha>
```

```
git push --force-with-lease origin <good-sha>:main
```

```
git log origin/main --oneline -5
```

Interview tip: Reverting is the safe default on shared branches. Force-resetting main is a last resort and needs agreement.

📚 Official docs

- Git documentation

certs.eknathalabs.com

Q148 How do you fix a repository where git pull says 'refusing to merge unrelated histories'?

Recovery & undo

🎙 Spoken answer · ~58 s

This message means that Git cannot find a merge base between my branch and the remote branch. Typical causes are a repository that was re-initialised and force-pushed, a history rewrite such as a filter-repo run, or a local repo created independently and later given the remote. Before overriding the warning, I find out which it is. If the remote was rewritten, the right fix is to rebase my local commits onto the new history using the onto option, naming the old base commit, or simply create a new branch from the remote and cherry-pick my work. If I intentionally want to combine two independent repositories, the allow-unrelated-histories option performs the merge and I resolve the resulting conflicts. Blindly merging after a rewrite brings old and new copies of the same commits together and creates a confusing, duplicated history.

Likely follow-ups

- What causes unrelated histories?
- How do you carry commits after a rewrite?

The two histories share no common ancestor, for example after reinitialising or a history rewrite. Decide whether to reclone, rebase onto the new history, or deliberately merge with --allow-unrelated-histories.

```
git merge-base HEAD origin/main
```

```
git log --oneline --graph --all | head
```

```
git pull --allow-unrelated-histories
```

```
git rebase --onto origin/main <old-base> mybranch
```

Interview tip: Usually this signals a rewritten or recreated remote, so understand why before forcing a merge.

📚 Official docs

- Git documentation

certs.eknathalabs.com

Q149 How do you undo a git rebase that you already finished, using ORIG_HEAD and the reflog?

Recovery & undo

🎙 Spoken answer · ~62 s

A rebase rewrites my commits, but the old ones remain in the object store, and the reflog remembers where the branch was before. If the rebase is still in progress and going badly, I abort it. If it completed and I realise the result is wrong, I look at the reflog, find the entry that says rebase started or the position just before, and reset the branch hard to that entry. Git also sets ORIG_HEAD to the previous tip when running a rebase, so resetting to it often works immediately, but another operation such as a merge or reset can overwrite it, so the reflog is the dependable source. If I had already force-pushed the rebased branch, restoring requires pushing the old tip again with a lease, and teammates who pulled the new version need to be told. This safety net is why rewriting local history is low risk.

Likely follow-ups

- Is ORIG_HEAD always reliable?
- What if you already force-pushed?

After a completed rebase, ORIG_HEAD or the reflog entry before the rebase points at the old tip. Reset the branch to it to restore the pre-rebase history, if you have not pushed or are willing to force-push.

```
git reflog
```

```
git reset --hard ORIG_HEAD
```

```
git reset --hard 'HEAD@{5}'
```

```
git rebase --abort
```

Interview tip: ORIG_HEAD is overwritten by later risky commands, so the reflog is the more reliable record.

📚 Official docs

- git-rebase
- git-reflog
- Git documentation

certs.eknathalabs.com

Q150 What is git fsck, git prune and git gc --prune, and what should you never run during an incident?

Recovery & undo

🎙 Spoken answer · ~58 s

Fsck checks the integrity of the object database and reports corruption, missing objects and dangling commits. It is read-only and safe. Prune permanently removes unreachable objects, and garbage collection repacks objects for efficiency and, by default, prunes unreachable ones older than a grace period. These are exactly the tools that can destroy my chance of recovering lost work. So the first step in any recovery is to copy the entire repository directory as a backup, and work on the copy. I avoid running gc, prune or reflog expire until I have found and branched the commits I need, and I can even set the prune expiry to never to be safe. Automatic gc can trigger after some commands, so for a precious situation I temporarily disable it in the config. Once everything is saved and verified, normal maintenance can resume.

Likely follow-ups

- Why copy .git first?
- What does auto gc prune?

fsck verifies and lists problems or dangling objects, prune deletes unreachable objects, and gc packs and optionally prunes. During recovery, avoid gc/prune or clearing reflog until you've saved what you need.

```
git fsck --full
```

```
git count-objects -vH
```

```
git gc --prune=never
```

```
git reflog expire --expire=never --all
```

Interview tip: Before recovery, copy the whole .git directory. Then experiment on the copy.

📚 Official docs

- Linux kernel docs: ext4
- Git: Maintenance & garbage collection
- Git documentation

certs.eknathalabs.com

Q151 How do you review a pull request effectively, and what do you look for?

Collaboration

🎙 **Spoken answer · ~59 s**

I begin with the description and linked ticket to understand the intent, then check whether the approach fits the design before reading details. Next I review correctness: edge cases, error handling, concurrency and data migrations. I check that tests cover the behaviour, including failure paths, and I look for security issues like input validation, secrets and permissions, plus performance concerns such as queries in loops. For operability, I ask about logging, metrics, rollout and rollback. For risky changes I check the branch out and run it. Style issues should be handled by linters, not humans. My comments are specific and actionable, distinguish must-fix from suggestions, and explain the why, in a respectful tone. I acknowledge good work too. Large pull requests I ask to split, because review quality drops sharply with size. When done I approve clearly or request changes with reasons.

Likely follow-ups

- What should linters handle instead of humans?
- How do you review a very large PR?

Start with intent and design, then correctness, tests, security, performance and operability, and finally style. Check the diff in context, run it if risky, and give specific, kind, actionable comments.

```
gh pr checkout 482
```

```
gh pr diff 482
```

```
git diff main...HEAD --stat
```

```
gh pr review 482 --comment -b 'Question about error handling'
```

Interview tip: Approve with confidence or ask for changes with reasons. Vague approvals and nitpick floods both waste time.

📚 **Start here (official docs)**

- Git documentation

certs.eknathalabs.com

Q152 How do you use git range-diff to review a force-pushed update to a pull request?

Collaboration

🎙 Spoken answer · ~57 s

When an author rebases and force-pushes a pull request after feedback, the hosting UI often shows the whole diff again, and reviewers cannot tell which parts changed in response to comments. Range-diff solves this by matching the commits of the old series to the new series and showing the differences between corresponding commits, with markers for unchanged, modified, added or dropped commits. As a reviewer, I keep the previous tip, which is available in my reflog or from the remote-tracking branch before fetching, and run range-diff with the common base and both ranges. As an author, I help reviewers by mentioning what changed in each revision, or by adding fixup commits during review and squashing only at the end. Some platforms now offer a compare-force-push view with similar value. It makes iterative review of rebased series practical.

Likely follow-ups

- What does range-diff output mean?
- How can authors make re-review easier?

range-diff compares two versions of a patch series commit by commit, so reviewers see what changed between the old and new push rather than rereading everything.

```
git range-diff main old-feature new-feature
```

```
git range-diff main...old-tip main...new-tip
```

```
git fetch origin pull/482/head:pr-new
```

```
git log --oneline main..pr-new
```

Interview tip: Save the old tip before the author force-pushes (via reflog or remote-tracking history) to be able to compare.

📚 Official docs

- GitHub: Protected branches
- Git documentation

certs.eknathalabs.com

Q153 What is the difference between --force and --force-with-lease, and how do you protect shared branches?

Collaboration

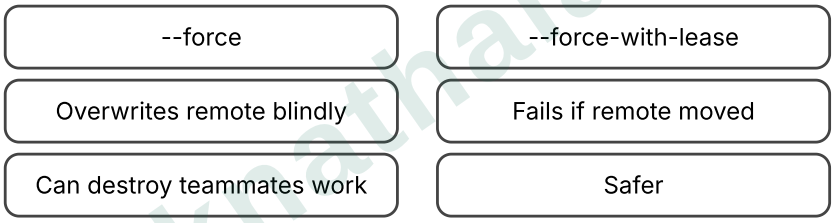
🎙 Spoken answer · ~59 s

After rewriting my own branch with rebase or amend, a normal push is rejected because histories diverged, so I need to force. The plain force option replaces the remote branch no matter what, so if a colleague pushed a commit to it meanwhile, it is destroyed. The lease variant adds a condition: the push only succeeds if the remote branch is still at the commit I last saw, so it fails if someone else has pushed, and I can fetch and review first. A subtlety is that an automatic background fetch can update my remote-tracking reference without me inspecting the changes, which defeats the safety, and the force-if-includes option guards against that. For shared branches such as main and release branches, I rely on server-side protection to forbid force pushes entirely, and the rule of rewriting only branches that I own.

Likely follow-ups

- When does lease protect you?
- Which branches should forbid force-push?

🗺 Force push



--force overwrites the remote ref unconditionally. --force-with-lease refuses if the remote moved since you last fetched, protecting others' work. Combine with branch protection that blocks force pushes on shared branches.

```
git push --force-with-lease
```

```
git push --force-with-lease=feature:abc1234
```

```
git push --force-if-includes
```

```
git config --global alias.pf 'push --force-with-lease'
```

Interview tip: Background fetches can silently update your lease reference. Use force-if-includes for extra safety.

📚 Official docs

- Git documentation

certs.eknathalabs.com

Q154 How do you work with Git across multiple remotes, such as mirrors and a vendor repo?

Collaboration

🎙 **Spoken answer · ~58 s**

A repository can have any number of named remotes. In a typical case, origin is my fork or the team repository and upstream is the original project, or a vendor remote carries a third-party codebase I patch. I add each with a name, fetch them individually or all together with pruning, and use remote-tracking branches such as vendor main to compare and merge. For redundancy, a remote can have multiple push URLs so a single push updates the primary and a backup mirror. The mirror option pushes all refs and deletes anything missing locally, which is powerful for creating exact copies but dangerous if pointed at the wrong place, so it is reserved for dedicated mirrors. I keep naming consistent across the team, check the configured remotes with the verbose list, and make upstream pushes impossible by using read-only credentials.

Likely follow-ups

- What does --mirror do?
- How do you merge vendor updates?

Add named remotes, fetch each separately, push to specific ones, and configure pushurl for mirroring. Track which remote is origin and keep upstream read-only to avoid confusion.

```
git remote add vendor https://vendor/repo.git
```

```
git remote set-url --add --push origin git@mirror:org/repo.git
```

```
git fetch --all --prune
```

```
git push mirror --mirror
```

Interview tip: A mirror push overwrites everything on the destination, so use it only for dedicated read-only mirrors.

📚 **Official docs**

- Git documentation

certs.eknathalabs.com

Q155 How do you handle line-ending and whitespace issues across a mixed Windows, Mac and Linux team using .gitattributes?

Collaboration

🎙 Spoken answer · ~59 s

Relying on each developer's autocrlf setting leads to inconsistent line endings and noisy diffs where every line appears changed. A better approach is to store the rules in the repository with a gitattributes file, so everyone gets the same behaviour. I set text to auto for general files, which lets Git detect text and normalise to LF in the repository, and add explicit rules where the working-tree ending matters: LF for shell scripts and Makefiles, CRLF for Windows batch files, and a binary marker for images and archives. When introducing the file into an existing repository, I run a renormalise once and commit the result in a separate, clearly labelled commit, and add it to the blame ignore file so history stays useful. An editorconfig file complements this by guiding editors, and a CI check can catch accidental CRLF in scripts.

Likely follow-ups

- Why pin scripts to LF?
- How do you renormalise an existing repo?

Commit a .gitattributes with text=auto and explicit eol rules for scripts, so normalisation is repo-wide and independent of each developer's autocrlf setting. Renormalise once when introducing it.

```
* text=auto
```

```
*.sh text eol=lf
```

```
*.bat text eol=crlf
```

```
git add --renormalize .
```

Interview tip: Shell scripts with CRLF endings fail on Linux with a confusing 'bad interpreter' error, so pin them to LF.

📚 Start here (official docs)

- Git documentation

certs.eknathalabs.com

Q156 How do you speed up CI checkouts for large repositories using partial clone, shallow fetch and caching?

CI/CD & GitOps

🎙 Spoken answer · ~60 s

Cloning a huge repository with full history on every CI job wastes minutes and bandwidth. I reduce the cost in layers. A shallow clone with a small depth fetches only recent commits, which is enough for most build and test jobs. A partial clone with the blob filter downloads commit and tree objects up front but fetches file contents lazily, so only what the job needs is transferred. Combined with sparse checkout, a job in a monorepo materialises only its own directories. For self-hosted runners, a persistent reference mirror or cache on the runner means a clone borrows objects locally. The trade-off is that some jobs need history, such as those that compute versions from tags, run blame-based analysis, or diff against the merge base, so I configure those with a deeper fetch or unshallow explicitly. Measuring checkout time per job shows where to focus.

Likely follow-ups

- What does the blob:none filter do?
- Which jobs need full history?

Use fetch-depth limits, blobless or treeless partial clones, sparse checkout and runner-local reference caches. Fetch full history only for jobs that truly need it, like version computation.

```
git clone --filter=blob:none --depth=1 URL
```

```
git sparse-checkout set services/api
```

```
git fetch --unshallow
```

```
git clone --reference /cache/repo.git URL
```

Interview tip: Release jobs that compute versions from tags need full history, so give them a separate checkout configuration.

📚 Official docs

- Git: Maintenance & garbage collection

certs.eknathalabs.com

Q157 How do you build a pipeline that tests only what changed in a monorepo? CI/CD & GitOps

🎙 Spoken answer · ~61 s

Running every test for every change does not scale in a monorepo. I find the changed files by diffing the pull request head against the merge base with the target branch, then map those files to projects. Simple path filters in the CI configuration work for independent services, but they miss indirect effects, so I prefer a build tool that understands the dependency graph and can list affected projects, meaning that a change in a shared library triggers tests for all its consumers. Changes to global configuration, the lockfile or the pipeline itself trigger the full suite. The main branch still runs everything, perhaps nightly or on merge, as a safety net, and a merge queue validates the combined result. Remote caching of build and test results avoids rerunning unchanged work. I monitor flaky tests and skipped-test accuracy, since a wrong dependency map silently lets bugs through.

Likely follow-ups

- Why isn't path filtering enough?
- Why still run everything on main?

Compute the changed paths between the PR and its merge base, map them to affected projects via a dependency graph, and run only those jobs, while always running full tests on main or on shared-config changes.

```
git diff --name-only origin/main...HEAD
```

```
git diff --name-only $(git merge-base origin/main HEAD) HEAD -- services/api/
```

```
paths: ['services/api/**']
```

```
nx affected --target=test --base=origin/main
```

Interview tip: Path filters alone miss dependency changes. A change to a shared library must trigger every consumer.

📚 Start here (official docs)

- Git documentation

certs.eknathalabs.com

Q158 How do you securely trigger deployments from Git tags or merges, and prevent unauthorised releases?

CI/CD & GitOps

🎙 **Spoken answer · ~58 s**

Anyone who can create a tag or merge to the release branch can trigger production changes, so those actions are guarded. Tag creation is limited by protected tag rules to the release team or the pipeline. The workflow triggered by a tag deploys to production only through a protected environment that requires manual approval from designated reviewers and optionally a wait timer. The job obtains short-lived cloud credentials by OIDC, scoped to the repository and environment, so no long-lived secrets are stored. It deploys an immutable artifact that was built and tested earlier, identified by digest, rather than rebuilding from source, which guarantees that what was tested is what ships. Signed tags let the job verify the release, and every deployment records the commit, the approver and the artifact version for audit. Rollback is redeploying a previous artifact.

Likely follow-ups

- Why build once and promote?
- What does an environment protection rule do?

Protect release branches and tags, require approvals through environment rules, use OIDC short-lived credentials in the deploy job, and restrict who can create tags. Deploy from immutable artifacts built once.

```
git tag -a v1.8.0 -m 'Release 1.8.0' && git push origin v1.8.0
```

```
on: push: tags: ['v*.*.*']
```

```
environment: production  # requires reviewers
```

```
git verify-tag v1.8.0
```

Interview tip: Build once and promote the same artifact through environments instead of rebuilding per stage.

📚 **Official docs**

- Git documentation

certs.eknathalabs.com

Q159 How do you handle flaky tests and failed CI blocking merges without weakening the quality gate?

CI/CD & GitOps

🎙 **Spoken answer · ~60 s**

Flaky tests erode trust: people re-run until green, real failures get ignored, and merges slow down. I treat flakiness as a defect with an owner. First, measure: collect pass-fail history per test so I know which are unreliable, and detect them by re-running suspicious tests many times in isolation or in random order. Next, move confirmed flaky tests to a quarantine suite that still runs and reports but does not block merges, with a ticket, an owner and a deadline. A limited automatic retry may be used for known infrastructure noise, but retries are logged and visible so they are not hiding problems. Then fix the root causes, which are commonly timing assumptions, shared state, order dependence, network calls and unmocked time. The required checks stay strict for healthy tests, so the gate keeps its meaning, and a dashboard shows the flake rate trending down.

Likely follow-ups

- How do you detect flakiness?
- Why is quarantine with deadlines important?

Detect and quarantine flaky tests with tracking, retry sparingly with visibility, fix root causes with owners and SLAs, and keep required checks reliable. Never normalise re-running until green.

```
pytest --lf --maxfail=1
```

```
pytest -p no:randomly --count=20 tests/test_payment.py::test_retry
```

```
gh run rerun --failed
```

```
git log --since=30.days --format='%h' -- tests/ | wc -l
```

Interview tip: Track flake rate per test. A quarantine list without owners and deadlines becomes a graveyard.

📚 **Start here (official docs)**

- Git documentation

certs.eknathalabs.com

Q160 How do you configure and enforce SSH commit signing and verify signatures in CI?

Security

🎙 Spoken answer · ~58 s

Signing proves that a commit really came from the holder of a key, not just someone who typed an author name. Modern Git supports signing with SSH keys, which is easier than GPG since developers already have them. I configure the signing format, point the signing key at the public key, and enable signing by default for commits and tags. The hosting platform needs the same public key registered as a signing key so it can display a verified badge. For enforcement, branch rules can require signed commits so unsigned work cannot be merged. In CI, I can verify signatures against an allowed signers file that maps identities to public keys, failing the build if a commit is unsigned or signed by an unknown key. Key rotation and revocation need a process, and hardware-backed keys add protection against theft.

Likely follow-ups

- Why choose SSH over GPG?
- How do you revoke a signing key?

Set gpg.format ssh and user.signingkey, enable commit.gpgsign, and register the public key as a signing key on the host. Verify with an allowed signers file in CI and enforce through branch rules.

```
git config --global gpg.format ssh
```

```
git config --global user.signingkey ~/.ssh/id_ed25519.pub
```

```
git config --global commit.gpgsign true
```

```
git verify-commit HEAD
```

Interview tip: SSH signing is simpler than GPG for most teams because developers already manage SSH keys.

📚 Official docs

- Git: Signing your work
- Git documentation

certs.eknathalabs.com

Q161 How do you scan Git history for secrets and set up preventive controls like pre-commit hooks and push protection?

Security

🎙 **Spoken answer · ~56 s**

Secrets in Git are dangerous because history is permanent and widely copied. I use scanners that examine every commit across all branches with entropy checks and patterns for known providers, ideally with verification against the provider so findings are real. A pre-commit hook catches mistakes on the developer machine before they enter history, a CI job scans pull requests as a backstop because hooks can be skipped, and the hosting platform's push protection blocks pushes containing known secret formats. For existing repositories, I run a full-history scan once, triage the results, and treat each live secret as compromised: rotate it first, then optionally rewrite history to remove it. I maintain an allowlist for false positives with reasons. Education matters too, so developers use environment variables, secret managers and injected credentials rather than committing config files.

Likely follow-ups

- Why isn't a pre-commit hook enough?
- What do you do with a found secret?

Run scanners such as gitleaks or trufflehog over full history, add pre-commit and CI scans, and enable server-side push protection. Treat any found secret as compromised and rotate it.

```
gitleaks detect --source . --log-opts='--all'
```

```
trufflehog git file://. --only-verified
```

```
pre-commit run gitleaks --all-files
```

```
git log --all -S'AKIA' --oneline
```

Interview tip: Defence in depth: local hook for speed, CI scan as a safety net, and server push protection that cannot be bypassed.

📚 **Official docs**

- githooks
- GitHub: About secret scanning
- Git documentation

certs.eknathalabs.com

Q162 How do you secure a Git hosting platform's access using SSO, SSH certificates, deploy keys and least-privilege tokens?

Security

🎙 **Spoken answer · ~56 s**

The goal is that the right identity gets the least access for the shortest time. Humans sign in through the company identity provider with SSO and multi-factor authentication, so offboarding removes access everywhere. For SSH, I prefer short-lived certificates issued by a certificate authority after authentication, instead of long-lived keys that sit on laptops forever. Automation uses scoped credentials: deploy keys limited to one repository and read-only by default, or app tokens and OIDC-issued tokens limited to specific repositories and permissions that expire within minutes. Personal access tokens are discouraged for CI because they inherit a person's broad rights and outlive their role. I organise access by teams with roles rather than individuals, review membership and token inventory periodically, and send audit logs to the SIEM to alert on unusual clone volumes or permission changes.

Likely follow-ups

- Why avoid PATs in CI?
- How do SSH certificates help offboarding?

Enforce SSO and MFA, use SSH certificates or short-lived tokens instead of static keys, give CI read-only deploy keys or app tokens scoped to a repo, and review access regularly.

```
ssh-keygen -s ca_key -I alice -n git -V +12h alice.pub
```

```
git config --get-regexp '^url\..*insteadof'
```

```
gh api orgs/acme/members --paginate -q '.[].login'
```

```
gh auth status
```

Interview tip: Prefer GitHub App or OIDC-issued tokens, which expire quickly and can be limited to specific repos and permissions.

📚 **Official docs**

- OpenSSH manual pages
- Git: Credential storage
- OAuth 2.0 / OIDC overview (OpenID)

certs.eknathalabs.com

Q163 What risks do Git hooks, .gitattributes filters and submodule URLs pose when cloning untrusted repositories?

Security

🎙 **Spoken answer · ~58 s**

A repository can contain content that executes when I run normal commands. Hooks themselves are not transferred by clone, but build scripts, test runners and package manifests obviously run code. Git-specific vectors include smudge and clean filters or external diff drivers referenced by attributes, which can run commands if the matching configuration exists, submodule URLs that use unexpected protocols or point at local paths, and past vulnerabilities in handling of symlinks and case-insensitive paths. To reduce exposure, I keep Git updated, restrict allowed transport protocols such as ext and local file, inspect the submodule file before updating submodules, and clone with checkout disabled to examine contents first. Reviewing unknown code happens in a disposable container or virtual machine with no credentials. Automated tooling that clones user-supplied repositories, like CI for forks, must run with minimal privileges and no secrets.

Likely follow-ups

- Are hooks cloned?
- Why sandbox untrusted repos?

Client hooks are not cloned, but malicious configs, filters, submodule URLs and aliases can execute code if you run commands in a hostile repo. Inspect first, disable risky features and keep Git patched.

```
git clone --no-checkout URL && cat .gitmodules
```

```
git -c core.hooksPath=/dev/null status
```

```
git config --global protocol.ext.allow never
```

```
git config --global protocol.file.allow user
```

Interview tip: Treat cloning and then building untrusted code the same as running untrusted code. Use a sandbox.

📚 **Official docs**

- githooks
- Pro Git: Submodules
- Git documentation

certs.eknathalabs.com

Q164 How do you meet audit and compliance requirements for Git, such as separation of duties and immutable history?

Security

🎙 Spoken answer · ~58 s

Compliance frameworks ask that no single person can push unreviewed code to production, that changes are traceable, and that records cannot be altered. I implement separation of duties with branch protection requiring approval from a reviewer who is not the author, passing required checks, and no bypass for administrators except via a logged break-glass process. Signed commits and tags support integrity and identity. Every change links to a ticket through branch or commit conventions, and the first-parent history of main shows each merged pull request with its approvals. The hosting platform's audit log, covering permission changes, protection edits and token usage, is exported to long-term storage per retention rules. Releases are tagged and associated with deployment records. At audit time I can sample a commit and show the ticket, review, test results and deployment, which provides the evidence trail.

Likely follow-ups

- How do you handle break-glass access?
- What evidence does an auditor sample?

Enforce protected branches, required reviews by someone other than the author, signed commits, audit log export, retention, and tagged releases linking commit to ticket and approval. Ensure admins cannot bypass without trace.

```
gh api repos/org/repo/branches/main/protection
```

```
git log --format='%H %an %GS %G?' origin/main | head
```

```
git log --merges --first-parent main --format='%h %s'
```

```
gh api orgs/acme/audit-log --paginate
```

Interview tip: Auditors want evidence: who approved, what checks passed, when it deployed. Make that traceable from a single commit.

📚 Official docs

- Git documentation

certs.eknathalabs.com

Q165 What are partial clones and promisor remotes, and how do they differ from shallow clones?

Advanced & internals

🎙 Spoken answer · ~55 s

A shallow clone limits how many commits of history are downloaded, so operations like log, blame and merge-base may not work properly beyond the cut. A partial clone keeps the complete commit history but uses a filter to skip some objects at clone time. For example, the blob none filter downloads commits and trees but not file contents, and Git fetches missing blobs on demand from the promisor remote, which is the server that promises to supply them later. This gives a fast initial clone with full history for log and graph operations, and file contents arrive when I check out or diff. The trade-off is network dependency during later operations, so offline work can fail on missing objects. Tree-less clones go further. I choose blobless for developers and shallow for ephemeral CI.

Likely follow-ups

- What is a promisor remote?
- What breaks offline in a partial clone?

Shallow clones truncate history by depth. Partial clones keep full history but defer downloading objects (blobs or trees) until needed, using a promisor remote. Partial keeps log and blame working.

```
git clone --filter=blob:none URL
```

```
git clone --filter=tree:0 URL
```

```
git clone --depth=1 URL
```

```
git config remote.origin.promisor
```

Interview tip: Blobless clones suit developers who need history. Shallow clones suit throwaway CI builds.

📚 Official docs

- Git: Maintenance & garbage collection

certs.eknathalabs.com

Q166 What are Git maintenance and scheduled background tasks, and how do they keep large repositories fast?

Advanced & internals

🎙 Spoken answer · ~55 s

As a repository grows, loose objects, many small pack files and stale metadata slow down status, log and fetch. Garbage collection can fix this, but running it automatically in the foreground interrupts work. The maintenance command schedules smaller background jobs using the operating system scheduler: hourly prefetch of remote changes, commit-graph updates that accelerate history traversal, incremental repacking that consolidates packs without a heavy full rewrite, and loose-object packing. I enable it per repository with the start command, which registers the repo and installs the schedule, and can choose the incremental strategy. Combined with the filesystem monitor and untracked cache for status speed, it keeps large monorepos responsive. I verify results with object statistics and timing common commands before and after, and keep an occasional full gc on the server side.

Likely follow-ups

- What does prefetch do?
- What is the commit-graph?

git maintenance runs prefetch, commit-graph updates, incremental repack and loose-object cleanup on a schedule, keeping operations fast without blocking developers. Register repos with maintenance start.

```
git maintenance start
```

```
git maintenance run --task=incremental-repack
```

```
git config maintenance.strategy incremental
```

```
git count-objects -vH
```

Interview tip: Prefetch keeps remote-tracking refs fresh in the background, so fetch and pull feel instant.

📚 Official docs

- Git: Maintenance & garbage collection
- Git documentation

certs.eknathalabs.com

Q167 How does Git's fsmonitor and untracked cache speed up git status in huge working trees?

Advanced & internals

🎙 Spoken answer · ~56 s

By default status must examine every tracked file's metadata and scan directories for untracked files, which on a repository with hundreds of thousands of files takes seconds. The built-in file system monitor registers with the operating system to be told which paths changed since the last query, so Git only inspects those. The untracked cache stores the modification times of directories, so it can skip rescanning directories that have not changed. Enabling them is a configuration change, and they work best together with periodic maintenance. The caveats are network file systems and some container mounts where change notifications are unreliable, and rare cases where a stale cache needs to be reset. I measure with performance tracing before and after to confirm the benefit, and I recommend them in the onboarding guide for monorepo developers.

Likely follow-ups

- Where does fsmonitor not work well?
- How do you measure the speedup?

fsmonitor uses OS file-change notifications to avoid scanning the whole tree, and the untracked cache remembers directory states. Together they make status near-instant on large repositories.

```
git config core.fsmonitor true
```

```
git config core.untrackedCache true
```

```
git update-index --untracked-cache
```

```
GIT_TRACE_PERFORMANCE=1 git status
```

Interview tip: Measure with trace performance before and after. Gains are largest on big trees with few changes.

📚 Official docs

- Git documentation

certs.eknathalabs.com

Q168 What is git notes and how can it attach metadata such as build or review info to commits without changing their hashes?

Advanced & internals

🎙 Spoken answer · ~59 s

Commit messages are immutable once shared, because changing them changes the hash. But sometimes I want to attach information after the fact, for example the deployment time, test run link or compliance approval. Git notes store annotations in a separate ref, keyed by the commit hash, so the commit itself stays unchanged. I can view them in log output and edit them as needed. The catch is that standard push and fetch do not transfer notes, so I either push and fetch the notes ref explicitly or add the refspec to the remote configuration. Many hosting platforms do not display notes, which limits adoption, and rewriting commits through rebase orphans their notes unless the rewrite is configured to copy them. For those reasons, many teams prefer tags, release records or external systems, but notes remain useful in tooling and specialised workflows.

Likely follow-ups

- Why are notes not shared by default?
- What happens to notes after a rebase?

Notes store extra text in a separate ref linked to a commit, leaving the commit and its hash untouched. They suit post-hoc metadata like CI results, but must be pushed and fetched explicitly.

```
git notes add -m 'Deployed to prod 2026-10-12' <sha>
```

```
git log --show-notes
```

```
git push origin refs/notes/*
```

```
git fetch origin refs/notes/*:refs/notes/*
```

Interview tip: Notes are not pushed or fetched by default, which is why teams often forget them. Configure refspecs.

📚 Official docs

- Git documentation

certs.eknathalabs.com

Q169 What is git sparse-index and how do cone-mode sparse checkouts reduce index size?

Advanced & internals

🎙 **Spoken answer · ~59 s**

In a monorepo, an engineer rarely needs every directory. Sparse checkout lets me materialise only selected folders in the working tree, while the repository still contains everything. The cone mode restricts patterns to whole directories, which allows fast matching, and it always includes files at the repository root. The index is a file listing every tracked path, so with a million files even a sparse working tree makes index operations slow. The sparse index feature collapses excluded directories into single placeholder entries, so the index scales with what I use, not with the whole repository, which speeds up status, add and checkout. Combined with a partial clone, I get small downloads and a small working tree. Tools that expand the index, like some IDE integrations or full-tree commands, may undo the benefit, so I check compatibility and document the recommended setup.

Likely follow-ups

- What is cone mode?
- What can expand a sparse index?

Cone-mode sparse checkout restricts the working tree to chosen directories efficiently. The sparse index additionally collapses excluded directories into single entries, shrinking the index and speeding commands.

```
git sparse-checkout init --cone
```

```
git sparse-checkout set services/api libs/common
```

```
git sparse-checkout list
```

```
git sparse-checkout reapply
```

Interview tip: Cone mode uses directory patterns, which are much faster than arbitrary gitignore-style patterns.

📚 **Official docs**

- Git documentation

certs.eknathalabs.com

Q170 How would you design a repository migration from SVN or Perforce to Git, preserving history and minimising disruption?

Advanced & internals

🎙 **Spoken answer · ~61 s**

I begin with discovery: repository size, history length, binary assets, branch and tag layout, users and integrations such as CI and issue links. Then I decide how much history to keep, since full conversion of decades of history may be unnecessary, while archiving the old system read-only. I create an authors mapping from usernames to names and emails. A conversion tool such as git svn or a dedicated converter produces a Git repository, followed by cleanup: convert svn branches and tags into Git refs, move large binaries to LFS, remove junk, and possibly split into multiple repositories if the original was a monolith of unrelated projects. I validate by comparing file trees at selected revisions with the source. A trial migration rehearses timing. Cutover includes a freeze, final sync, updating CI, hooks and permissions, and training with cheat sheets, followed by support during the first weeks.

Likely follow-ups

- How much history do you convert?
- How do you validate the conversion?

Plan scope and history depth, map authors, convert with git svn or a migration tool, split or trim large files, validate against the source, run both systems briefly, then cut over with training and updated CI.

```
git svn clone --stdlayout --authors-file=authors.txt URL repo
```

```
git filter-repo --strip-blobs-bigger-than 10M
```

```
git lfs migrate import --include='*.psd'
```

```
git count-objects -vH
```

Interview tip: Often the right choice is migrating recent history only, keeping the old system read-only as an archive.

📚 **Official docs**

- Git documentation

certs.eknathalabs.com

Q171 Your CI pipeline reports that main is broken after a merge that passed on the PR branch. How do you diagnose and fix it?

Recovery & undo

🎙 Spoken answer · ~64 s

A change can pass CI on its branch yet break main when combined with another change merged in the meantime, a semantic conflict that Git does not detect as a textual one. My first priority is to get main green again for everyone: identify the merge that introduced the failure by looking at the first-parent history and the last green build, and revert it if the fix is not obvious and quick, since a fast revert unblocks the team. Then I investigate calmly, using bisect with an automated test script when several merges are candidates, to find the exact interaction, and prepare a corrected change. To prevent recurrence, I enable a merge queue, which tests each pull request on top of the latest main plus the queued changes before merging, or require branches to be up to date before merging. I also record the incident and improve the tests that failed to catch it.

Likely follow-ups

- Why revert before investigating?
- How does a merge queue prevent this?

Two PRs each passed alone but conflict semantically on main. Revert or fix forward quickly, find the culprit with the merge history or bisect, and prevent recurrence with a merge queue or up-to-date branch requirement.

```
git log --first-parent main --oneline -10
```

```
git revert -m 1 <merge-sha>
```

```
git bisect start HEAD <last-green>
```

```
git bisect run ./ci/test.sh
```

Interview tip: Restore green first (revert), investigate second. A broken main blocks everyone.

📚 Official docs

- Git documentation

certs.eknathalabs.com

Q172 A developer's laptop was lost with unpushed work and the repo contains secrets in config. What do you do?

Recovery & undo

🎙 **Spoken answer · ~59 s**

The immediate concern is security, not the lost commits. I ask the owner to report it, then revoke the SSH keys and personal tokens registered to their account, end active sessions, and rotate any secrets that were stored on the device, such as cloud credentials and environment files. I review the audit log for unusual activity from their account after the loss, and confirm whether disk encryption and a lock screen were enabled, which determines how likely it is that data can be read. Then I deal with lost work: check whether any of it exists on remote branches, in the CI system or in editor backups, and help the developer reconstruct the rest. Going forward I encourage pushing work-in-progress to personal remote branches daily, enforce full-disk encryption through device management, use short-lived credentials, and avoid keeping secrets in local files.

Likely follow-ups

- What gets revoked first?
- How do you reduce the impact of lost devices?

Treat credentials as compromised: revoke tokens and SSH keys, rotate secrets, review audit logs for use. Assess which work was lost, and put in place habits like pushing WIP branches and using disk encryption.

```
gh api user/keys
```

```
gh auth token --revoke
```

```
gh api orgs/acme/audit-log -f phrase='actor:alice'
```

```
git push origin HEAD:wip/alice-$(date +%F)
```

Interview tip: Full-disk encryption makes a lost device an inconvenience rather than a breach, so verify it is mandatory.

📚 **Start here (official docs)**

- Git documentation

certs.eknathalabs.com

Q173 Two teams keep overwriting each other's changes in a shared config file. How do you fix the workflow, not just the conflicts?

Workflows

🎙 Spoken answer · ~58 s

Repeated conflicts and overwrites show that a file serves too many owners. I start with data: how many commits touched it recently, who changed it, and what kinds of edits. Then I address the structure. If different teams manage different sections, I split the file into per-team or per-service files that are combined at build or deploy time, so edits rarely collide. If it is generated information, I generate it from a single source of truth. CODEOWNERS makes ownership explicit, requiring each team to approve changes to its own part. Process changes help too: shorter-lived branches, rebasing before merge, and a merge queue, so conflicts surface early in CI rather than after merge. If overwrites came from careless conflict resolution, I add review for the config and automated validation, such as a schema check, that fails when keys disappear unexpectedly.

Likely follow-ups

- Why is this a design smell?
- How would a schema check help?

Find why they edit the same file: split it into per-team files, add CODEOWNERS, generate it from sources, or move to an API or schema. Short-lived branches and a merge queue reduce collisions.

```
git log --format='%an' -- config/app.yaml | sort | uniq -c
```

```
git log --since=1.month --oneline -- config/app.yaml | wc -l
```

```
git blame -w config/app.yaml | head -30
```

Interview tip: Frequent conflicts in one file are a design smell: the file has too many reasons to change.

📚 Start here (official docs)

- Git documentation

certs.eknathalabs.com

Q174 A release tag was pushed to the wrong commit and the artifact was already published. How do you handle it?

CI/CD & GitOps

🎙 **Spoken answer · ~64 s**

Once an artifact built from a tag is published and consumers may have pulled it, the tag becomes part of the record. Deleting and re-pushing it to a different commit means the same version number would refer to two different contents, breaking checksums, caches and trust. So I compare what the tag points to with what should have been released, decide the impact, and release a new corrected version number from the right commit. The wrong version is marked as deprecated or yanked in the package registry, with notes explaining the problem, and consumers are informed through release notes and, if serious, an advisory. Internally, I enable protected tag rules so existing tags cannot be updated or deleted, and make releases go through an automated pipeline that tags only after verifying that the commit is on the release branch and passes checks. I then run a review to find how the mistake happened.

Likely follow-ups

- Why not move the tag?
- What does yanking a release do?

Do not silently move the tag. Publish a corrected new version, deprecate or yank the bad one, document, and protect tags from moves. Moving a published tag breaks reproducibility and caches.

```
git tag -v v1.8.0
```

```
git rev-parse v1.8.0^{commit}
```

```
git tag -a v1.8.1 <correct-sha> -m 'Fix wrong release commit'
```

```
git push origin v1.8.1
```

Interview tip: Treat published versions as immutable. Fix by releasing forward, not by rewriting tags.

📚 **Official docs**

- Git documentation

certs.eknathalabs.com

Q175 A contributor's pull request contains an obfuscated change in a build script. How do you review it and protect CI?

Security

🎙 **Spoken answer · ~60 s**

Obfuscated code in a build script is a red flag, because legitimate changes rarely need to hide what they do. I stop CI from running it with privileges: for pull requests from forks, secrets should not be exposed and workflows should require maintainer approval before running. I read the diff closely, with particular attention to scripts, workflow files, package install hooks and anything that downloads or evaluates remote content. To understand the obfuscated part, I decode it in an isolated sandbox with no credentials or network access to internal systems, never on my workstation. I ask the author to explain the reason and to replace it with readable code, and involve the security team if the explanation is unsatisfactory or the account looks compromised. Defensive controls include CODEOWNERS on pipeline files, pinning actions to commit hashes, least-privilege tokens, and scanning for suspicious patterns.

Likely follow-ups

- Why are workflow files high risk?
- Why do fork PRs not get secrets?

Do not run it with secrets. Review the diff carefully, decode the obfuscation in a sandbox, ask the author for justification, run CI without secrets for forks, and require maintainer approval for workflows.

```
gh pr diff 731 | less
```

```
git diff main...pr-731 -- scripts/ .github/
```

```
echo 'ZWNobyBoaQ==' | base64 -d
```

```
gh pr checkout 731 --detach
```

Interview tip: Workflow files and build scripts are the highest-risk parts of a PR, since they execute with CI privileges.

📚 **Start here (official docs)**

- Git documentation

certs.eknathalabs.com

Q176 You accidentally committed to the wrong branch locally. How do you move the commit to the correct branch cleanly? Fundamentals

🎙 Spoken answer · ~63 s

This is common and easy to fix if I have not pushed. The safest first step is to keep the commit reachable: either create a new branch at my current position, which captures it, or note the hash. If the right branch already exists, I switch to it and cherry-pick the commit. If the right branch does not exist yet, the new branch I just made is already it. Then I go back to the wrong branch and move it back to where it should be, with a hard reset to the previous commit or to its remote counterpart. Because the commit lives on the other branch, nothing is lost, and the reflog is also a backup. If I had already pushed the mistake to a shared branch, I would not rewrite it, and use revert instead and then re-apply the change on the correct branch through a normal pull request.

Likely follow-ups

- What if you already pushed?
- Why create the branch first?

Create or switch to the right branch, cherry-pick the commit there, then reset the wrong branch back to its previous commit. If unpushed, this is entirely local and safe.

```
git switch correct-branch
```

```
git cherry-pick <sha>
```

```
git switch wrong-branch && git reset --hard HEAD~1
```

```
git branch new-branch && git reset --hard origin/main
```

Interview tip: Creating a new branch at the current commit first is the simplest way to keep the work safe before resetting.

📚 Official docs

- Git documentation

certs.eknathalabs.com

Q177 A new engineer asks why their git pull created a messy merge commit every time. How do you explain and fix their setup?

Collaboration

🎙 Spoken answer · ~61 s

A pull is a fetch followed by an integration step, and by default the integration is a merge. When my local branch has commits that the remote does not, and the remote also has new commits, Git creates a merge commit just to join them, which clutters history with commits like merge branch main into main. I explain the model and offer settings. Setting pull to rebase replays my local commits on top of the updated remote branch, giving linear history, which many teams prefer for feature branches. The fast-forward only setting refuses to create merge commits and tells the developer to decide consciously. I recommend the habit of fetching first and looking at what changed before integrating. I also explain that rebasing is for local, unpushed commits, and shared branches should not be rewritten, and enable autostash so uncommitted changes do not block the rebase.

Likely follow-ups

- When is rebase on pull a bad idea?
- What does ff-only do?

Pulling merges by default, creating a merge commit when branches diverged. Set pull.rebase or pull.ff only, fetch and inspect first, and explain when each is appropriate.

```
git config --global pull.rebase true
```

```
git config --global pull.ff only
```

```
git fetch && git log HEAD..origin/main --oneline
```

```
git config --global rebase.autoStash true
```

Interview tip: Fetch first, look, then decide. Pull hides that decision inside a single command.

📚 Official docs

- git-merge
- Git documentation

certs.eknathalabs.com